

Instituto Politécnico Nacional
Escuela Superior de Cómputo



Trabajo Terminal: 2026-A114

Simulación de sistemas complejos mediante autómatas celulares neuronales

---

Presenta:

Angeles Lopez Erick Jesse

eangelesl1700@alumno.ipn.mx

Director:

Dr. Genaro Juárez Martínez

Ciudad de México, 7 de septiembre de 2026

Índice general

1. Introducción **1**

1.1. Resumen 1

1.2. Planteamiento del problema 1

1.2.1. Preguntas de investigación 3

1.3. Objetivo 3

1.3.1. Objetivos específicos 4

1.4. Hipótesis 4

1.5. Justificación 4

1.6. Alcances y limitaciones 6

2. Marco teórico: Sistemas, caos y simulación **7**

2.1. Sistemas 8

2.1.1. Clasificación de sistemas 9

2.1.1.1. Sistemas abiertos y cerrados 9

2.1.1.2. Sistemas deterministas y estocásticos 10

2.1.1.3. Sistemas estáticos y dinámicos 10

2.2. Teoría del caos 12

2.2.1. Atractores 12

2.2.1.1. Tipos de atractores 13
2.3. Sistemas complejos 14
2.4. Modelado de sistemas complejos 16

3. Marco teórico: Autómatas celulares 19
3.1. Autómata celular 20
3.2. Vecindad 21
3.3. Estados 21
3.4. Condiciones de frontera 24
3.5. Regla local 26
3.6. Ejemplos 28
3.6.1. John Von Neumann 28
3.6.2. El juego de la vida 29
3.6.3. Autómata celular elemental 33
3.7. Clasificación 34
3.8. Extensiones y cambios de la definición 39

4. Marco teórico: Inteligencia artificial y redes neuronales 41
4.1. Inteligencia artificial 42
4.2. Neurona biológica 44
4.3. Neurona artificial 44
4.4. Redes neuronales 46
4.4.1. Funciones de activación 47
4.4.1.1. Binary step (escalón binario) 48
4.4.1.2. Lineal 48

4.4.1.3. Sigmoide 49
4.4.1.4. tanh (hiperbólica) 50
4.4.1.5. ReLU (Rectified Linear Unit) 50
4.4.2. Aprendizaje en redes neuronales 51
4.4.2.1. Función de pérdida 51
4.4.2.2. Optimización 52
4.4.2.3. Entrenamiento 53
4.5. Tipos de redes neuronales 54
4.5.1. Redes neuronales convolucionales 55
4.5.2. Redes neuronales recurrentes 56

5. Marco teórico: Autómatas celulares neuronales 58
5.1. Estado del arte, aplicaciones y líneas actuales de investigación . . . 61
5.1.1. Morfogénesis 62
5.1.1.1. Autómatas celulares neuronales isotrópicos 63
5.1.1.2. Autoclasificación 64
5.1.1.3. Representaciones tridimensionales 68
5.1.2. Autómata celular neuronal en malla 70
5.1.3. Modelos generativos 72
5.1.4. Representaciones y aplicaciones adicionales 75

6. Marco metodológico 78
6.1. Diseño de la investigación 78
6.1.1. Diseño experimental 78
6.1.1.1. Diseño cuasiexperimental 79

6.2. Enfoque de la investigación 79
6.3. Población 80
6.4. Muestra y muestreo 80
6.5. Tipo de investigación 81
6.6. Técnicas e instrumentos de recolección de datos 81
6.6.1. Técnicas de recolección de datos 81
6.6.2. Instrumentos de recolección de datos 82
6.7. Procedimiento 82
6.8. Técnicas y herramientas de procesamiento de datos 83
6.8.1. Procesamiento de datos cuantitativos 83
6.9. Contextualización de la tecnología 84
6.9.1. Fundamentos y contexto de las tecnologías 84
6.9.1.1. C++ 85
6.9.1.2. CUDA 85
6.9.1.3. Python 86
6.9.1.4. Wolfram Mathematica 87
6.9.2. Entornos de trabajo 88
6.9.2.1. Plataformas de desarrollo 88

7. Análisis de software **91**

7.1. Metodología de desarrollo 91
7.2. Reglas de negocio 93
7.3. Requerimientos 93
7.3.1. Requerimientos funcionales 93
7.3.2. Requerimientos no funcionales 94

7.3.3. Requerimientos de sistema 94
7.3.3.1. Recomendado 94
7.3.3.2. Mínimo 95
7.4. Análisis de riesgos 95
7.4.1. Identificación de riesgos 95
7.4.1.1. Riesgos técnicos 95
7.4.1.2. Riesgos operativos 96
7.4.1.3. Riesgos de requerimientos 96
7.4.1.4. Riesgos de estimación 96
7.4.1.5. Riesgos de alcance 96
7.4.2. Probabilidad e impacto 97
7.4.3. Plan de mitigación 99
7.4.4. Plan de contingencia 100
7.5. Estudio de factibilidad 102
7.6. Casos de uso 103
7.7. Diccionario 104
7.7.1. Interfaces gráficas 104
7.7.2. Entradas 104
7.8. Especificaciones de casos de uso 106
7.8.1. Controles de simulación 106
7.8.2. Configuración del autómata celular 117
7.8.3. Interacción con el espacio celular 124
7.8.4. Entrenamiento 125
7.9. Diagramas de secuencia 131

8. Diseño de software **136**

8.1. Arquitectura 136

8.1.1. Vista 136

8.1.2. Modelo 137

8.1.3. Controlador 137

8.2. Diagrama de clases 138

8.3. Diseño de Datos 140

8.3.1. Espacio Celular 141

8.3.2. Vecindad 141

8.3.3. Regla de Transición (CA Clásico) 142

8.3.4. Configuración del Autómata Celular (CA) 142

8.3.5. Configuración del Autómata Celular Neuronal (NCA) 143

8.3.6. Historial de Evoluciones 143

8.4. Diseño de Algoritmos 144

8.4.1. Indexación de la regla semitotalística 144

8.4.2. Paso de evolución del autómata celular clásico 145

8.4.3. Manejo de condiciones de frontera 146

8.4.4. Forward pass del autómata celular neuronal 148

8.4.5. Entrenamiento del autómata celular neuronal 150

8.4.6. Prefetch asíncrono con invalidación por generación 152

8.5. Interfaces de usuarios 153

9. Implementación **157**

9.1. Entorno de desarrollo 157

9.1.1. core 158

9.1.2. GUI 159
9.1.3. training 159
9.2. Estructura del proyecto 159
9.2.1. Directorio core 160
9.2.2. Directorio GUI 160
9.2.3. Directorio training 161
9.2.4. Flujo de artefactos entre directorios 162
9.3. Entrenamiento del modelo 163
9.3.1. Preparación del conjunto de datos 163
9.3.2. Construcción dinámica del modelo 163
9.3.3. Ciclo de optimización 165
9.3.4. Persistencia de los pesos 165
9.3.5. Integración con la interfaz gráfica 166
9.4. Validación y pruebas 166
9.4.1. Validación funcional del autómata celular clásico 166
9.4.2. Validación del autómata celular neuronal 167
9.4.3. Pruebas de rendimiento 167
9.4.3.1. Velocidad de renderizado 168
9.4.4. Pruebas de interfaz 168
9.4.5. Robustez y manejo de errores 169
9.4.6. Cambios 169

10.Experimentación y resultados **170**

10.1. Duplicación de reglas del juego de la vida 170

Referencias **182**

Capítulo 1

Introducción

1.1. Resumen

En el presente documento se presenta la primera etapa de una investigación orientada a la simulación de sistemas complejos en espacios bidimensionales mediante autómatas celulares neuronales. El objetivo principal es utilizar una red neuronal para definir la función de transición de un autómata celular, con el fin de modelar y simular las representaciones de sistemas complejos sobre una red regular bidimensional.

Los resultados incluyen el desarrollo del marco teórico, que recopila el conjunto de definiciones formales necesarias; el estado del arte, donde se analizan investigaciones previas relacionadas con diferentes enfoques y configuraciones; la descripción del marco metodológico, en donde se especifican las técnicas, herramientas y procedimientos utilizados; y los avances obtenidos hasta el momento, que abarcan la implementación y documentación de herramientas para la simulación de autómatas celulares en dos dimensiones.

1.2. Planteamiento del problema

La simulación de sistemas complejos ha surgido como una herramienta indispensable para la comprensión y el análisis de fenómenos en diversas áreas del conocimiento, desde la física y la biología hasta las ciencias sociales y económicas. Tradicionalmente, los autómatas celulares han servido como un modelo matemáti-

co fundamental para estudiar estos sistemas mediante la representación de procesos dinámicos.

Los autómatas celulares son un modelo matemático definido como una 4-tupla que contiene:

- Un espacio de celdas en un dominio discreto.
- Un conjunto de estados posibles.
- Una vecindad que define las dependencias entre células.
- Una función de transición que determina cómo evolucionan simultáneamente las células [1, 2].

Ejemplos clásicos de autómatas celulares incluyen el Juego de la Vida de John Conway y los 256 autómatas elementales descritos por Stephen Wolfram. Estas herramientas han sido aplicadas en múltiples contextos como la simulación de incendios, modelado de distribuciones poblacionales, simulación de partículas y el análisis de ciclos climáticos [3]. No obstante, una limitación crítica de los modelos convencionales reside en la rigidez de sus funciones de transición; estas deben definirse de manera estática y anticipada, restringiendo la posibilidad de adaptarse a condiciones imprevistas o cambios en el entorno. Esta limitación motiva la búsqueda de nuevas metodologías capaces de incorporar mayor flexibilidad y adaptabilidad en la simulación de sistemas dinámicos.

En este contexto, se busca la integración de redes neuronales en el marco de los autómatas celulares. Las redes neuronales, inspiradas en el funcionamiento del cerebro y su capacidad de aprendizaje, ofrecen la posibilidad de redefinir las reglas de transición de forma dinámica. Esto permite que el autómata celular se ajuste de manera autónoma a partir de la experiencia y el entrenamiento [4].

El presente proyecto de investigación propone superar las limitaciones de los autómatas celulares mediante la integración de redes neuronales en la definición de las reglas de transición, dando origen a los *autómatas celulares neuronales*. En este enfoque, las reglas de actualización dejan de ser fijas y adquieren la capacidad de aprender y adaptarse a partir de los datos, lo que permite que el sistema se ajuste de forma dinámica ante comportamientos complejos y entornos variables [5, 6]. Esta aproximación no solo busca ampliar el campo de aplicación de los autómatas celulares, sino que también busca abrir nuevas oportunidades para modelar y simular fenómenos de formas realistas.

Dadas las infinitas configuraciones de autómatas celulares, este proyecto se limita a un espacio bidimensional finito (con condiciones de frontera) y un conjunto finito y numerable de estados. Adicionalmente, se busca experimentar con diferentes vecindades, incluyendo Moore y Neumann. Aunque el objetivo no es determinar las condiciones para la evolución de las células, sí se busca realizar pruebas bajo diferentes configuraciones que permitan mostrar los resultados sin comprometer su calidad.

Por otro lado, la red neuronal recibe como parámetros los estados de las celdas vecinas y devuelve el nuevo estado de la celda a la cual se aplique la función de transición. Este proceso es el mismo para todas las células.

Es por esto que surge la pregunta central de esta investigación: ¿Cómo los autómatas celulares neuronales pueden simular sistemas complejos en un espacio bidimensional? A partir de este cuestionamiento, surgen las siguientes preguntas de investigación.

1.2.1. Preguntas de investigación

- ¿Qué es un sistema complejo?
- ¿Qué es simular?
- ¿Cómo se simula un sistema complejo?
- ¿Qué es un autómata celular?
- ¿Qué es una red neuronal?
- ¿Qué es un autómata celular neuronal?
- ¿Cómo simular un sistema complejo?

1.3. Objetivo

Desarrollar un sistema de simulación basado en autómatas celulares neuronales bidimensionales que permita analizar la generación, mantenimiento, recuperación y propagación de patrones de información en entornos complejos y dinámicos.

1.3.1. Objetivos específicos

- Diseñar e implementar un simulador de espacios discretos bidimensionales que sirva de plataforma para estudiar los autómatas celulares neuronales.
- Entrenar y evaluar el desempeño de los autómatas celulares neuronales bajo diferentes configuraciones.
- Replicar configuraciones clásicas con comportamientos y soluciones conocidas.
- Analizar y comparar diferentes comportamientos con base en la red neuronal establecida.

1.4. Hipótesis

Si un sistema de simulación está basado en autómatas celulares neuronales bidimensionales, entonces es posible analizar y reproducir la generación, mantenimiento y propagación de patrones de información en entornos complejos y dinámicos.

1.5. Justificación

Los autómatas celulares son modelos lo bastante simples para permitir un análisis teórico riguroso, pero lo suficientemente complejos para exhibir fenómenos emergentes. Definir las reglas que describen comportamientos erráticos o no lineales de forma sincronizada resulta una tarea compleja [**?**].

Por esta razón, este proyecto de investigación nace de la necesidad de superar las limitaciones de los modelos tradicionales de autómatas celulares, que se basan en reglas de transición fijas y ya establecidas. Este enfoque limitado, aunque ha sido útil para entender algunos fenómenos, no siempre refleja la complejidad y la capacidad de adaptación de los sistemas dinámicos que encontramos en la naturaleza, en la sociedad y en la tecnología.

Cada autómata celular resuelve un problema particular; sin embargo, su definición matemática implica la existencia de una infinidad de posibles configuraciones y comportamientos. Por ello, llevar a cabo ajustes manuales hasta obtener el resultado deseado se convierte en un proceso computacionalmente inviable [7].

Se sugiere usar redes neuronales para definir las reglas de transición, con el fin de crear un modelo de autómata celular neuronal que pueda aprender y adaptarse por sí mismo a los cambios en su entorno. Esto no solo amplía el alcance y la aplicabilidad de los autómatas celulares, sino que también abre un novedoso campo de estudio en el que se combinan herramientas de inteligencia artificial y simulación de sistemas complejos.

La principal innovación de este enfoque reside en reemplazar las funciones de transición estáticas por modelos ajustables. Usando técnicas de aprendizaje automático, se puede entrenar la función de transición con datos representativos. Esto permite que el sistema se ajuste por sí solo a los cambios en el entorno [8].

La elección de redes neuronales para modelar la función de transición no es arbitraria. Varios estudios recientes han mostrado que las redes neuronales, sobre todo las convolucionales y recurrentes, son muy efectivas para aproximar funciones no lineales y entender relaciones complejas en el espacio y el tiempo [9, 10]. Estas características las hacen particularmente adecuadas para emular las dinámicas locales y temporales propias de los autómatas celulares. Este proyecto se basa en una línea de investigación establecida que intenta unir la capacidad expresiva de las redes neuronales con la estructura organizada y distribuida de los autómatas celulares. Así, se busca crear un modelo más flexible y que se pueda aplicar en diferentes situaciones.

Adicionalmente, este proyecto profundiza en técnicas de simulación, modelado matemático y redes neuronales, integrando conocimientos de ciencias de la computación, vida artificial y matemáticas.

La experiencia adquirida durante el desarrollo e implementación de estos modelos abrirá puertas a futuras investigaciones y proyectos en el campo de la inteligencia artificial. Además, la creación de un entorno de código abierto contribuirá al avance del estado del arte y facilitará que otros investigadores puedan extender y aplicar estos conceptos.

En resumen, este proyecto tiene como objetivo ayudar en el desarrollo de la simulación de sistemas complejos. Para lograrlo, se creará un modelo de autómatas celulares neuronales que sea flexible, adaptable y que se pueda ampliar. Esto unirá la teoría de sistemas dinámicos con las técnicas actuales de aprendizaje automático.

1.6. Alcances y limitaciones

Alcances

- El estudio se limita a la simulación en espacios bidimensionales finitos con condiciones de frontera establecidas en una red regular definida.
- Se emplean redes neuronales supervisadas para la definición de la función de transición.
- Se analizarán comportamientos de patrones conocidos y emergentes bajo distintas configuraciones de estados y vecindades.

Limitaciones

- No se contempla la generalización del modelo a diferentes dimensionalidades o topologías de red.
- El análisis se centra en la simulación de sistemas complejos, sin abordar la optimización computacional del entrenamiento.
- Los resultados se interpretan únicamente dentro del contexto de los parámetros y condiciones experimentales definidos.

Capítulo 2

Marco teórico: Sistemas, caos y simulación

Estudiar fenómenos naturales y artificiales requiere el diseño de estructuras que permitan simplificar la realidad sin perder su esencia. En el presente capítulo se establecen los fundamentos teóricos para comprender la simulación computacional de sistemas, abarcando desde la Teoría General de Sistemas hasta la caracterización de los sistemas complejos.

En primer lugar, se explora la Teoría General de Sistemas como un enfoque interdisciplinario utilizado para comprender la complejidad en diferentes contextos. Esta perspectiva se apoya en dos pilares fundamentales: la concepción de que cualquier fenómeno puede ser entendido como un sistema y la premisa de que las conexiones entre las partes son esenciales para su funcionamiento.

Luego, se presenta la teoría del caos y el concepto de atractores, elementos que permiten diferenciar y analizar sistemas deterministas que presentan comportamientos aparentemente aleatorios.

Finalmente, dado que resulta impráctico experimentar directamente con sistemas complejos en la naturaleza, se justifica el uso del modelado y la simulación computacional como herramientas fundamentales para la experimentación y la inferencia científica.

2.1. Sistemas

La teoría general de sistemas es una forma ordenada de aproximación o simulación del mundo real con base en un procedimiento científico. Su objetivo es difundir una terminología general que pueda describir las características, funciones y comportamientos de cualquier sistema. No busca dar soluciones a problemas, sino que busca formular, matemáticamente, las teorías y definiciones conceptuales que se aplican a la realidad [11, 12].

De acuerdo con [13, 14, 15] la teoría general de sistemas establece tres principios básicos: (1) Los sistemas no son fenómenos aislados; un sistema siempre pertenece a otro sistema, por lo tanto (2) todos los sistemas son abiertos, y (3) las funciones de un sistema dependen únicamente de su estructura.

Un sistema es un conjunto de partes o "subsistemas" interrelacionados que actúan de forma coordinada, dicho de otro modo:

Definición 1. *Un sistema es la abstracción de un evento que existe en la naturaleza, que tiene un objetivo para el observador y cumple con los axiomas que lo caracterizan.*

Es decir, un sistema es un modelo matemático cuyas funciones están limitadas por su estructura, la cual recibe una entrada, realiza una función y devuelve una salida. Un modelo es cualquier abstracción de un proceso realizado por el observador y un proceso es cualquier fenómeno presente en la naturaleza que tenga un objetivo.

Los sistemas, y en general los modelos, son abstracciones inherentes al proceso original, es decir, un sistema puede referenciar múltiples procesos que tengan ciertas propiedades en común. Por ejemplo, el modelo que examina las fuerzas que interactúan en un objeto en caída libre no depende de especificaciones del objeto como su color o su forma.

El sistema permite probar las respuestas de un proceso a estímulos de interés para el observador. Siguiendo el ejemplo de caída libre, no es necesario soltar diferentes objetos de diferentes alturas para identificar las fuerzas que están interactuando; únicamente es necesario cambiar las entradas del modelo.

De acuerdo con [16], los sistemas se caracterizan principalmente por

- Estar compuestos por elementos internos independientes que pueden organizarse en subsistemas.

- Operar de manera organizada para cumplir un objetivo.
- Tener elementos que se puedan distinguir como parte del sistema o del ambiente.
- Existir en un marco temporal.

Los elementos que constituyen un sistema pueden ser materia, energía o información. Los dos primeros se encuentran regidos por las leyes de conservación. Por otro lado, la información se calcula como la cantidad ya existente más la información entrante y la generada, es decir, la información no se pierde [12].

2.1.1. Clasificación de sistemas

De acuerdo con [12], los sistemas se categorizan según su entitividad, origen y relación con el ambiente. Complementando esta visión, diferentes autores añaden clasificaciones con base en el dominio, evolución, continuidad, origen, etc. A continuación se listan algunas de las clasificaciones que resultarán relevantes para la definición de sistemas complejos.

2.1.1.1. Sistemas abiertos y cerrados

Un sistema es abierto si mantiene un intercambio permanente de elementos con su entorno [12]. Tienen una interacción activa con el mundo que los rodea o con su ambiente. Estas influencias externas pueden ajustar las respuestas del propio sistema y se caracterizan por:

- Adaptabilidad: Pueden ajustar el comportamiento y las estructuras internas para mantener su funcionamiento y equilibrio en respuesta a perturbaciones externas.
- Homeostasis dinámica: Pueden mantener un equilibrio interno a lo largo del tiempo.
- Entradas y salidas: Sus entradas adquieren información del entorno y las salidas le devuelven una respuesta [17].

Por su parte, los sistemas cerrados se encuentran aislados de su entorno. Tiene límites que actúan como barreras para el intercambio. Estos operan de forma autocontenida y no tienen una interacción con el entorno que los rodea.

Siguiendo a [17], todas las fluctuaciones y cambios dependen únicamente de los elementos que se encuentren dentro del sistema. Estos sistemas tienden a mantener su estructura y equilibrio interno de manera constante; esto se debe principalmente a que:

- La frontera impide interacción significativa con el entorno, por lo que no se ve afectado el funcionamiento interno.
- Conservan la energía y la materia que los contienen, lo que contribuye a la estabilidad de la estructura y el equilibrio.

Con base en los principios básicos de un sistema, es imposible que existan sistemas cerrados porque un sistema siempre pertenece a otro. La clasificación de sistemas según su relación con el ambiente busca disminuir esa interacción con el entorno para propiciar condiciones ideales de aislamiento.

2.1.1.2. Sistemas deterministas y estocásticos

Un sistema determinista es aquel en donde las mismas entradas producirán invariablemente las mismas salidas. No se contempla la existencia del azar, variables aleatorias ni principio de incertidumbre.

Por otro lado, un sistema estocástico es aquel que se rige por el azar. El resultado depende de probabilidades que cambian o pueden variar en función del tiempo [18].

No hay que confundir los sistemas estocásticos con los sistemas caóticos. En los sistemas estocásticos existe una variable aleatoria que impide que las mismas entradas tengan las mismas salidas. Los sistemas caóticos son deterministas, pero establecer el sistema en evoluciones futuras resulta complicado.

2.1.1.3. Sistemas estáticos y dinámicos

Un sistema estático o sin memoria es aquel que permanece en un estado constante o inmutable, es decir, no cambia con el tiempo. Se dice entonces que un sistema es estático si su salida en cualquier instante de tiempo depende solamente de entradas en ese mismo momento y, de acuerdo con [19], estos se caracterizan por:

- Inmutabilidad: No cambia con el tiempo y mantiene su estado constante.
- Estabilidad: Es sencillo predecir y controlar su comportamiento.
- Simplicidad: Usualmente son más sencillos en diseño y operación, lo que facilita su implementación.

Por otro lado, un sistema dinámico o con memoria es aquel que cambia con el tiempo. Estos tienen la capacidad de adaptarse y responder ante diferentes condiciones de entrada. En otras palabras, es aquel cuya salida en un instante de tiempo depende tanto de entradas en otros y en ese mismo momento. Este se caracteriza por:

- Cambio en el tiempo: Se adapta ante diferentes situaciones y entradas.
- Flexibilidad: Responde ante diferentes condiciones y requisitos.
- Complejidad: Son más complejos en diseño y operación debido a su capacidad de cambio.

De acuerdo con [20], no podemos encontrar procesos estáticos en la naturaleza, pero en ocasiones resulta impráctico modelarlos como sistemas dinámicos.

Este es un ejemplo de un sistema estático:

$$y[n] = 2x[n] + 2$$

Se considera estático porque en cualquier instante n, la salida $y[n]$ depende solamente del valor de entrada $x[n]$ en ese mismo momento.

Por otro lado, este es un ejemplo de un sistema dinámico:

$$y[n] = x[n] + x[n-1]$$

Ya que el valor de $y[n]$ en el instante n requiere conocer el valor actual $x[n]$ y el valor previo $x[n-1]$.

La evolución de un sistema se lleva a cabo con base en estados: el sistema comienza en un estado x_0 y la evolución al estado siguiente depende del estado anterior; además, existe una ley que rige su evolución, y, conociendo esta ley, es posible determinar cualquier estado posterior [21].

2.2. Teoría del caos

De acuerdo con [22], el caos trata con sistemas cuyo comportamiento es impredecible a largo plazo. Suele ser asociado a sistemas aleatorios o que no tienen un orden, pero en teoría del caos se refiere a todos aquellos sistemas deterministas cuya predicción en instantes de tiempo futuro se vuelve difícil de calcular.

Las condiciones iniciales suelen contener errores que incrementarán exponencialmente con el tiempo, lo que se conoce como sensibilidad a las condiciones iniciales, haciendo imposible la predicción a largo plazo. Estos se conocen como sistemas caóticos.

Definición 2. *Sea X un espacio métrico y sea $\phi : X \to X$ una aplicación continua. Se dice que un sistema es caótico si cumple las siguientes condiciones:*

- *La aplicación $\phi : X \to X$ es topológicamente transitiva.*
- *El conjunto de puntos de ϕ es denso en el atractor.*
- *La aplicación $\phi : X \to X$ tiene dependencias sensitivas a las condiciones iniciales.*

El hecho de que la aplicación sea topológicamente transitiva implica que el atractor caótico es irreducible, es decir, si iteramos la función ϕ un número determinado de veces (evolucionar el sistema durante un periodo de tiempo suficiente), se puede acceder a cualquier punto del atractor desde cualquier otro.

Si el conjunto de puntos es denso en el atractor, entonces cualquier punto en el espacio está arbitrariamente cerca de una órbita periódica.

Finalmente, la sensibilidad a las condiciones iniciales refleja el hecho de que en un sistema caótico, dos trayectorias que inician relativamente cerca acabarán comportándose de una forma diferente a largo plazo.

2.2.1. Atractores

En un espacio disipado, la región del espacio de fases que atrae a las trayectorias vecinas se conoce como atractor. Es un subconjunto en el espacio de fases, es decir, un conjunto de estados a los que tiende a evolucionar el sistema en función de las condiciones iniciales.

Definición 3. *Sea X el espacio de fases de un sistema dinámico disipado y sea $\phi_t : X \to X$ la transformación que describe el movimiento. Se dice que $A \subset X$ es un atractor si verifica las siguientes condiciones:*

- *$\exists U$ entorno de A, llamado cuenca de atracción de A, tal que $\forall V$ abierto con $A \subset V$, existe un T_0 tal que $\phi_t(U) \subset V, \forall t > T_0$.*
- *$\phi_t(A) = A, \forall t > 0$*
- *$\nexists S \subset A$ subconjunto propio no vacío que satisfaga las dos propiedades anteriores.*

Es decir, un punto que se encuentra en el atractor no puede salir de ahí y existe una órbita alrededor de este en donde, para cualquier tiempo, la trayectoria permanece en la órbita.

2.2.1.1. Tipos de atractores

A continuación, se describen los cuatro tipos de atractores que se encuentran en la figura 2.1.

1. Atractores de punto fijo: Un atractor es de punto fijo si existe un punto fijo tal que todas las trayectorias cercanas convergen a el cuando $t \to \infty$.

   La figura 2.1a muestra el comportamiento de un péndulo con resistencia al viento que responde a la ecuación:

   $$\theta'' + \gamma\theta' + \beta\theta'|\theta'| + \frac{g}{L}\sin(\theta) = 0$$

   La energía se disipa por la fricción γ y por la resistencia del aire β, obligando al sistema a perder velocidad hasta detenerse en el origen.

2. Atractores de ciclo límite: Las trayectorias vecinas se acercan o se alejan del ciclo límite en forma de espiral.

   En la figura 2.1b se muestra el oscilador de Van der Pol, cuya dinámica esta definida por:

   $$x'' - \mu(1 - x^2)x' + x = 0$$

   El sistema aumenta la energía cuando la amplitud es pequeña y la disipa si es grande, por lo que todas las trayectorias convergen a una órbita periódica estable.

3. Atractores cuasiperiódicos: Cuando dos ciclos límites con diferentes trayectorias se superponen, estas se enroscan en las direcciones longitudinales y latitudinales, creando una figura toroidal.

   La Figura 2.1c ilustra un oscilador lineal forzado con una frecuencia externa basada en la razón áurea ϕ:

   $$x'' + x = A\cos(\phi t)$$

   Debido a que ϕ es irracional respecto a la frecuencia del oscilador, el movimiento resultante es una superposición de trayectorias que nunca se repiten exactamente.

4. Atractores extraños: Se dice que un atractor es extraño si tiene dimensión fractal (patrón de autosimilitud a diferentes escalas). Además, un atractor extraño es caótico si las trayectorias del sistema son atraídas hacia él, pero dentro del atractor son inestables. Esto significa que son sensibles a las condiciones iniciales.

   La Figura 2.1d muestra el atractor de Rössler, definido por el siguiente sistema de ecuaciones:

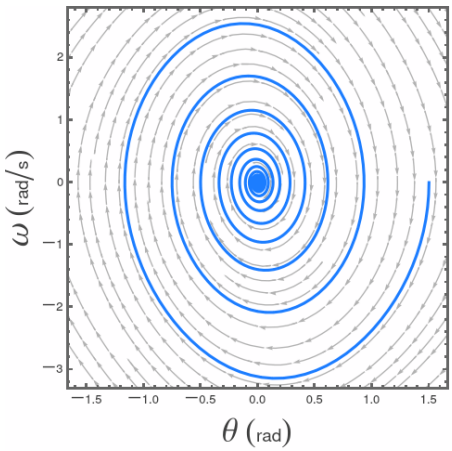
   $$\begin{cases} x' = -y - z \\ y' = x + ay \\ z' = b + z(x - c) \end{cases}$$

   Las trayectorias se atraen hacia una estructura geométrica compleja donde se estiran y se pliegan continuamente, lo que genera un comportamiento impredecible a largo plazo a pesar de ser un sistema determinista.

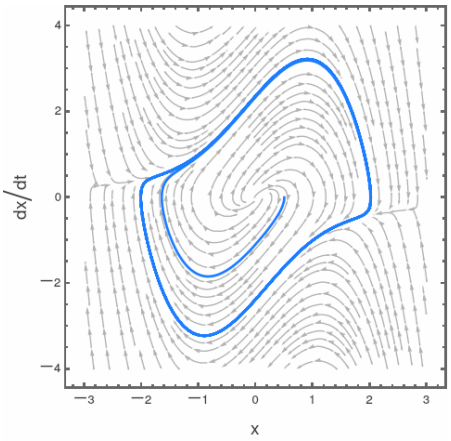
2.3. Sistemas complejos

Los sistemas complejos son sistemas dinámicos con muchas componentes que interactúan entre sí mediante el intercambio de materia, energía o información y que, eventualmente, sufren la acción de agentes externos [23]. Las colonias de hormigas, el mercado bursátil, el clima, el sistema nervioso, las células y los organismos vivos son algunos ejemplos de sistemas complejos. Estos se caracterizan por tener un conjunto de elementos que interactúan entre sí con el fin de alcanzar un objetivo [24, 25].

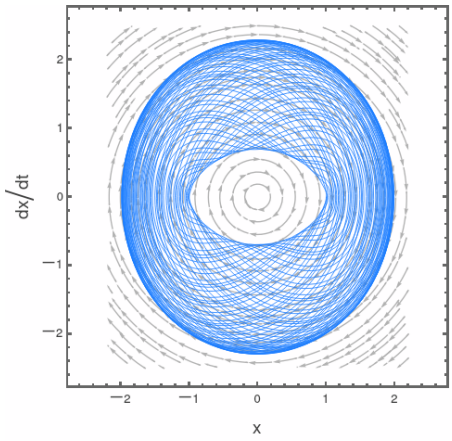
Los sistemas complejos tienen dos implicaciones fundamentales: (1) Existe una influencia mutua entre sus elementos, donde el cambio afecta inevitablemente al



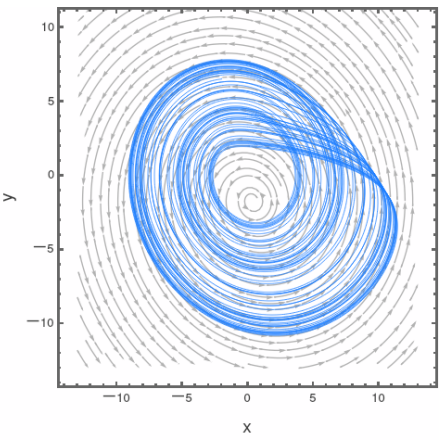
(a) Punto fijo: Convergencia al reposo en un péndulo amortiguado.



(b) Ciclo límite: Oscilador de Van der Pol.



(c) Cuasiperiódico: Trayectoria densa generada por frecuencias inconmensurables.



(d) Extraño: Atractor de Rössler que exhibe una estructura fractal y caos.

Figura 2.1: Representación de los cuatro tipos principales de atractores en el espacio de fases.

resto de elementos, y (2) persiguen un objetivo [25]. Además, de acuerdo con [26], los sistemas complejos están estrechamente relacionados con la teoría del caos, pues la interacción de múltiples elementos locales puede llevar al sistema completo a lo que llama "borde del caos", una región donde el sistema es lo suficientemente estable para mantener su estructura, pero se mantiene flexible para generar nuevos patrones.

Definición 4. *Un sistema complejo es una entidad compuesta de partes interconectadas, de tal forma que el comportamiento colectivo es más que la suma de sus componentes individuales.*

Para entender el comportamiento de un sistema es necesario conocer los elementos y sus interacciones. Es decir, se requiere conocer los estados, valores instantáneos de todos los elementos; y sus transiciones, es decir, los cambios dinámicos en los estados.

Los sistemas complejos se caracterizan principalmente por:

- **No linealidad**: No cumple el principio de superposición, es decir, el resultado no puede expresarse como la suma de las respuestas individuales a cada estímulo por separado. Resulta difícil predecir su evolución dinámica futura [25, 27].
- **Retroalimentación**: Las salidas pasadas influyen en las siguientes salidas. Este comportamiento hace alusión a la memoria del sistema [27].
- **Orden espontáneo**: Existe un orden emergente resultado de las interacciones locales no coordinadas, el cual no puede ser descrito mediante sus interacciones locales [27].

  Aparición (*emergence*): Entidades, patrones, comportamientos y propiedades emergen de la interacción de entidades más pequeñas, las cuales no exhiben estos atributos [27].

  Elementos múltiples: Compuesto de múltiples elementos relativamente idénticos con interacciones locales [25].

2.4. Modelado de sistemas complejos

Resulta evidente que predecir el comportamiento de estos sistemas es una tarea complicada. Una aproximación es el uso de modelos científicos y matemáticos,

los cuales se construyen para desarrollar procesos de inferencia sobre aspectos observados en la vida real. El modelo de inferencia se realiza para entender el sistema real observado.

El proceso, de manera simplificada, se compone de tres fases, como se observa en la figura 2.2:

- Se abstrae el sistema real a un modelo, el cual puede trascender a otros sistemas reales.
- El modelo realiza un proceso de inferencia que busca averiguar las implicaciones derivadas de las premisas.
- Los resultados se utilizan para analizar, interpretar y aplicar a un sistema real. Esta aplicación no requiere ser práctica; es una relación entre las inferencias obtenidas con el modelo y el sistema real.

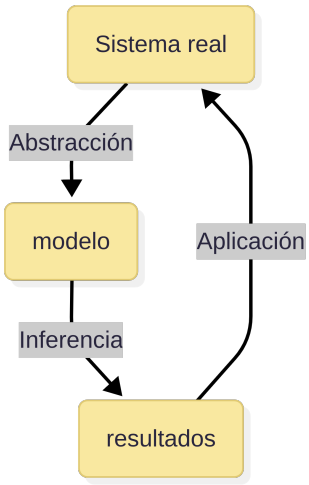


Figura 2.2: Representación gráfica del modelado científico. Este proceso busca relacionar eventos del mundo real utilizando un modelo que realiza una inferencia.

Sin embargo, estos modelos suelen carecer de las herramientas necesarias que les permiten deducir las implicaciones lógicas necesarias. Es aquí donde la computación permite agilizar este proceso realizando simulaciones.

Las computadoras permiten explorar y analizar modelos formales para resolver modelos matemáticos. Además, dado que un modelo computacional puede ser

definido mediante un modelo matemático, el rol de la computación es simplificar este proceso. Dicho esto, [28] define a la simulación de la siguiente manera:

Definición 5. *Simulación es un proceso computacional que permite imitar el comportamiento de un sistema mediante la utilización de un modelo matemático.*

Los resultados de la simulación provienen del uso de reglas algorítmicas. Por eso, es necesario ampliar el proceso de abstracción y crear un modelo formal que será utilizado por la computadora, como se muestra en la figura 2.3.

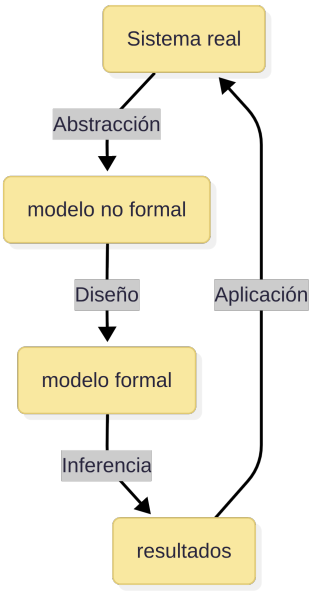


Figura 2.3: Representación gráfica del modelado computacional. Relaciona eventos del mundo real utilizando un modelo computacional que ayuda a agilizar el proceso de inferencia.

Se requiere de un experto en el sistema real que pueda abstraer el problema y un modelador que formalice dicha abstracción para ejecutarla en una computadora que infiera los resultados, mismos que utilizará nuevamente el experto en el sistema real para entablar la relación con el sistema real [29].

Para estudiar sistemas complejos hay varias herramientas de simulación. Una de ellas son los autómatas celulares [23, 26, 30], los cuales siguen la caracterización de los sistemas complejos: elementos idénticos con interacciones locales que pueden generar orden espontáneo, comportamientos emergentes e incluso retroalimentados.

Capítulo 3

Marco teórico: Autómatas celulares

La investigación de los sistemas complejos es un reto actual de la ciencia moderna. Estos sistemas, caracterizados por la interacción entre múltiples componentes locales, son capaces de generar comportamientos emergentes y, a veces, caóticos. Para entender esta complejidad necesitamos herramientas que puedan capturar la esencia de la autoorganización, y aquí los autómatas celulares surgen como un modelo computacional clave para simular fenómenos tanto naturales como artificiales.

En este capítulo se definen los fundamentos matemáticos y estructurales de un autómata celular. Este se entiende como un sistema dinámico discreto, que combina un conjunto de celdas interconectadas y que pueden producir patrones complejos a partir de reglas locales. Se mencionarán sus elementos esenciales, tales como:

- El vecindario: El subconjunto de celdas que afecta el cambio de una unidad.
- El estado de conjunto: Los valores finitos que puede tomar cada celda.
- Las condiciones de frontera son: El comportamiento del sistema en sus límites en el espacio.
- Las funciones de transición son: El algoritmo que rige la evolución del sistema.

Se revisan, asimismo, ejemplos que han sido de una importancia capital para el desarrollo de esta disciplina, tales como el Juego de la Vida de John Conway y los autómatas celulares elementales de Stephen Wolfram.

Finalmente, el capítulo amplía la definición clásica hacia terrenos más avanzados. Se presentan autómatas celulares que operan en espacios no euclidianos y se examinan extensiones recientes que incluyen estados continuos, reglas de probabilidad o actualizaciones asíncronas. Estas variaciones hacen que los AC continúen siendo, décadas después de ser concebidos por Von Neumann, una herramienta de vanguardia en campos tan diversos como la biología teórica, la criptografía y la física de partículas.

3.1. Autómata celular

Los autómatas celulares, concebidos por John von Neumann en 1948 [31], son una colección de células donde cada una de ellas puede adquirir un estado y se agrupan en una red. Además, estas células evolucionan en intervalos y cada una de ellas actualiza su estado en función del estado de sus células vecinas [32, 33, 34].

Siguiendo de manera general a [35, 36, 37], y considerando una red regular de células, un conjunto finito de estados y evoluciones en pasos discretos de tiempo, se define a un autómata celular como:

Definición 6. *Un autómata celular es una 4-tupla $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ donde:*

- $\mathcal{L} \subseteq \mathbb{Z}^D$ *es el espacio celular D-dimensional. El conjunto de células vive en el espacio* $\mathcal{L}$ *y cada célula* $\vec{i}$ *se define como un vector D-dimensional que representa su posición en el espacio.*
- $\mathcal{S}$ *es un conjunto de k estados. Cada célula* $\vec{i}$ *solo puede adquirir un valor de un conjunto finito de diferentes estados:*

$$\sigma_{\vec{i}}(t) \in \mathcal{S} \equiv \{s_0, s_1, s_2, \ldots, s_{k-1}\},\ \forall \vec{i} \in \mathcal{L}$$

  Donde $\vec{i} \in \mathcal{L}$ *es una célula en el espacio celular,* $\sigma_{\vec{i}}(t)$ *es el estado de la i-ésima célula en el tiempo t y* $\mathcal{S}$ *es un anillo conmutativo finito, como un intervalo de números enteros, que generalmente se define por los enteros módulo k (*$\mathbb{Z}_k$*).*
- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_m)$ *es el conjunto de los m vectores de desplazamiento que definen la vecindad de una célula en* $\mathcal{L}$*. Así, los vecinos de las células ubicadas en* $\vec{i} \in \mathcal{L}$ *se encuentran en las posiciones* $\vec{i} + \vec{v}, \forall \vec{v} \in \mathcal{N}$
- $\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$ *es la regla local del autómata:*

$$\sigma_{\vec{i}}(t+1) = \mathcal{R}(\{\sigma_{\vec{i}+\vec{v}_j}\}_{j=1}^{m})$$

donde $\mathcal{R}$ es la función de transición local que asigna el nuevo estado de una célula a partir de los estados de sus m vecinos.

3.2. Vecindad

Como ya se mencionó, la vecindad es el conjunto de vectores de desplazamiento, por lo que está estrechamente relacionado con la dimensionalidad del autómata celular. Por ejemplo, John von Neumann propuso una vecindad para $\mathbb{Z}^2$ de cinco células que actualmente se conoce como vecindad de von Neumann. Esta vecindad considera a la célula y a sus cuatro vecinas ortogonales más cercanas [38], dicho de otro modo:

$$\mathcal{N} = ((0,-1),(-1,0),(0,0),(1,0),(0,1))$$

La cual se puede representar como en la figura 3.1a.

Por otro lado, Moore añade los cuatro vecinos diagonales más cercanos a la vecindad de von Neumann, dando una vecindad de nueve elementos [39]. Esta vecindad, representada por la figura 3.1b, fue utilizada por John Conway en el "Juego de la Vida" [40].

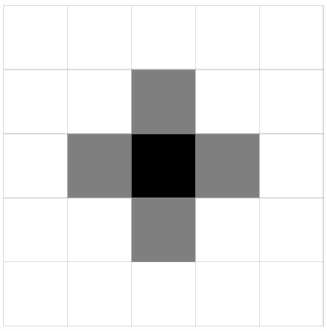
Otro ejemplo se encuentra en la figura 3.1c, la cual representa una vecindad de radio uno en un espacio unidimensional, es decir, todas aquellas células a una distancia menor o igual a uno, incluyéndose a su misma. Fue utilizada por Stephen Wolfram para definir los autómatas celulares elementales [41].

Las vecindades no solo son cuadradas; también pueden tener otras formas, como hexágonos o triángulos, que se muestran en las figuras 3.2a y 3.2b. Además, existen vecindades con comportamiento dinámico, como la de Margolus mostrada en la figura 3.2c, cuya configuración alterna entre dos disposiciones en cada paso evolutivo [42].

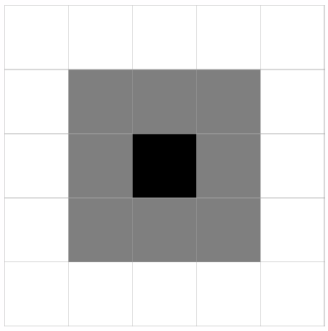
Finalmente, los vecinos no tienen que ser estrictamente adyacentes entre sí ni ser simétricos. Existen vecindades, como en la figura 3.3, donde algunas células no son contiguas a la central.

3.3. Estados

Cada célula adquiere un único estado s en cada paso de tiempo, donde s pertenece a un conjunto de estados $s \in \mathcal{S}$. Por ejemplo, el autómata celular de John von



(a) Vecindad de von Neumann

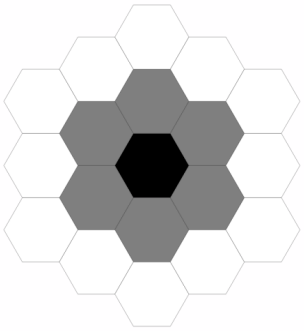


(b) Vecindad de Moore

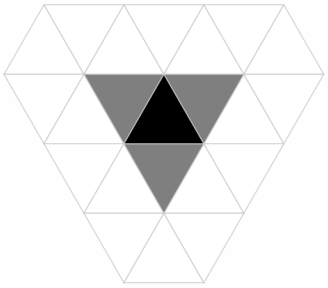


(c) Vecindad de radio uno en $\mathbb{Z}^1$

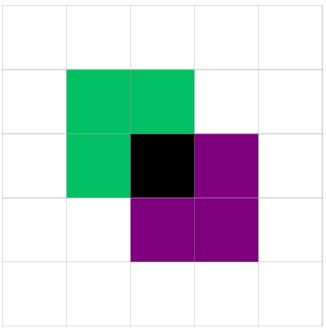
Figura 3.1: Las vecindades 3.1a y 3.1b son para CA bidimensionales; la 3.1c es para CA unidimensionales. La célula negra representa la célula actual y, junto con las grises, conforma su vecindad.



(a) Vecindad hexagonal



(b) Vecindad triangular



(c) Vecindad de Margolus

Figura 3.2: En 3.2a y 3.2b, la célula negra es la actual y las grises su vecindad. En 3.2c, la célula negra es la actual, mientras que las células verdes y moradas representan las vecindades activas en tiempos pares e impares.



Figura 3.3: Vecindad en $\mathbb{Z}^1$ con vecinos de adyacentes.

Neumann contempla 29 estados, aunque algunos autores han propuesto variantes reducidas a cuatro estados, conservando su capacidad de autoreproducirse [35].

Por otro lado, el juego de la vida se compone de dos estados que denominó "vivas" y "muertas" [40]. De igual forma, Stephen Wolfram utiliza dos estados en los autómatas celulares elementales [41].

Idealmente, el espacio celular $\mathcal{L}$ es infinito, lo que permitiría infinitas combinaciones de estados. Sin embargo, esto es inviable computacionalmente. Para esto, se requiere establecer una restricción de finitud, como lo marca [35].

Supóngase un espacio discreto unidimensional infinito. Idealmente, conocemos el estado de las infinitas células, pero computacionalmente no podemos almacenar ni procesar toda esa información. Por lo que se restringe a los valores que únicamente nos interesan.

A todas aquellas células que son de interés se les define con un estado activo, el cual puede ser cualquier $s \in \mathcal{S}$, por otro lado, las que no son de interés se les define con un estado inactivo σ_q:

Definición 7. *Sea $\mathcal{R}$ la regla local de un autómata celular. Un estado $\sigma_q \in \mathcal{S}$ es inactivo si $\mathcal{R}(\sigma_q, \sigma_q, \ldots, \sigma_q) = \sigma_q$.*

Es decir, si una célula y todos sus vecinos están en estado inactivo, la célula permanece inactiva tras aplicar $\mathcal{R}$. Cabe notar que todos los vecinos deben estar en dicho estado, por lo que, si alguno cambia a cualquier estado s, entonces se activa. Este evento ocurre cuando el espacio en donde evoluciona el autómata celular incrementa.

Para trabajar computacionalmente, [35] define las configuraciones finitas de la siguiente manera:

Definición 8. *Sea $\sigma_q \in \mathcal{S}$ un estado inactivo, $\sigma_p \neq \sigma_q \in \mathcal{S}$ un estado no inactivo y sea $\mathcal{L}$ un espacio celular infinito. Se considera que un autómata celular tiene configuración finita si solo un número finito de celdas están en estados activos σ_p.*

Si la configuración inicial es finita, la propiedad de finitud se preserva en cada paso de tiempo, como se observa en la tabla 3.1. Imaginemos un autómata celular en $\mathbb{Z}^1$ con una vecindad de radio 1 y una sola célula activa en la posición n con estado σ_p, mientras que el resto de infinitas células tienen estado inactivo σ_q.

En el tiempo $t+1$ las células en $n-1$ y en $n+1$ tienen un vecino que no es inactivo, por lo que pasan a ser activas por la definición 7. Dado que este

t	$\vec{\sigma}(t)$										
0				$\ldots$	q	p	q	$\ldots$			
1			$\ldots$	q	p	p	p	q	$\ldots$		
2		$\ldots$	q	p	p	p	p	p	q	$\ldots$	
3	$\ldots$	q	p	p	p	p	p	p	p	q	$\ldots$

Tabla 3.1: Evolución temporal de un autómata celular unidimensional con una configuración inicial finita. Donde p representa estados activos y q estados inactivos. Los conjuntos $\vec{\sigma}(t)$ y $\vec{\sigma}(t+n)$ son finitos.

incremento está acotado por el tamaño de la vecindad, la configuración resultante sigue siendo finita. Por inducción, este proceso se repite para cualquier tiempo $t+m$; por lo tanto, mientras el tiempo de la evolución no tienda a infinito ($t \not\to \infty$) el autómata celular mantendrá una configuración finita, garantizando su viabilidad computacional.

Sea $\mathcal{C} = \mathcal{S}^{\mathcal{L}}$ el conjunto de todas las configuraciones. Definimos $\mathcal{C}_F \subset \mathcal{C}$ como el subconjunto de todas las configuraciones finitas. Esta restricción es necesaria para hacer computacionalmente viable la simulación de la evolución.

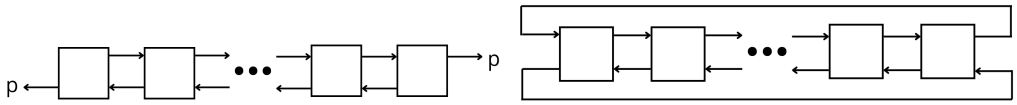
3.4. Condiciones de frontera

Idealmente, los autómatas celulares pueden evolucionar en espacios infinitos; sin embargo, se pueden establecer límites o fronteras, las cuales restringen el espacio celular $\mathcal{L}$ [35, 43].

Definición 9. *Un autómata celular es finito si el espacio celular $\mathcal{L} \subseteq \mathbb{Z}^D$ es finito; en caso contrario, se denomina infinito.*

Un autómata celular finito siempre tiene fronteras en los límites de su espacio. A la forma en como se comporta la frontera se le conoce como condición de frontera, la cual suele ser abierta o periódica [43].

- **Condición de frontera abierta**: Se les asigna un estado fijo a todos aquellos vecinos faltantes en las células de la frontera.
- **Condición de frontera periódica**: Los vecinos faltantes de las células en la frontera son otras células en la frontera. La figura 3.4b muestra cómo se conectan las células de la frontera en un espacio unidimensional.



(a) Condición de frontera abierta (b) Condición de frontera periódica

Figura 3.4: Condiciones de frontera con dos vecinos. La figura 3.4a asigna un estado no inactivo al vecino faltante. La figura 3.4b asigna como vecino el estado de la célula en la frontera opuesta, formando un anillo.

Un autómata infinito que tiene una configuración periódica equivale a un autómata finito con frontera periódica [35].

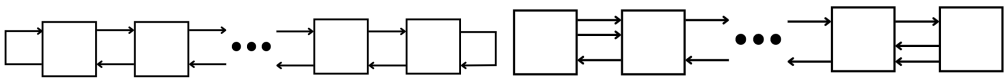
Definición 10. *Una configuración $\vec{\sigma} \in \mathcal{C}$ de un autómata celular D-dimensional se dice periódica si es invariante ante un conjunto de traslaciones linealmente independientes en D direcciones. Es decir:*

$$\vec{\sigma} = T_i^{n_i}(\vec{\sigma}),\ \forall i \in \{1, 2, \ldots, D\}$$

donde $T_i^{n_i}$ denota un desplazamiento de n_i posiciones en la dimensión i.

Definimos entonces a $\mathcal{C}_P \subset \mathcal{C}$ como el conjunto de todas las configuraciones periódicas. Estas configuraciones preservan la condición de frontera periódica en cada traslación [35].

La frontera no solo puede ser abierta o periódica, existen diferentes comportamientos como la reflectiva y la reflexiva. La frontera reflectiva asigna a los vecinos faltantes el mismo estado de la célula en la frontera (figura 3.5a), por lo que son vecinas de si mismas. La frontera reflexiva, en cambio, genera una doble relación de vecindad en los extremos (figura 3.5b), [35, 43].



(a) Condición de frontera reflectiva (b) Condición de frontera reflexiva

Figura 3.5: Condiciones de frontera con dos vecinos. La figura 3.5a muestra que las células en la frontera son vecinas entre sí, mientras que la figura 3.5b indica que las células que están al lado de las células de la frontera pueden tener múltiples relaciones de vecindad.

3.5. Regla local

Sea $\mathcal{S}$ el conjunto de estados y $m = |\mathcal{N}|$ el número de vecinos. La función de transición o regla local se define como

$$\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$$

donde $\mathcal{R}$ asigna un nuevo estado a cada célula en función de los estados de sus m vecinos (incluyéndose a sí misma si aplica).

Para autómatas celulares unidimensionales, dado un rango arbitrario r, la regla puede ser escrita como:

$$\sigma_i(t+1) = \mathcal{R}(\sigma_{i-r}(t), \sigma_{i-r+1}(t), \ldots, \sigma_i(t), \ldots, \sigma_{i+r}(t)),\ \mathcal{R} : S^{2r+1} \to S$$

Donde el radio r especifica el tamaño de la vecindad para cada célula (la vecindad es de tamaño $2r+1$ como se muestra en la figura 3.1c), la cual puede adquirir un valor k.

Un ejemplo de regla local es el juego de la vida [40]. En este caso, el conjunto de estados es binario $\mathcal{S} = \{0, 1\}$, donde 0 representa células muertas y 1 las vivas. La regla de transición realiza el siguiente mapeo:

$$\mathcal{R} : \{0, 1\}^9 \to \{0, 1\}$$

Ya que la vecindad de Moore en 2D involucra 8 vecinos más la propia célula, es decir, $m = 9$.

El juego de la vida sigue las siguientes tres reglas:

- **Nacimiento**: Una célula muerta en el tiempo t nace en el tiempo $t+1$ si tiene exactamente tres vecinos vivos en el tiempo t.
- **Muerte**: Una célula viva en el tiempo t muere en $t+1$ por:
  - Sobrepoblación; si hay más de tres vecinos vivos en t.
  - Soledad: si hay menos de dos vecinos vivos en t.
- **Supervivencia**: Una célula viva en el tiempo t se mantiene viva en $t+1$ si tiene dos o tres vecinos vivos en t.

111	110	101	100	011	010	001	000
↓	↓	↓	↓	↓	↓	↓	↓
$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$

Tabla 3.2: Cada posible combinación de tres células puede adquirir uno de dos valores. Cuando la célula evaluada encaja en este patrón, su valor cambia al establecido por la regla. Este conjunto de 256 reglas se le conoce como autómata celular elemental.

Esta se conoce como regla semitotalística, ya que la transición depende de la suma de los valores en la vecindad, pero distingue entre diferentes posiciones, como el centro y los bordes [44].

Por otro lado, una regla es totalística si la transición depende únicamente de la suma de los estados en todos los vecinos, sin distinguir posiciones [45]. Para un autómata celular unidimensional con $|\mathcal{S}| = 3$ estados y radio $r = 1$, solo hay $(|\mathcal{S}| - 1)|\mathcal{N}| + 1 = (2 - 1) \cdot 3 + 1 = 4$ posibles valores de suma.

Finalmente, una regla local es arbitraria si distingue las posiciones de cada elemento en la vecindad. Cabe mencionar que las reglas arbitrarias contienen todas las posibles combinaciones de reglas, incluyendo las semitotalísticas, las cuales incluyen a su vez las totalísticas:

$$\mathcal{R}_{\text{totalísticas}} \subset \mathcal{R}_{\text{semitotalísticas}} \subset \mathcal{R}_{\text{arbitrarias}}$$

El objetivo de las reglas semitotalísticas y totalísticas será entonces simplificar ciertos comportamientos.

La configuración dada para el autómata celular elemental considera $k = 2$ (es decir, $\sigma_i \in \{0, 1\}$), un radio $r = 1$ ($|\mathcal{N}| = 3$) y la regla $\mathcal{R} : \{0, 1\}^3 \to \{0, 1\}$ que se define como:

$$\phi(\sigma_{i-1}(t), \sigma_i(t), \sigma_{i+1}(t))$$

Esta dinámica está dada por las posibles configuraciones que las tres células adyacentes pueden adquirir en el tiempo t. Podemos encontrar $k^{|\mathcal{N}|} = 2^3 = 8$ patrones posibles en la vecindad, número correspondiente a la combinatoria de dos estados en tres posiciones. Cada vez que encontremos un patrón, podemos asignarle un nuevo estado a $\sigma_i(t + 1)$ ($\{0, 1\}$), por lo que el número total de reglas asciende a $k^{k^{|\mathcal{N}|}} = 2^8 = 256$, como se muestra en la tabla 3.2.

Estas reglas se pueden representar de forma tabular y también como un número decimal. La tabla 3.3 muestra la asociación entre la forma tabular y la regla decimal.

Binario	111	110	101	100	011	010	001	000	
Decimal	7	6	5	4	3	2	1	0	**regla**
Valor	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	1	1
					⋮				⋮
	0	0	0	1	1	1	1	0	30
					⋮				⋮
	0	1	1	0	1	1	1	0	110
					⋮				⋮
	1	1	1	1	1	1	1	1	255

Tabla 3.3: Los valores binarios representan las posibles configuraciones de estados que pueden aparecer en el espacio, cada uno asociado a un valor decimal, y el valor es la potencia asociada a esa combinación. Para convertir la regla tabular en la decimal, solo es necesario convertir el número binario.

Nótese que la regla decimal solo aplica cuando las condiciones son las mismas. Es decir, cuando el número de vecinos $|\mathcal{N}|$ y estados $|\mathcal{S}|$ son invariantes; a cada regla se le puede asociar un entero único, pero dos reglas que difieren en vecindad o estados pueden compartir el número decimal. Por ejemplo, considerando la regla 10 con $|\mathcal{N}| = 2$ y $|\mathcal{S}| = 2$ es 1010, la cual es completamente diferente cuando $|\mathcal{N}| = 3$ y $|S| = 2$, lo que da como resultado 000000101.

3.6. Ejemplos

3.6.1. John Von Neumann

El matemático y físico John von Neumann (1903-1957) fue uno de los pioneros en el estudio de los autómatas celulares, especialmente en lo que atañe a la reproducción automática y la universalidad computacional. En su obra póstuma, *Theory of Self-Reproducing Automata*, presentó un modelo de autómata celular bidimensional diseñado para construir máquinas autorreplicantes.

A continuación se presentan los principales elementos de su autómata celular, adaptados al formato de definición formal de este tipo de autómata:

- **Espacio celular:** Von Neumann definió un espacio bidimensional como una cuadrícula regular de celdas, idealmente infinita.
- **Estados:** En su versión original, cada célula podía adoptar uno de 29 estados posibles que permiten distinguir células en reposo, células en transmisión de "señal", células constructoras, etc.
- **Vecindad:** Utiliza la vecindad que más tarde se denominaría "vecindad de von Neumann" [46].
- **Función de transición:** Todas las células actualizan su estado de forma síncrona, de acuerdo con una regla común que depende del estado actual de la propia célula y de los estados de sus vecinos. En el modelo de von Neumann, esta regla está diseñada de modo que permita que ciertas configuraciones de células funcionen como un constructor universal capaz de autorreplicarse.

Su objetivo era demostrar de manera formal que se puede crear un autómata celular que se reproduzca sin fin y que, además, pueda hacer construcción universal dentro de ese sistema celular [38, 47].

3.6.2. El juego de la vida

El juego de la vida es un autómata celular propuesto por el matemático John Horton Conway en 1970. Consiste en una cuadrícula de celdas que pueden estar en uno de dos estados posibles: vivas o muertas. A lo largo del tiempo, las celdas evolucionan de acuerdo con un conjunto de reglas locales que determinan su supervivencia, muerte o nacimiento, en función del número de vecinos vivos que posean [48].

Siguiendo la definición formal de un autómata celular, el *Juego de la Vida* se caracteriza por los siguientes elementos:

- **Espacio celular**: Se define sobre una cuadrícula bidimensional regular, teóricamente infinita.
- **Estados**: Cada célula puede encontrarse en un estado $\{\text{muerto}, \text{vivo}\}$, representados comúnmente como $\{0, 1\}$.
- **Vecindad**: Se utiliza una vecindad de Moore semitotalística, la cual incluye las ocho celdas adyacentes a la célula central.

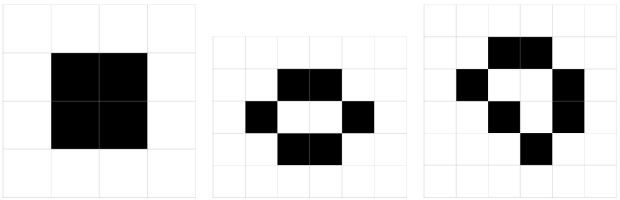
- **Función de transición**: El estado futuro de cada célula depende de su estado actual y del número de vecinos vivos, siguiendo las reglas:
  - Una célula viva sobrevive cuando tiene dos o tres vecinos vivos.
  - Una célula viva con más de tres vecinos vivos muere por sobrepoblación.
  - Una célula viva que tiene menos de dos vecinos vivos muere por soledad.
  - Una célula muerta con exactamente tres vecinos vivos renace.

Estas reglas pueden modificarse para crear variantes del autómata, cambiando las condiciones de nacimiento o supervivencia. En [40] se analizan distintas funciones de transición y configuraciones espaciales.

Una notación común es *Birth/Survival*, que especifica los números de vecinos vivos requeridos para cada caso. El juego de la vida tradicional corresponde a la configuración $B3/S23$: una célula muerta nace con tres vecinos vivos, y una viva sobrevive si tiene dos o tres vecinos vivos [40].

Dentro de este sistema emergen patrones característicos, que se agrupan en tres categorías principales:

- **Naturaleza muerta**: Estructuras estáticas que permanecen invariantes en el tiempo.



(a) *Block* (b) *Beehive* (c) *Loaf*

Figura 3.6: Ejemplos de configuraciones de naturaleza muerta.

- **Osciladores**: Patrones que repiten su configuración después de un número finito de iteraciones, manteniendo su posición en la cuadrícula.

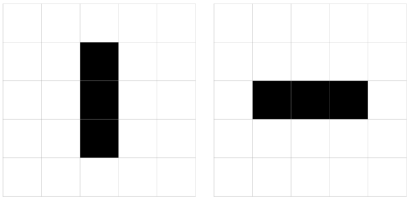


Figura 3.7: *Blinker*.

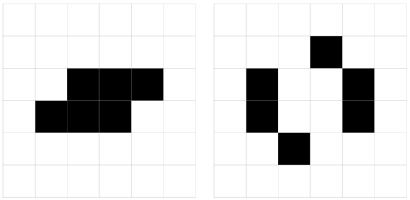


Figura 3.8: *Toad*.

- **Naves espaciales**: Son osciladores que, además de repetir su patrón, se desplazan por el espacio celular.

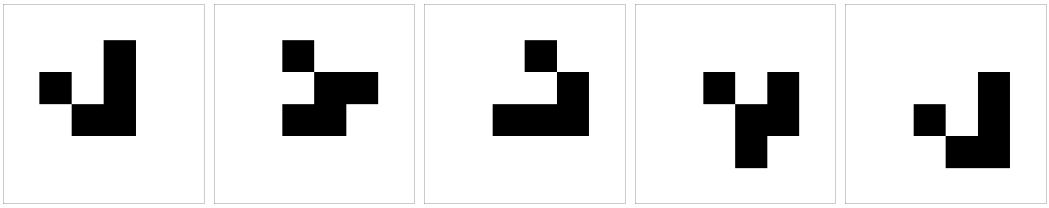


Figura 3.9: *Glider*.

Paul Rendell [49] demostró que, a partir de una configuración inicial apropiada, el *Juego de la Vida* es Turing completo, es decir, tiene la capacidad de realizar cualquier cálculo computable mediante la simulación de una máquina de Turing. La figura 3.10 muestra la configuración inicial del autómata celular diseñado por Paul Rendell.

Debido a su naturaleza semitotalística, la vecindad puede dividirse en dos conjuntos, y con ello determinar el número de posibles estados que puede adoptar cada uno:

Así, puede definirse un vector de 18 elementos que representa la función de transición, como se muestra en la tabla 3.4.

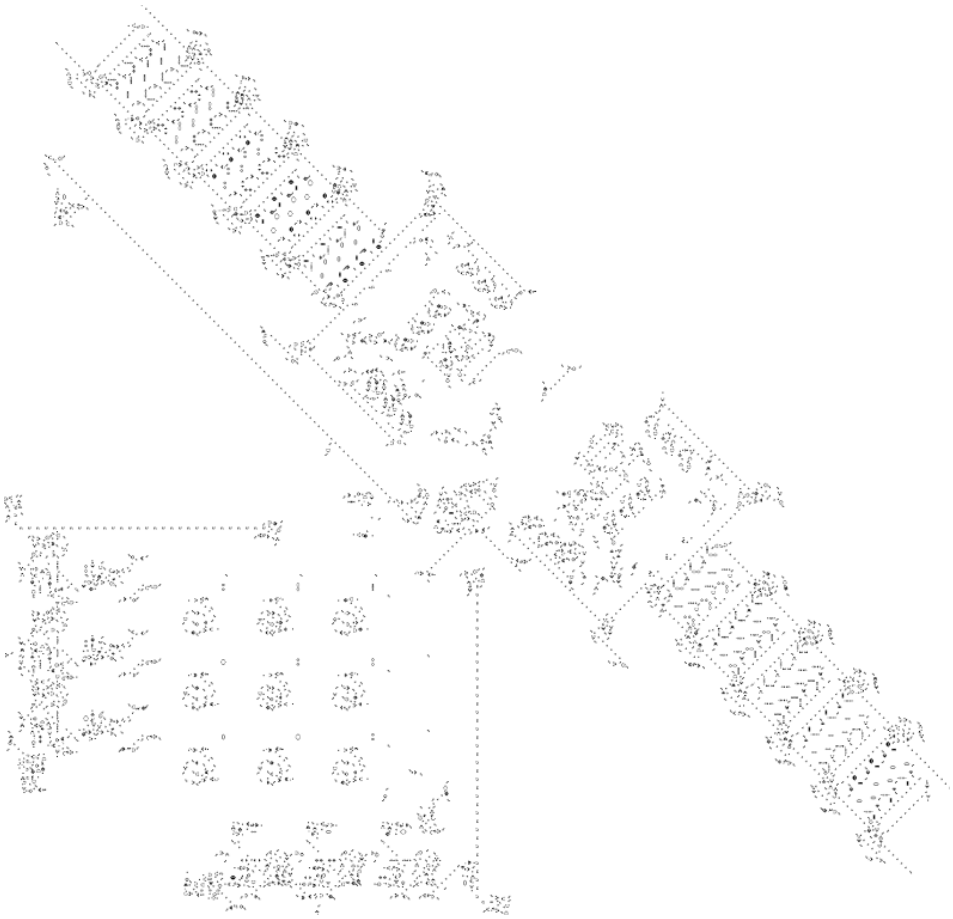


Figura 3.10: Máquina de Turing implementada en el *Juego de la Vida* [50].

Vecindad	**Estados**	**Total**
Frontera	$\{0,1,2,3,4,5,6,7,8\}$	9
Centro	$\{0,1\}$	2

Frontera	8	7	6	5	4	3	2	1	0	8	7	6	5	4	3	2	1	0
Centro	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
$B3/S23$	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	0	0	0

Tabla 3.4: Representación binaria de la regla $B3/S23$.

De manera general, el tamaño del vector que define una regla semitotalística puede calcularse mediante la ecuación 3.1. En este caso, dicho vector tiene tamaño 18, por lo que existen $|\mathcal{S}|^{18} = 2^{18} = 262,144$ posibles reglas.

$$\prod_{i=0}^{n}(|\mathcal{S}| - 1)(|n|) + 1, \quad \forall n \in \mathcal{N} \tag{3.1}$$

3.6.3. Autómata celular elemental

Stephen Wolfram propuso una versión más simple de los autómatas celulares, llamada autómata celular elemental. En esta versión, se disminuye la dimensionalidad del espacio celular y, por lo tanto, se reduce el tamaño de la vecindad. A continuación se detallan sus características según la definición formal de autómata celular:

- **Espacio celular**: Está formado por una cadena unidimensional de celdas dispuestas linealmente.
- **Estados**: Cada célula puede encontrarse en uno de dos estados posibles, $\{0, 1\}$.
- **Vecindad**: Se define mediante un radio r que determina cuántas celdas adyacentes influyen en la evolución de cada célula. En el caso del autómata celular elemental, el radio es $r = 1$, por lo que la vecindad está compuesta por la célula central y un vecino a cada lado.
- **Función de transición**: Wolfram no propuso una única regla de evolución, sino el conjunto completo de funciones de transición posibles bajo estas condiciones.

Dado que cada célula considera tres posiciones (la propia y sus dos vecinas) y cada una puede tener dos posibles estados, existen $2^3 = 8$ combinaciones de vecindad. Por tanto, el número total de reglas posibles es $2^8 = 256$.

Wolfram asignó a cada una de estas funciones un número entero entre 0 y 255, conocido como número de regla, que identifica de forma unívoca el comportamiento del autómata. Además, propuso una clasificación de las reglas en cuatro categorías, según los patrones que emergen durante su evolución temporal (estas clases se detallan en la siguiente sección).

A continuación, se presentan tres reglas representativas por su relevancia teórica y sus propiedades computacionales.

- **Regla 30**: La Regla 30 pertenece a la clase de comportamiento caótico. A partir de una configuración inicial simple, genera patrones altamente complejos y aparentemente aleatorios [51] como se muestra en las figuras 3.11.

  Esta regla plantea tres interrogantes fundamentales sobre la naturaleza del caos y la computabilidad:

  1. ¿La columna central permanece no periódica a lo largo del tiempo?
  2. ¿Existe equilibrio estadístico entre los estados vivos y muertos?
  3. ¿Es posible calcular el estado de la célula central en el paso n en tiempo $O(n)$?

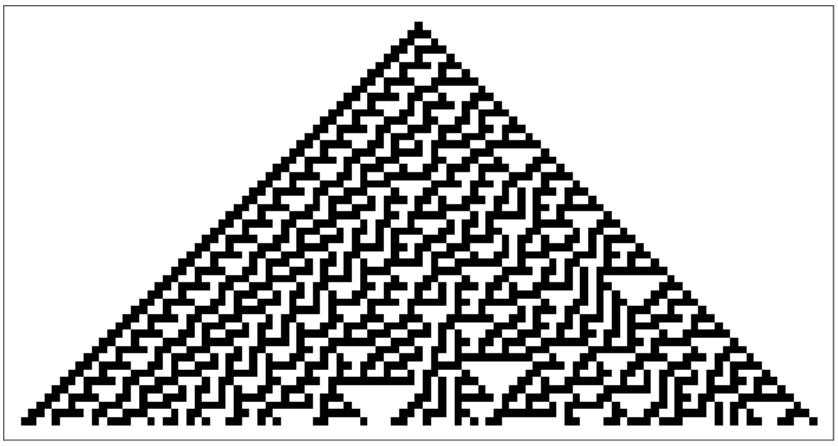
- **Regla 110**: La Regla 110 exhibe un comportamiento mixto: combina regiones periódicas y caóticas que interactúan entre sí de manera compleja. A partir de una única célula activa, genera estructuras móviles y patrones que pueden coexistir en el tiempo [52].

  Lo más notable de esta regla es su universalidad computacional. Wolfram y Matthew Cook demostraron que la Regla 110 es Turing completa, es decir, capaz de simular cualquier máquina de Turing y, por tanto, realizar cualquier cómputo posible en un sistema discreto determinista [52]. Su evolución se muestra en las figuras 3.12.

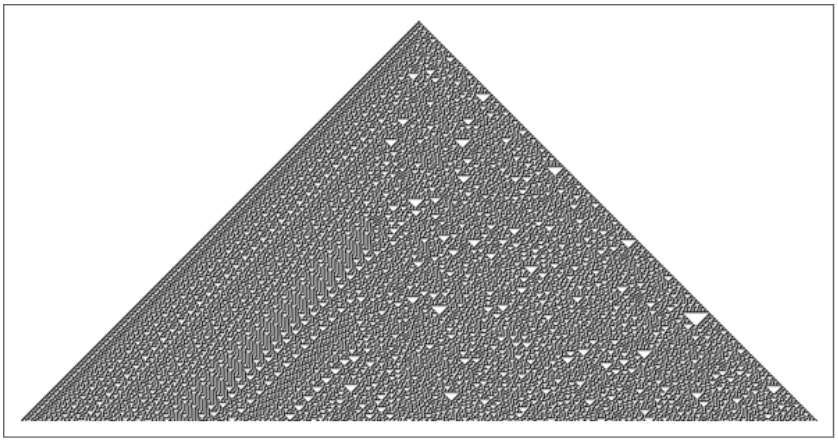
- **Regla 90**: La Regla 90 se distingue por generar estructuras fractales. Iniciando desde una única célula activa, produce el conocido triángulo de Sierpinski como se muestra en la figura 3.13, el cual es un patrón autosimilar que aparece en múltiples sistemas naturales y matemáticos [53].

3.7. Clasificación

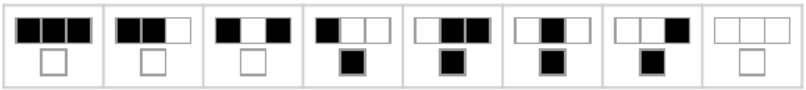
Los autómatas celulares elementales fueron utilizados por diferentes autores por representar la máxima simplicidad y mínima configuración que un autómata puede adquirir y que tenga un amplio rango de comportamientos. Múltiples autores han intentado definir una clasificación; a continuación se muestran algunas clasificaciones propuestas por diferentes autores.



(a) 50 evoluciones.

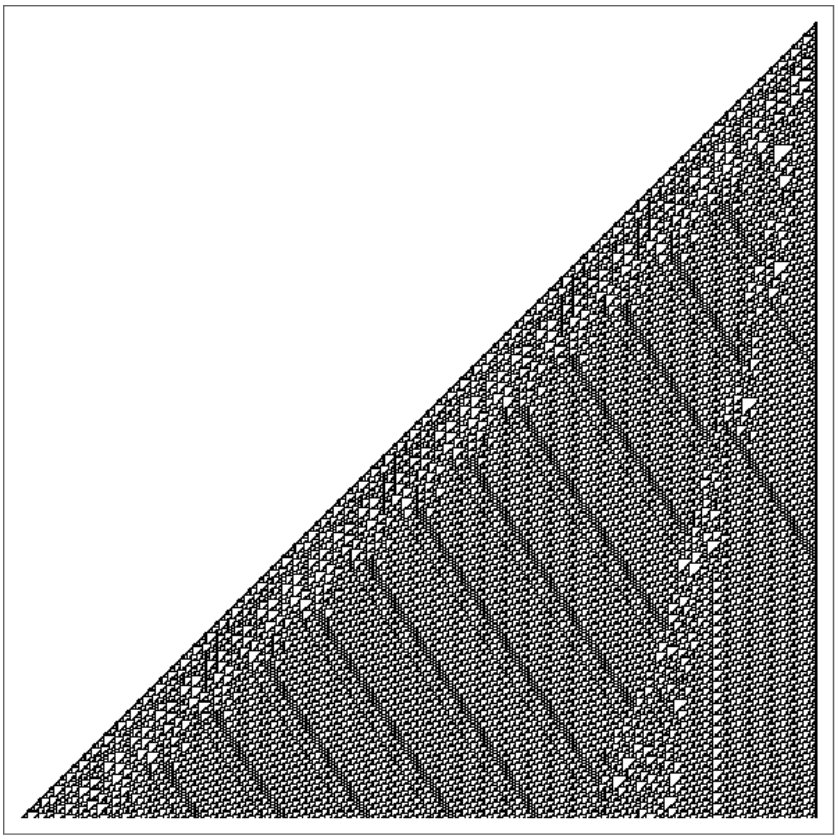


(b) 350 evoluciones.

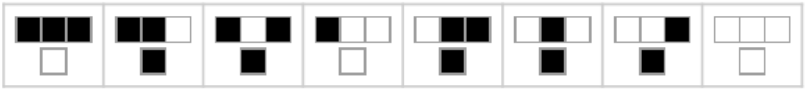


(c) Función de transición

Figura 3.11: Evolución de la regla 30

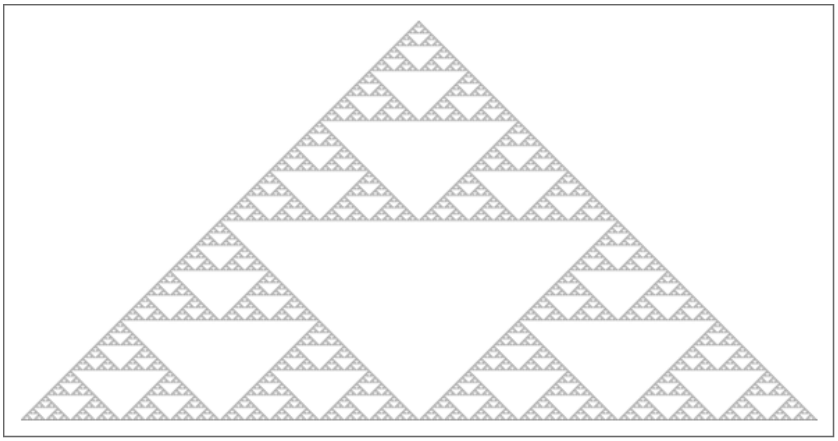


(a) 500 evoluciones.

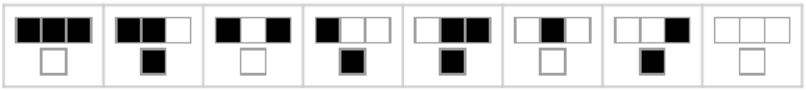


(b) Función de transición

Figura 3.12: Evolución de la regla 110



(a) 1025 evoluciones.



(b) Función de transición

Figura 3.13: Evolución de la regla 90

Stephen Wolfram realiza una clasificación basada en el comportamiento que exhiben ciertas reglas y que es inherente a la configuración inicial. Es por esto que define las siguientes clases de comportamiento, ilustradas en la figura 3.14:

WI Evolucionan a un estado uniforme.

WII Evolucionan a estados cíclicos aislados.

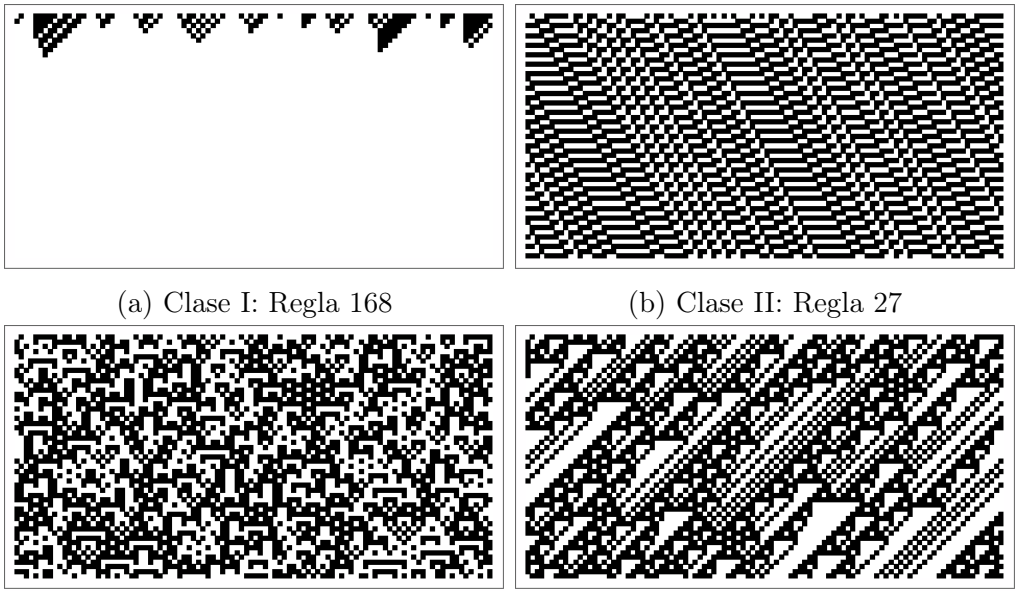
WIII Evolucionan en comportamientos caóticos.

WIV Evolucionan en estados complejos aislados.

Sin embargo, existen reglas bajo diferentes configuraciones que pueden pertenecer a múltiples clases o, en el peor de los casos, no pertenecen a ninguna de ellas [54].

Li y Packard ofrecen otra clasificación que considera seis clases para categorizar el comportamiento dinámico de los autómatas celulares [55]:

LPI La regla evoluciona a un estado homogéneo donde todas las células adquieren el mismo valor.



(a) Clase I: Regla 168 (b) Clase II: Regla 27

(c) Clase III: Regla 105 (d) Clase IV: Regla 106

Figura 3.14: Clasificación de Wolfram. Las figuras evolucionan 50 pasos con la misma configuración inicial y con condición de frontera cíclica.

LPII Evoluciona hasta una configuración estática no necesariamente homogénea.

LPIII Reglas con un periodo igual a dos.

LPIV Reglas con un periodo de L.

LPV Evolucionan a estructuras caóticas.

LPVI Conduce a estructuras largas y estructuras complejas.

Por otro lado, Culick y Yu [55, 56] proponen una clasificación que sigue las siguientes limitaciones:

- Esta clasificación es para autómatas celulares no totalísticos.
- Cada clase es un subconjunto de la siguiente clase.

CY I Un autómata celular es de clase I si existe un estado estable $s \in \mathcal{S}$ tal que todas las configuraciones finitas evolucionan a una configuración homogénea de s.

Es decir, dada cualquier configuración inicial en un autómata celular finito, todas las células adquieren el estado s tras cierto número de pasos evolutivos. Esto se aplica a todas las finitas configuraciones iniciales.

CY II Un autómata celular es de clase II si existe un estado estable $s \in \mathcal{S}$ tal que se repite a n pasos evolutivos.

Esto implica que la evolución se repite cada n número de pasos ($\sigma(t) = \sigma(t+n)$). Resulta evidente que la clase I está contenida en la clase II, haciendo que $n = 1$, lo que implica un periodo de uno.

Una clasificación es s-finita si solo difiere del estado estable en una región finita.

CY III Un autómata celular es de clase III si existe un estado estable $s \in \mathcal{S}$ tal que para cualquier par de configuraciones s-finitas C_1 y C_2, es decidible si C_1 evoluciona a C_2.

Es decir, si podemos determinar en tiempo finito si el CA evoluciona de C_1 a C_2, entonces pertenece a clase III. En caso contrario, pertenece a la clase IV, la cual contiene a todos los autómatas celulares [56].

CY IV Todos los autómatas celulares pertenecen a la clase IV.

3.8. Extensiones y cambios de la definición

Otra clasificación considera extensiones o modificaciones en la definición que alteran el comportamiento del autómata celular. A continuación, se describen algunas de estas extensiones para el modelado de sistemas complejos.

- **Autómatas celulares en grafos**: Amplían la idea de la estructura de red clásica al permitir que las vecindades no sean iguales: cada célula puede tener un número diferente de vecinos. Esta flexibilidad los hace adecuados para modelar fenómenos sobre grafos, como redes sociales o flujos en infraestructura. Sin embargo, diseñar manualmente las reglas de transición en estos entornos es una tarea compleja, lo que ha motivado propuestas como los autómatas celulares neuronales en grafos, que permiten aprender dichas reglas automáticamente [57].
- **Autómatas celulares probabilísticos**: Estos extienden el modelo clásico al incorporar reglas de transición estocásticas. En vez de una evolución que

sigue un camino fijo, los estados futuros dependen de probabilidades que se ven afectadas por el entorno. Esto ayuda a entender la incertidumbre, el ruido y los comportamientos que surgen de manera inesperada. Son especialmente útiles para simular dinámicas como propagación de epidemias o sistemas sociales, donde los mecanismos de transición no siempre están completamente definidos [58].

- **Autómatas celulares no homogéneos**: Permiten que diferentes células tengan diferentes funciones de transición. Para esto, se utilizan dos planos llamados "plano de datos", el cual contiene la evolución del sistema; y "plano de control", el cual define las reglas de transición para cada célula. Esto permite estudiar el comportamiento y la interacción entre diferentes reglas que comparten espacio [59].
- **Autómatas celulares continuos**: Son autómatas celulares que pueden ser continuos en los estados, en el tiempo y en el espacio [41]:
  - Los estados continuos adquieren valores reales y no solo enteros.
  - El tiempo continuo se define mediante una función que determina el intervalo de espera para actualizar la célula en función de su estado actual.
  - El espacio continuo se obtiene al asignar diferentes pesos a las células dentro de la vecindad [60].

Estas variantes no son exclusivas y además han ampliado el uso de los autómatas celulares. Sin embargo, un problema que se repite es definir correctamente las reglas de transición para lograr un fenómeno específico. Este desafío ha llevado a la creación de autómatas celulares neuronales. En estos, la función de transición no se define de manera clara, sino que se aprende a través de una red neuronal artificial. Pero antes de adentrarse en la definición, es necesario comprender qué son las redes neuronales.

Capítulo 4

Marco teórico: Inteligencia artificial y redes neuronales

Las redes neuronales artificiales son una herramienta de inteligencia artificial inspirada en el pensamiento humano, más específicamente, en cómo un conjunto de neuronas interconectadas comparten información para realizar inferencias.

Las redes neuronales tienen su origen en 1943, donde McCulloch y Pitts propusieron un modelo que establecería las bases del perceptrón. Posteriormente, en 1957, Rosenblatt formalizó su implementación computacional, pero no fue hasta los años ochenta donde tuvo un impacto en su implementación gracias al surgimiento de la retropropagación y de las redes recurrentes [61].

Las redes neuronales artificiales son un conjunto de neuronas o módulos computacionales interconectados que realizan cálculos y transmiten información para llevar a cabo una operación, imitando el comportamiento del cerebro.

Sin embargo, para entender este modelo, es necesario tener los fundamentos de inteligencia artificial y la clasificación de aprendizaje supervisado. Es por esto que en este capítulo, se describen términos como inteligencia artificial, aprendizaje supervisado, la composición, estructura y comportamiento de las redes neuronales y algunos tipos de ellas.

4.1. Inteligencia artificial

La inteligencia artificial fue definida por primera vez por John McCarthy en 1955 como *la ciencia e ingeniería de hacer máquinas inteligentes* [62]. Es un término que ha ido evolucionando a lo largo de los años y con diferentes interpretaciones. Para el objetivo de esta investigación y siguiendo a [62, 63, 64], se sigue la siguiente definición:

Definición 11. *La inteligencia artificial es la ciencia que utiliza modelos computacionales para crear sistemas capaces de realizar tareas que, si fueran hechas por humanos, requerirían inteligencia.*

Una de las ramas de la inteligencia artificial es el aprendizaje automático, el cual se centra en desarrollar sistemas con la capacidad de aprender en función de los datos [65]. Esto permite reconocer patrones y realizar predicciones a partir de grandes cantidades de datos [66].

Por lo que, con base en [65, 67] podemos definir al aprendizaje automático como:

Definición 12. *El aprendizaje automático es la disciplina que permite a las computadoras mejorar su desempeño en una tarea mediante la experiencia con datos, sin necesidad de instrucciones explícitas.*

Esta tecnología funciona trabajando a través del entrenamiento de algoritmos en conjuntos de datos para conseguir un resultado esperado. Es el proceso de optimizar el modelo para que pueda predecir la respuesta correcta. A mayor calidad en los datos de entrenamiento, mayor precisión se tendrá en el modelo [66].

El entrenamiento se refiere al proceso de enseñarle a un modelo de inteligencia artificial a realizar una o varias tareas mediante la exposición a grandes cantidades de datos [68]. Este proceso facilita a los humanos el proceso de análisis de extensos volúmenes, especialmente en la era actual, donde se generan más datos que nunca en la historia [69].

Dentro del aprendizaje automático se definen tres métodos en cómo una computadora puede aprender. Estos métodos son aprendizaje supervisado, no supervisado y por refuerzo.

- El aprendizaje supervisado se caracteriza por el uso de conjuntos de datos etiquetados para entrenar algoritmos que clasifiquen datos o predigan resultados con precisión. Durante el proceso de entrenamiento, el modelo recibe

entradas junto con las salidas correctas, lo que le permite al modelo ajustar sus parámetros hasta que sea capaz de realizar predicciones precisas sobre nuevos datos [67].

- El aprendizaje no supervisado, a diferencia del supervisado, no necesita de un conjunto de datos etiquetados generados por el ser humano. Este tipo de aprendizaje puede descubrir patrones y estadísticas sin ninguna orientación o instrucción explícita [70].
- El aprendizaje por refuerzo busca imitar la forma de aprendizaje humana de prueba y error. Este tipo de aprendizaje permite a las máquinas aprender a tomar decisiones y mejorar sus resultados con base en los errores cometidos [71].

IBM [67] menciona algunos de los algoritmos más comunes de aprendizaje automático:

1. **Regresión lineal:** Predice valores numéricos, con base en una relación lineal entre diferentes valores.
2. **Agrupación en clústeres:** Identifican patrones en los datos y los agrupan en centroides.
3. **Árboles de decisión:** Utilizan una secuencia ramificada de decisiones vinculadas que se representan con un diagrama de árbol.
4. **Redes neuronales:** Simulan el funcionamiento del cerebro humano, con un gran número de nodos de procesamiento conectados entre sí, llamados neuronas.

Las redes neuronales presentan ventajas que pueden llegar a agilizar y optimizar recursos de las organizaciones que lo implementan [72]: mejora la productividad, reduce el error humano, mejora la disponibilidad de servicios, reduce tareas repetitivas, descubre patrones ocultos a simple vista, entre otros.

Para comprender cómo las redes neuronales logran estas capacidades, es necesario explorar primero los fundamentos que inspiraron el desarrollo. En particular su diseño es una abstracción de las neuronas biológicas, cuya estructura y funcionamiento ofrecen una analogía para el procesamiento de la información.

4.2. Neurona biológica

Las neuronas son células especializadas del sistema nervioso cuyo objetivo es recibir, procesar y transmitir información por medio de señales químicas y eléctricas [73, 74].

Cada neurona se compone (principalmente) de tres elementos, como se muestra en la figura 4.1:

- Las dendritas, que reciben señales provenientes de otras neuronas.
- El núcleo o cuerpo, que integra las señales provenientes de las dendritas y produce una nueva señal eléctrica.
- El axón, el cual conduce la señal producida por el cuerpo hacia otras neuronas haciendo sinapsis, es decir, transmite la señal de una célula a otra.

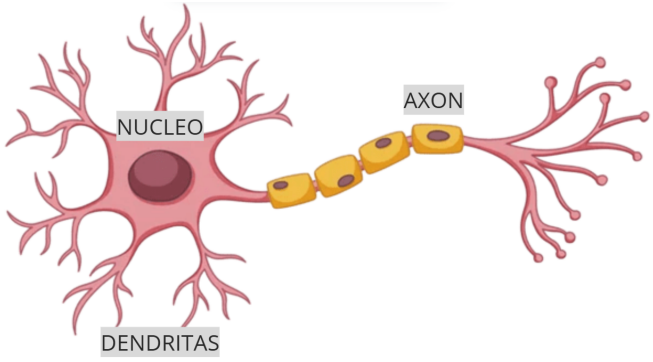


Figura 4.1: Representación de una neurona [75]. Se encuentran señaladas las dendritas, el núcleo y el axón, encargado de recibir, procesar y transmitir la información, respectivamente.

Las redes neuronales naturales son un sistema de neuronas que permite el procesamiento cerebral de la información; son altamente complejas, no lineales y con procesamiento en paralelo.

4.3. Neurona artificial

La neurona artificial se considera como una unidad de procesamiento de información [73]. Sigue la misma estructura del modelo biológico con algunas conside-

raciones adicionales, como lo marcan [73, 76, 77]:

- Las dendritas equivalen al conjunto de datos de entrada. Sin embargo, no toda la información tiene la misma relevancia; para esto se le asigna a cada entrada un peso sináptico.
- El cuerpo se sustituye por dos funciones: (1) una función de agregación que combina todos los datos de entrada (generalmente es una combinación lineal entre las entradas y los pesos sinápticos) y (2) una función de activación que determina si la neurona está activa y en qué grado.

La cual se suele representar como en la figura 4.2.

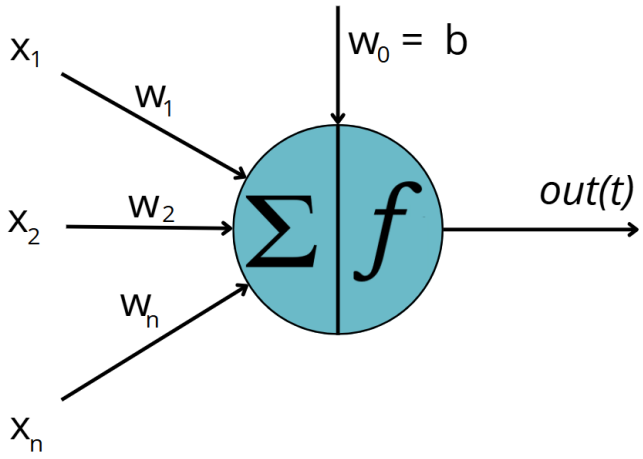


Figura 4.2: Representación gráfica de una neurona artificial. Contiene las entradas $(x_1, x_2, \ldots, x_n)$, los pesos sinápticos $(w_1, w_2, \ldots, w_n)$, el sesgo (w_0 o b), la función de agregación (Σ), la función de activación (f) y la salida.

Definición 13. *Una neurona artificial es una función* $f : \mathbb{R}^n \to \mathbb{R}$ *dada por:*

$$f(x) = \phi(\omega(X, W, b))$$

donde:

- $X = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}^n$ *es el conjunto de datos que recibe cada neurona.*
- $W = \{w_1, w_2, \ldots, w_n\} \in \mathbb{R}^n$ *es el conjunto de pesos sinápticos que indican la importancia de cada entrada.*

- *b o $w_0 \in \mathbb{R}$ es un sesgo que desplaza la función de activación y controla el umbral de aceptación.*
- *$\omega(X, W, b)$ es una función de agregación $\omega : \mathbb{R}^n \to \mathbb{R}$ que combina las entradas. Usualmente se define como una combinación lineal:*

$$\omega(X, W, b) = \sum_{i=1}^{n} w_i x_i + b$$

- *$\phi(\omega(X, W, b))$ es la función de activación $\phi : \mathbb{R} \to \mathbb{R}$ que transforma $\omega(X, W, b)$ en una salida activada.*

Además de la combinación lineal, la función de agregación puede adoptar otras formas, como el máximo, el mínimo o combinaciones lógicas (AND, OR) en redes con entradas binarias. No obstante, la suma ponderada continúa siendo la más utilizada en aplicaciones prácticas [61].

La función de activación es la que determina el valor de salida de cada neurona. Este valor, normalmente, se encuentra acotado dentro de un rango específico (como $[0, 1]$ o $[-1, 1]$) para normalizar el comportamiento de la red [78]. Además, su naturaleza no lineal es fundamental para que el modelo pueda aproximar funciones complejas y resolver problemas que no son linealmente separables.

4.4. Redes neuronales

Una neurona únicamente puede realizar una operación. Para aumentar la capacidad de entender patrones más complejos, es necesario añadir múltiples neuronas interconectadas, lo que se conoce como red neuronal artificial [79]. Esta puede definirse como un grafo dirigido [80].

Definición 14. *Una red neuronal artificial es un grafo dirigido $G = (V, E)$ donde cada nodo $v \in V$ representa una neurona y cada arista $(u, v) \in E$ transporta la salida y_u desde u hasta v ponderada por el peso w_{uv}.*

Existe un conjunto de nodos de entrada $X \subseteq V$ que reciben los datos iniciales y un conjunto de nodos de salida $T \subseteq V$ cuya activación final representa la salida de la red.

Aunque la topología de la red puede ser arbitraria, en la práctica las redes se organizan en tres conjuntos de capas. Esto se muestra en la figura 4.3, las cuales

son (1) una de entrada que recibe los datos, (2) una o varias ocultas y (3) una de salida que devuelve el resultado. Adicionalmente, suelen diseñarse redes neuronales completamente conectadas, es decir, todas las neuronas tienen una relación con todas las de la siguiente capa, pero no es una restricción [81].

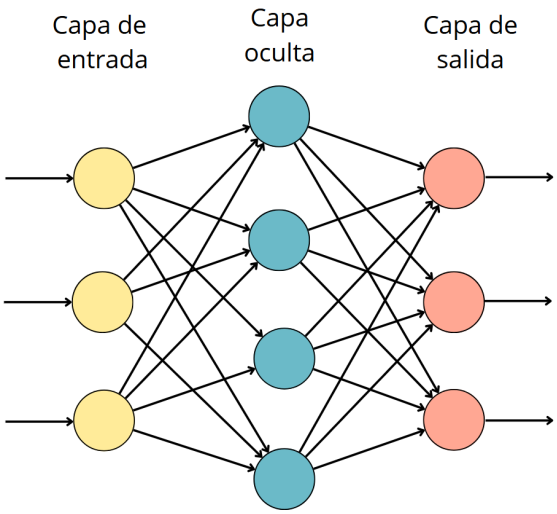


Figura 4.3: Red neuronal de tres capas: tres entradas, una capa oculta y tres salidas.

La red en su conjunto implementa una función parametrizada.

$$F_\theta : \mathbb{R}^n \to \mathbb{R}^m$$

donde n es el número de datos de entrada, m el número de datos de salida y θ el conjunto de parámetros entrenables. El entrenamiento de una red neuronal consiste en ajustar θ.

Por lo que se puede concluir con la definición de [61], la cual dice que

Definición 15. *Una red neuronal es una función* $F_\theta : \mathbb{R}^n \to \mathbb{R}^m$ *donde* $\theta = \{w_{uv}, b_v : (u, v) \in E, v \in V\}$ *es el conjunto de parámetros entrenables.*

4.4.1. Funciones de activación

La capacidad de F_θ para modelar relaciones no lineales depende de las funciones de activación aplicadas en cada nodo v. Sin la función de activación, la red neuronal podría definirse únicamente como una función lineal.

En esta sección se presentan las funciones de activación más comunes utilizadas en redes neuronales. Para cada función se muestra su definición matemática, una breve descripción de su comportamiento práctico y una figura ilustrativa.

4.4.1.1. Binary step (escalón binario)

La función escalón binario se define por partes:

$$f(x) = \begin{cases} 1 & \text{si } x \geq 0, \\ 0 & \text{si } x < 0. \end{cases}$$

Produce salidas discretas (0/1) y fue utilizada en perceptrones tempranos como función de decisión. No es diferenciable en el punto de umbral y su derivada es nula en casi todas partes, por lo que no es adecuada para entrenamiento mediante gradiente (backpropagation). Aun así, se usa conceptualmente para tomar decisiones discretas o como capa de postprocesado/umbral en modelos ya entrenados.

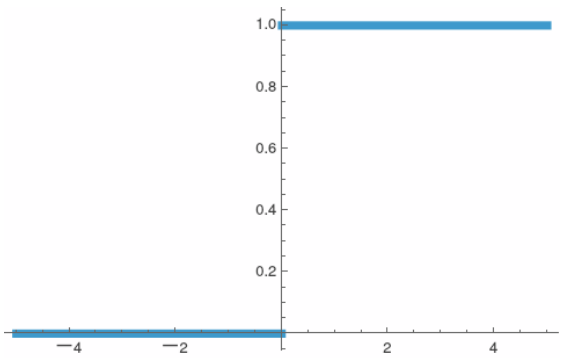


Figura 4.4: Función de activación escalón binario.

4.4.1.2. Lineal

La función lineal (o identidad) se define como:

$$f(x) = ax + b \quad \vee \quad f(x) = x.$$

Produce una salida proporcional a la entrada y mantiene la linealidad en la red. Se utiliza principalmente en la capa de salida para tareas de regresión donde se requiere predecir valores continuos. No es adecuada para capas ocultas cuando se desea que la red aprenda transformaciones no lineales.

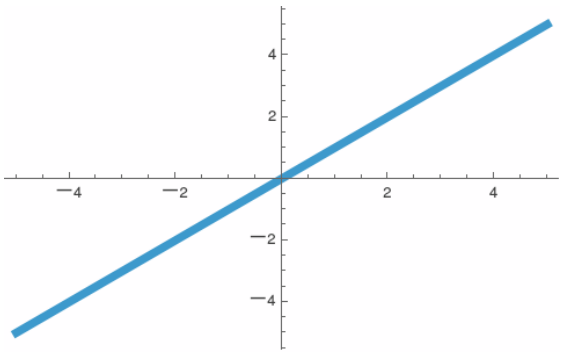


Figura 4.5: Función de activación lineal.

4.4.1.3. Sigmoide

La función sigmoide logística se define como:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Tiene la propiedad de:

$$\sigma'(x) = \sigma(x)\,(1 - \sigma(x)).$$

Comprime la entrada en el intervalo $(0, 1)$, por lo que es apropiada para modelar probabilidades y se ha usado históricamente en la salida de clasificadores binarios. Sufre saturación cuando los valores son grandes, lo que provoca gradientes pequeños (esto se llama el problema del gradiente desvanecido) y una salida que no está centrada (es positiva). Por eso, en las capas ocultas modernas, a menudo se reemplaza por ReLU o tanh.

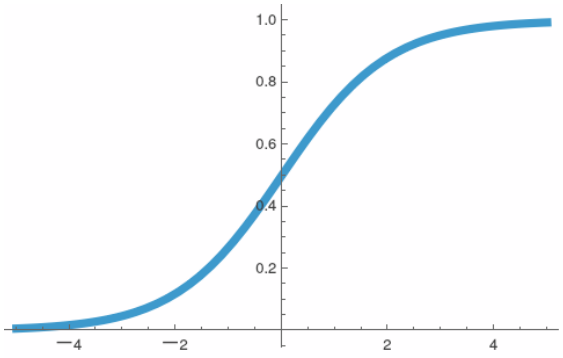


Figura 4.6: Función de activación sigmoide.

4.4.1.4. tanh (hiperbólica)

La tangente hiperbólica se define como:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Su rango es $(-1, 1)$ y su derivada puede expresarse en función de sí misma.

Es una versión centrada en cero de la sigmoide, lo que facilita el aprendizaje (gradientes con medias cercanas a cero). Se usa en capas ocultas cuando se necesita una salida limitada y centrada. Era una opción común en problemas secuenciales y en redes neuronales tradicionales antes de que se popularizaran arquitecturas y funciones de activación más modernas.

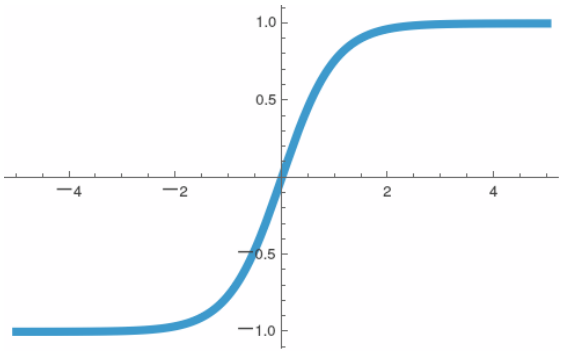


Figura 4.7: Función de activación `tanh`.

4.4.1.5. ReLU (Rectified Linear Unit)

La función ReLU se define por partes:

$$f(x) = \text{máx}(0, x).$$

Activa linealmente las entradas positivas y pone a cero las negativas. Es sencillo y computacionalmente eficiente, favorece la esparcidad de activaciones y reduce en la práctica el problema del gradiente desvanecido en redes profundas. Muy utilizada en capas ocultas de arquitecturas modernas. Tiene la desventaja de poder provocar neuronas "muertas" (gradiente cero para entradas negativas), problema que funciones como Leaky ReLU intentan corregir.

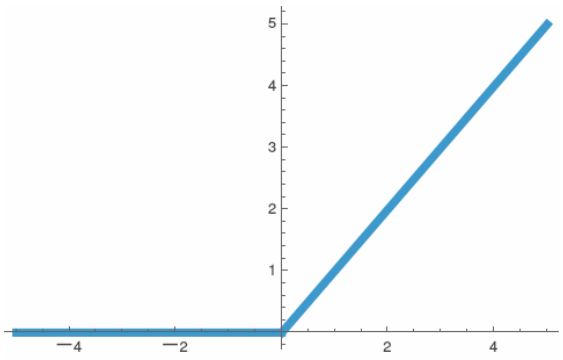


Figura 4.8: Función de activación ReLU.

4.4.2. Aprendizaje en redes neuronales

Dada una red neuronal artificial con los parámetros entrenables totalmente configurados, es capaz de realizar su clasificación o predicción de manera correcta. Sin embargo, resulta imposible configurar estos pesos manualmente, por lo que requiere de un "entrenamiento" que ayude a definir los pesos sinápticos. A continuación se define la función de pérdida, optimización y entrenamiento, elementos necesarios para obtener esa configuración específica.

En los primeros modelos, la actualización de los pesos se realizaba a partir del error cuadrático del valor de activación. La regla de actualización se expresaba como:

$$\Delta w_j = -\alpha(c - a)x_j$$

Donde a representa la activación y α la tasa de aprendizaje. Este método permitió superar la no diferenciabilidad de la función escalón en los perceptrones originales [61].

4.4.2.1. Función de pérdida

La función de pérdida se usa para medir el rendimiento del modelo. Calcula la desviación que existe entre el valor predicho y el valor esperado. Esta función puede ser un escalar o un vector que permite aproximar la tasa de error para llegar a una solución óptima [82].

Definición 16. *Sea F_θ una red neuronal artificial y $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ un conjunto de datos de entrenamiento donde $x^{(i)}$ es la entrada y $y^{(i)}$ es la salida esperada;*

una función de pérdida es un mapeo que tiene la siguiente forma:

$$\mathcal{L} : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}_{\geq 0}$$

Tal que mide la discrepancia entre la predicción del modelo y la salida real:

$$\mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

Es importante que la función de pérdida sea diferenciable. El gradiente de la función de pérdida permite conocer la dirección hacia la cual debe moverse para disminuir la pérdida.

Las funciones de pérdida se pueden utilizar para regresión y para clasificación. La primera mide el error en valores continuos y se utiliza para estimar valores contables como el precio, tamaño y tiempo. La segunda mide los errores en valores discretos, como la clasificación o la categoría.

Distintas funciones de pérdida priorizan diferentes tipos de error; la elección de la función de pérdida dependerá de la tarea que se desea realizar. Además, la función de pérdida únicamente se aplica a una muestra; para una población de muestras, se utiliza la función de costo, que se define a continuación [82, 83]:

Definición 17. *Sea F_θ una red neuronal artificial y $\mathcal{L}$ la función de pérdida; la función de costo es el promedio de las pérdidas:*

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(F_\theta(x^{(i)}), y^{(i)})$$

4.4.2.2. Optimización

El entrenamiento de una red consiste en ajustar los parámetros entrenables θ para minimizar la función de pérdida. La optimización consiste en resolver el problema de minimización dado por:

$$\theta^* = \arg\min_{\theta} J(\theta)$$

Esto se puede conseguir mediante diferentes técnicas, siendo el descenso del gradiente la más común. Este método actualiza los parámetros en la dirección opuesta al gradiente de la función de pérdida:

$$\theta_{t+1} = \theta_{(t)} - \eta \nabla_\theta J(\theta^{(t)})$$

donde $\eta > 0$ es la tasa de aprendizaje. Una tasa de aprendizaje alta puede divergir en la solución, mientras que una tasa baja puede tener una convergencia lenta [61].

En la práctica, es común iniciar el entrenamiento con una tasa de aprendizaje relativamente alta y luego reducirla gradualmente a medida que el modelo converge. También se suele incorporar un término de *momentum* para acelerar la convergencia y reducir oscilaciones [61].

4.4.2.3. Entrenamiento

Antes del entrenamiento, las variables de entrada deben normalizarse para ajustarse al rango de activación de la red (por ejemplo, $[0, 1]$ o $[-1, 1]$). Este paso evita que una variable con valores grandes domine la función de agregación y mejora la estabilidad del entrenamiento. Es importante que la normalización preserve la distribución original de los datos y la proporción entre atributos [61].

Finalmente, el entrenamiento consiste en ajustar todos los parámetros $\theta = \{W^{(l)}, b^{(l)}\}$ para minimizar la función de costos $J(\theta)$ en un conjunto de datos de entrenamiento. Este proceso se compone de las siguientes etapas:

1. Se inicializan los parámetros, generalmente de forma aleatoria: $\theta_0 \sim \mathcal{D}_{init}$
2. Se hace la propagación hacia adelante o *forward pass*, es decir, para cada entrada $x^{(i)}$ se evalúa $\hat{y}^{(i)} = F_\theta(x^{(i)})$
3. Se calcula la pérdida: $\mathcal{L}^{(i)} = \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$
4. Se calculan los gradientes propagando la información hacia atrás o *backpropagation* utilizando la regla de la cadena.

   $$\nabla_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

   Se actualizan los parámetros mediante la función de optimización: $\theta_{t+1} = \theta_{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$
5. Se repite este proceso sobre un *dataset* hasta la convergencia de la solución o hasta alcanzar un criterio de parada [61].

   Una vez entrenada la red, puede evaluarse la importancia relativa de las entradas mediante un análisis de sensibilidad. Este procedimiento consiste en:

a) Calcular el valor medio de cada entrada.

b) Evaluar la salida de la red con todos los valores medios.

c) Modificar una entrada a la vez (aumentándola o disminuyéndola) y medir el cambio en la salida.

d) Repetir el proceso para cada variable de entrada.

De esta manera, se identifica qué variables influyen más en el resultado, lo cual aporta interpretabilidad al modelo [61].

La elección de la topología de la red, en particular el número de neuronas ocultas, suele determinarse empíricamente. Una regla práctica consiste en comenzar con un número de neuronas ocultas similar al número de entradas y ajustar a partir del rendimiento obtenido. Otras heurísticas recomiendan iniciar con el doble de nodos de entrada y reducir gradualmente si se detecta sobreajuste [61].

En resumen, el entrenamiento de la red neuronal es un proceso iterativo de optimización que consiste en:

1. Propagar los datos de entrada por la red.
2. Comparar la salida con el valor real usando una función de pérdida.
3. Propagar el error hacia atrás permite obtener gradientes.
4. Actualizar los parámetros con un algoritmo de optimización.
5. Repetir hasta que el modelo aproxime adecuadamente la función deseada.

4.5. Tipos de redes neuronales

Las redes neuronales pueden clasificarse atendiendo a diferentes criterios, ya sea por el número de capas, el tipo de conexiones, el grado de conectividad o la arquitectura general. En este trabajo se han seleccionado dos tipos principales, de manera enunciativa, mas no limitativa, que son especialmente relevantes para su eventual implementación en un autómata celular neuronal: las redes neuronales convolucionales y las recurrentes. La razón de esta elección es la siguiente:

Por un lado, las evoluciones de un autómata celular ocurren de forma secuencial en el tiempo, es decir, las entradas y salidas de la red corresponden a estados del

espacio celular y, en algunos casos, las reglas pueden tener una relación temporal o reversible. De ahí que las redes recurrentes resulten particularmente apropiadas.

Por otro lado, como se explicará más adelante, se ha demostrado que los autómatas celulares y las redes neuronales convolucionales tienen algunas similitudes, especialmente porque el *kernel* de la red convolucional es similar a la vecindad del autómata celular.

4.5.1. Redes neuronales convolucionales

Las redes neuronales convolucionales constituyen una clase de redes profundas cuya estructura está diseñada para explotar la topología de los datos que tienen forma de rejilla. Por ejemplo, imágenes en dos dimensiones.

Una red neuronal convolucional consta de una capa de entrada, varias capas ocultas (que pueden incluir capas de convolución, normalización, *pooling*, activación, etc.) y una capa de salida.

Se asume un volumen de entrada con dimensiones $(H \times W \times C)$, es decir, altura H, anchura W y C canales (por ejemplo, RGB en imágenes). La red maneja este volumen usando filtros deslizantes. Aunque normalmente no se describen en términos de estados discretos de autómatas, se puede entender que cada neurona de una capa representa un estado continuo real (por ejemplo, una activación $a \in \mathbb{R}$).

El elemento clave es su operación de convolución: un filtro (o *kernel*) de tamaño $k \times k$ se desliza sobre la entrada, compartiendo pesos y produciendo mapas de activación. Se puede ver como una vecindad local de la neurona que influye en su valor de activación que se aplica en cada capa de convolución:

$$Y = f(X * K + b)$$

Donde X es el volumen de entrada, K es el filtro (o filtros) de convolución, b es el sesgo y f es una función de activación. A continuación pueden seguir operaciones de *pooling* y capas totalmente conectadas para producir la salida final [84].

Se aplica principalmente en clasificación de imágenes, segmentación, reconocimiento de video y procesamiento de audio. En el contexto de los autómatas celulares, se puede ver que el filtro de la red neuronal convolucional es similar a la vecindad del autómata. Esto permite entenderlo como una operación de actualización local que se repite en un espacio organizado, igual que en un autómata celular.

4.5.2. Redes neuronales recurrentes

Las redes neuronales recurrentes son un tipo de redes que tienen conexiones que vuelven sobre sí mismas. Esto permite que la información de momentos anteriores afecte el procesamiento que se está realizando en el presente.

Una red neuronal recurrente trabaja con una serie de entradas $(x_1, x_2, \ldots, x_T)$ y genera una serie de salidas $(y_1, y_2, \ldots, y_T)$. En cada momento, la red tiene un estado oculto h_t que se actualiza de la siguiente manera:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h),$$
$$y_t = g(W_{hy}h_t + b_y),$$

Donde W_{xh}, W_{hh} y W_{hy} son matrices de pesos, b_h y b_y son sesgos, y f, g son funciones de activación.

La red funciona en una secuencia de tiempo, donde cada momento puede verse como una çapa virtual.º una repetición del autómata. En cada instante t, el estado oculto h_t muestra la "memoria.º el contexto acumulado hasta ese momento.

La vecindad temporal está dada por la conexión de la unidad de tiempo t con la anterior $t-1$ (y en variantes bidireccionales, también con $t+1$). Es decir, la salida de un paso se inserta como parte de la entrada del siguiente paso.

Como se mostró más arriba, la actualización recursiva del estado oculto actúa como la función de transición temporal de la red. La red integra información histórica para producir la salida en cada paso de tiempo.

Estas redes se utilizan frecuentemente en el procesamiento de lenguaje natural, reconocimiento de voz, series temporales, traducción automática y otras tareas en las que el orden temporal de los datos es esencial [85].

Dado que un autómata celular evoluciona en pasos discretos de tiempo y cada uno puede depender del estado previo del sistema, las redes neuronales recurrentes aportan un modelo natural para capturar dependencias temporales entre estados sucesivos del autómata. Esto resulta especialmente útil si la regla del autómata celular tiene memoria o se desea que aprenda dinámicas espacio-temporales.

En conclusión, mientras que las redes neuronales convolucionales capturan la estructura espacial y la vecindad local del sistema, las redes neuronales recurrentes modelan su evolución y persistencia temporal. La integración de ambas capacidades

permite que un autómata celular neuronal no solo emule reglas estables, sino que aprenda dinámicas complejas a partir de datos, superando las limitaciones de las reglas de transición diseñadas manualmente.

Capítulo 5

Marco teórico: Autómatas celulares neuronales

En los últimos años, los autómatas celulares neuronales surgieron como una extensión de los clásicos, con las capacidades de aprendizaje y generalización propias de las redes neuronales artificiales. Esta integración permite que la regla de transición, tradicionalmente fija, sea parametrizable y entrenable mediante técnicas de aprendizaje supervisado.

Las primeras aproximaciones copiaban el funcionamiento de un autómata celular usando una red neuronal convolucional [86]. Este método fue dado a conocer por Mordvintsev et al. [9], quienes entrenaron un modelo que aprende de dinámicas locales a partir de ejemplos. Como resultado, las interacciones locales básicas cumplen un objetivo en específico, mostrando propiedades de crecimiento, regeneración, autoorganización y morfogénesis.

Los autómatas celulares neuronales son entonces un tipo de autómatas celulares, pero cuya función de transición está definida como una red neuronal artificial. Retomando las definiciones previas, podemos decir entonces que:

Definición 18. *Un autómata celular es neuronal si la regla local $\mathcal{R}$ está dada como:*

$$\mathcal{R} = F_\theta : \mathbb{R}^{|\mathcal{N}|} \to \mathbb{R}^1$$

donde:

- *F_θ es una red neuronal parametrizada por θ.*
- *$\theta = \{w_{uv}, b_v : (u, v) \in E, v \in V\}$ es el conjunto de parámetros entrenables.*

- $\mathbb{R}^{|\mathcal{N}|}$ *corresponde al espacio de entrada definido por la vecindad* $\mathcal{N}$*.*
- $\mathbb{R}^1$ *define el valor de salida que representa el nuevo estado de la célula central.*

La figura 5.1 muestra la relación existente entre estas dos tecnologías. En el paso N se tiene una configuración bidimensional con vecindad de Neumann. El estado de cada elemento dentro de la vecindad funciona como valor de entrada para la red neuronal entrenada y devuelve un único resultado. Este resultado corresponde a la salida de la célula central, es decir, a la cual se le está aplicando.

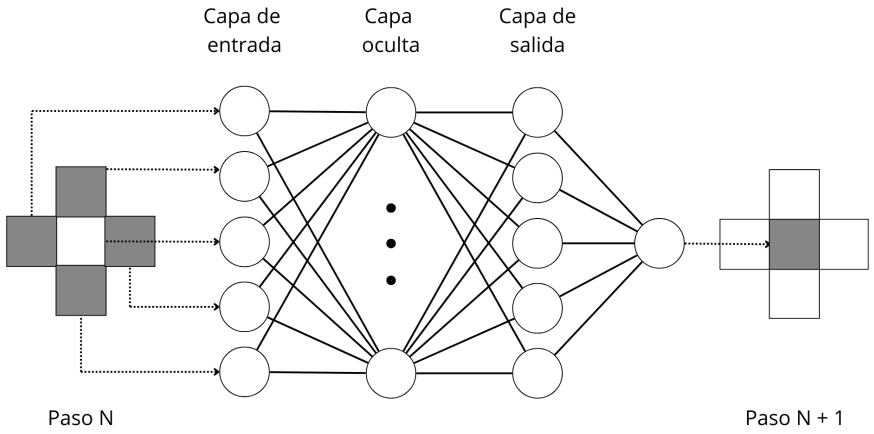


Figura 5.1: Representación gráfica de la red neuronal.

Cada célula aplica una misma red neuronal local F_θ sobre su vecindad inmediata. La entrada de la red consiste en los estados de las celdas dentro de la vecindad y la salida es el nuevo estado de la célula central.

F_θ suele representarse como una red neuronal artificial convolucional que permite procesar localmente la información espacial preservando la estructura D-dimensional.

A diferencia de los autómatas clásicos, donde las reglas son explícitas y definidas manualmente, los autómatas celulares neuronales aprenden sus reglas mediante un proceso de entrenamiento. Se establece un grupo de condiciones iniciales X_0 y un estado final X^*. Luego, la red F_θ se ajusta para reducir una función de pérdida $\mathcal{L}(X_T, X^*)$, que mide la diferencia entre la configuración obtenida después de T pasos y el estado deseado. Este proceso puede expresarse como:

$$\theta^* = \arg\min_\theta \mathcal{L}(F_\theta^{(T)}(X_0), X^*).$$

Durante el entrenamiento, se emplean técnicas de retropropagación a través del tiempo y optimización por gradiente descendente. Esto permite que la red aprenda

comportamientos dinámicos complejos, tales como el crecimiento morfológico, la regeneración parcial o la adaptación frente a perturbaciones.

Como las redes neuronales generan valores reales, se puede medir la tasa de error entre la predicción que hacen y el comportamiento real del sistema al usar la regla que han aprendido. Esta métrica de error se propone como base para el desarrollo del presente trabajo. Sin embargo, también se puede comparar la pérdida con otros elementos cercanos, ya que dos células pueden tener muchos vecinos en común, lo que podría explicar el error que se observa en la predicción.

Como se muestra a continuación, esta herramienta ha demostrado ser capaz de generar patrones autoorganizados y adaptativos sin necesidad de una programación explícita de reglas. Algunos ejemplos incluyen:

- **Morfogénesis:** Crecimiento de formas y estructuras similares a organismos biológicos.
- **Regeneración:** Capacidad de reconstruir regiones dañadas o eliminadas de la configuración.
- **Modelado de sistemas complejos:** Simulación de fenómenos físicos, ecológicos o sociales mediante interacciones locales entrenables.

Esta mezcla de comportamiento que aparece por sí mismo y la habilidad de aprender hace que los autómatas celulares neuronales sean una herramienta muy útil para estudiar sistemas que se organizan solos y que se adaptan.

Los autómatas celulares neuronales surgen de la necesidad de definir funciones de transición más precisas que permitan mejorar la simulación de sistemas complejos, como por ejemplo la propagación del fuego.

Consideremos un autómata celular bidimensional con tres posibles estados: $\{materia\ viva, fuego, materia\ quemada\}$, una vecindad de Moore y una función de transición definida por las siguientes reglas: si una célula en estado de materia viva se encuentra adyacente a una célula en estado de fuego, entonces se quema; y si una célula en estado de fuego está rodeada principalmente de materia quemada, entonces se apaga.

Sin embargo, esta formulación que se basa en reglas claras es una simplificación excesiva que reduce la capacidad del modelo para imitar comportamientos que se ven en sistemas reales. Para superar esta limitación, los autómatas celulares

neuronales incorporan mecanismos de aprendizaje automático que permiten definir la función de transición de manera adaptativa.

En este enfoque, el autómata aprende a partir de ejemplos. Por ejemplo, secuencias de video que muestran la propagación del fuego, donde cada estado puede representarse mediante un color y cada célula corresponde a un píxel o conjunto de píxeles. De esta manera, la red neuronal subyacente aprende una función de transición capaz de imitar la dinámica del sistema complejo, ajustando sus parámetros a partir de los datos observados.

Adicionalmente, los autómatas celulares tradicionales suelen operar con configuraciones reducidas, es decir, con pocos estados, vecindades mínimas y reglas básicas, ya que el espacio de configuraciones crece exponencialmente al incrementar cualquiera de estos factores. Esta limitación dificulta modelar sistemas complejos presentes en la naturaleza, los cuales suelen involucrar múltiples variables e interacciones de alta dimensionalidad.

5.1. Estado del arte, aplicaciones y líneas actuales de investigación

Un primer avance en esta línea fue presentado por Gilpin [86], demostrando que una red neuronal convolucional puede aprender el comportamiento de un autómata celular clásico. Su estudio destaca que tanto los autómatas celulares como las redes neuronales convolucionales comparten dos propiedades fundamentales: (1) la actualización basada en una vecindad o *kernel*, y (2) la aplicación simultánea de dicha función en todo el espacio.

En particular, Gilpin entrenó una red neuronal convolucional con un *kernel* de 3×3 (mismo tamaño que la vecindad de Moore) para imitar el comportamiento del *glider*[1] en el Juego de la Vida de John Conway. Este trabajo estableció una conexión entre los autómatas celulares y las redes neuronales convolucionales, mostrando que estas redes artificiales pueden actuar como mecanismos de transición local aprendidos.

Posteriormente, esta línea de investigación fue extendida por Mordvintsev et al. [9], quienes exploraron representaciones continuas para morfogénesis artificial; y por Niklasson et al. [87], que aplicaron el enfoque a la generación de texturas dinámicas sobre superficies tridimensionales. Otros autores también han sugerido

[1]Una de las configuraciones más importantes por su capacidad de moverse en el espacio

usar autómatas celulares neuronales en robótica, clasificación de patrones y simulación de sistemas fotónicos. Esto ha fortalecido su papel como herramienta para modelar sistemas que se autoorganizan.

5.1.1. Morfogénesis

Mordvintsev diseñó un autómata celular neuronal que permite a una célula crecer para formar un patrón, persistir en el tiempo, regenerarse e incluso rotar. Fue diseñado un modelo que utiliza un autómata celular continuo con un vector de estados de 16 dimensiones. Este modelo incorpora gradientes espaciales utilizando filtros de Sobel.



Figura 5.2: Evolución del *emoji* de una lagartija a partir de una única célula activa después de 219 pasos evolutivos [9].

El autómata celular neuronal diseñado tiene las siguientes características (las cuales fueron retomadas por múltiples autores que tomaron a [9] como referencia):

- El vector de cada célula (cada valor en un rango de $[0, 1]$) incluye los canales RGB y un canal α que funciona como umbral para definir si una célula está viva o muerta (establece el resto de valores del vector a 0 si $\alpha \leq 0.1$).

  Considerando las dimensiones $(\text{largo}, \text{ancho}, 16)$ y un kernel de 3×3 y se calcula el gradiente en las dos direcciones (x y y) usando un filtro Sobel. Esto genera un nuevo vector que se utilizará como entrada para la red neuronal.

  $$16(\text{estado propio}) + 16(\text{gradiente en } x) + 16(\text{gradiente en } y) = 48$$

- Se usaron convoluciones de 1×1 con una función $ReLU$ y alrededor de 8000 parámetros entrenables y devuelve un δ que se suma al estado actual de célula antes del siguiente paso.

- Se omite el cálculo para las células muertas y la actualización de cada célula se realiza con una probabilidad del 50 %. La actualización debe ser estocástica para lograr un mejor rendimiento [87].

Eyvind Niklasson et al. [87] compararon el rendimiento del modelo en [9] con actualización síncrona y asíncrona. Donde, tras haber llevado a cabo pruebas con diferentes tamaños de paso, concluyeron que la función de pérdida crece exponencialmente cuando la actualización es síncrona. Sin embargo, esto ocurre únicamente cuando el tamaño de paso es mayor que uno.

Una hipótesis de este comportamiento es que, al tener células que se actualizan en diferentes periodos de tiempo, cada célula puede conocer parte del contexto de su vecindad. Esto no se logra únicamente conociendo los valores actuales de cada vecino.

Se realizaron los siguientes experimentos en [9]:

1. Crecer una única célula activa para formar una figura dada mediante *backpropagation*. Cuando el autómata celular llega a la figura deseada, si se siguen aplicando las reglas de evolución, puede crecer sin límites, quedarse en la misma figura, eliminar todas las células o crear patrones que son caóticos o que se repiten.
2. Convertir las figuras objetivo en atractores usando *sample pool*. Se entrenó mediante múltiples condiciones iniciales, obtenidas a partir de entrenamientos fallidos, para llegar al objetivo desde diferentes configuraciones.
3. Regenerar partes de la figura. Se eliminaron partes de la figura objetivo y se añadieron al conjunto de entrenamiento.
4. Rotaron la figura objetivo. Dado que los filtros de Sobel funcionan como direcciones ortogonales, se puede aplicar una rotación mediante:

$$\begin{bmatrix} K_x & K_y \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \cdot \begin{bmatrix} Sobel_x & Sobel_y \end{bmatrix}$$

   Para modificar el ángulo sin necesidad de reentrenar el modelo.

5.1.1.1. Autómatas celulares neuronales isotrópicos

En [88], [89] y [90], Mordvintsev et al. presentan una nueva configuración que no tenga una dirección preferida, la cual nombraron autómatas celulares neuronales isotrópicos.

Para que esta regla sea isotrópica, es necesario que sea independiente de la dirección (sin distinguir "arriba" de "izquierda"). Esto permite crear rotaciones o simetrías sin que el sistema se descontrole.

Se sustituyen las funciones de los filtros de Sobel en [9] por un único filtro laplaciano:

$$K_{lap} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

El cual se concatena al vector de cada célula y alimenta a una red neuronal artificial de 9408 parámetros.

A continuación, se describen los resultados obtenidos con dos diferentes enfoques:

1. Estrategia de semilla estructurada: Se necesitan al menos tres células con valores diferentes y no colineales para definir la orientación y la posición. Modificar la posición de estos puntos implica una evolución acorde a la posición de los mismos.

2. Estrategia de una única semilla: Se modifica la función de pérdida para que sea invariante ante rotaciones y reflejos. Para evitar comparar la figura resultante con todas las rotaciones posibles, se mapean ambas figuras a coordenadas polares y, para cada posible ángulo de rotación, se calcula la diferencia. Para disminuir la complejidad computacional, se aplica la transformada rápida de Fourier y se hace uso del teorema de la convolución:

   $$s * t = \mathcal{F}^{-1}(\mathcal{F}(S) \cdot \mathcal{F}(T))$$

   Se obtienen todas las diferencias únicamente como una multiplicación de transformadas.

   Para evitar que se formen patrones simétricos mezclados (como figuras con cabeza-cabeza o cola-cola), se agrega un nuevo canal llamado *binary auxiliary channel* que da valores de -0.5 y 0.5 a la parte superior e inferior de la imagen, respectivamente.

   Con estas nuevas configuraciones se puede obtener el mismo resultado que en [9], pero sin la dependencia de una dirección explícita y con una red neuronal artificial con menos parámetros.

5.1.1.2. Autoclasificación

En [91] Mordvintsev utiliza los autómatas celulares neuronales para la autoclasificación de MNIST[2] las cuales fueron binarizadas con un umbral de 0.1.

[2]Conjunto de datos de imágenes 28×28 de números en el rango $[0, 9]$

El objetivo es que, a partir de una configuración inicial de células vivas, el sistema pueda reconocer la forma solo usando reglas locales. En este caso, se trata de identificar los números de MNIST.

Estos son los cambios del autómata celular MNIST respecto al modelo original [9]:

- Vector de la célula: Originalmente se usaban los canales RGB; ahora se emplean 10 canales que funcionan como distribución de probabilidad de cada posible número. Durante la evaluación se utiliza el mayor de ellos.
- Célula viva o muerta: En [9] se define mediante el canal α, pero las células muertas siguen siendo visibles para el resto. En [91] únicamente interactúan las células vivas.
- Topología: Todas las células son un único componente conexo.

Con el nuevo modelo, Mordvintsev pudo realizar clasificaciones con una precisión del 95.3 % con una función de pérdida L_2 y ruido añadido. Este experimento demostró que los autómatas celulares neuronales no solo pueden generar patrones y autorregenerarse, sino también realizar tareas de clasificación distribuidas sin una capa de decisión global.

Aunque los modelos en [9] y [91] son robustos frente a diferentes tipos de perturbaciones, su comportamiento se puede ver afectado por estados individuales de las células, parámetros del modelo y el campo perceptivo (la manera en cómo las células perciben su vecindad).

Es por esto que Randazzo et al. [92] buscan enfrentar estrategias adversarias en el mismo espacio celular para estudiar la reprogramación de los autómatas celulares mediante dos experimentos:

1. **Inyección de adversarios**: Se entrena un nuevo modelo de autómata celular, que se coloca en el entorno que ejecuta uno de los modelos originales, para secuestrar el comportamiento del colectivo.
2. **Perturbación de estados**: A un modelo ya entrenado, se le aplican perturbaciones a todas las células del sistema de forma simultánea.

Para la inyección de adversarios se utilizan dos modelos: el modelo original del autómata celular para MNIST con probabilidad de 90 % y un nuevo modelo

que clasifica a todos los números como "ocho" (10 %). Se introducen varias células invasoras en figuras que ya han sido clasificadas, y estas células muestran un comportamiento dominante al clasificar todos los elementos como "ocho". Además, una vez retiradas las células invasoras, el sistema es capaz de recuperarse la mayoría de las veces.

Sin embargo, al aplicar el mismo procedimiento para [9], los resultados no tuvieron el mismo rendimiento. Probaron modificar la morfología de la imagen de una lagartija sin cola y con otro color, pero ninguna estrategia fue capaz de invadir por completo el modelo original.

Una hipótesis es que para MNIST no necesita cambiar completamente su valor para producir algo diferente de lo que generaba antes, ya que el estado de la mutación es un estado que el modelo original ya tenía en cuenta. Por otro lado, al cambiar el color y la forma en [9], se están explorando configuraciones nunca antes vistas por el modelo original.

Para el segundo experimento (perturbación de estados), se sigue la línea de modificar el color y la forma de la imagen de una lagartija. Se aplica una perturbación global de estados cada paso de tiempo con el objetivo de identificar la perturbación específica que dé como resultado un patrón estable.

En cada paso evolutivo, se multiplica el vector de cada célula por una matriz simétrica A de 16×16 [9]. Para definir los valores de la matriz, se inicia con una matriz simétrica I y, a diferencia del modelo original, la configuración inicial se establece como la figura objetivo del mismo.

Esta configuración permite crecer un sistema con perturbaciones, alterar su forma y color una vez crecido y modificar el impacto y el número de perturbaciones posibles. Al manejar matrices simétricas, se puede representar como

$$A = Q\Lambda Q^T$$

donde:

- Λ es la matriz diagonal de los valores propios.
- Q es la matriz unitaria de los vectores propios.

Si $\Lambda = I \rightarrow A = I$, por lo que no existe perturbación (multiplicación por matriz identidad). Bajo esta premisa, se propone la siguiente fórmula:

$$A_k = Q(kD + I)Q^T$$

Donde:

- A es la matriz original.
- k es el factor de escala.
- D es la dirección de la perturbación $\Lambda - I$ en el espacio de valores propios.

Si $k = 0$, no existe perturbación en el sistema:

$$A_k = Q(0D + I)Q^T$$
$$A_k = QIQ^T$$

Si $k = 1$, se conserva la perturbación original:

$$A_k = Q(1D + I)Q^T$$
$$A_k = Q(\Lambda - I + I)Q^T$$
$$A_k = Q\Lambda Q^T$$

Mostrando que se puede escalar el tamaño de la perturbación, incluso con perturbaciones inversas con $k = -1$.

Si consideramos dos perturbaciones cuyos vectores propios son similares: $A_k = Q(k_A D_A + I)Q^T$ y $B_k = Q(k_B D_B + I)Q^T$, podemos combinarlos como

$$comb(A_k, B_k) = Q(k_A D_A + k_B D_B + I)Q^T = k_A A + k_B B + (1 - k_A - k_B)I$$

Esta nueva combinación permite configurar múltiples perturbaciones, las cuales se vuelven más sensibles conforme aumentan los k.

Estos experimentos muestran cómo los autómatas celulares neuronales pueden ser vulnerables a diferentes estrategias y cómo las mutaciones "exactas" pueden modificar la estructura de todo el sistema.

Todas estas representaciones han tenido lugar en un espacio bidimensional. A continuación se presentan algunas propuestas en espacios tridimensionales.

5.1.1.3. Representaciones tridimensionales

Zhang et al. [93] proponen un modelo generativo tridimensional denominado autómata celular generativo que extiende el paradigma de los autómatas celulares hacia la generación progresiva de formas tridimensionales. A diferencia de enfoques que intentan reconstruir directamente la forma completa, el autómata celular generativo formula el proceso como una cadena de Markov, donde cada paso corresponde a una transición local sobre una rejilla tridimensional.

El modelo se entrena para que estas reglas locales, al repetirse, converjan hacia formas específicas. Para ello, se representa la forma como puntos de superficie en una rejilla 3D y se emplea una red convolucional dispersa que opera únicamente sobre las regiones activas.

Durante el entrenamiento, se utilizan secuencias ligeramente distintas a las cadenas de muestreo reales, pero que convergen hacia las formas del conjunto de datos. Esto permite que el modelo aprenda reglas de transición robustas, capaces de generar formas diversas desde configuraciones parciales. Con una sola inferencia, el sistema puede tomar diferentes ejemplos de la distribución que ha aprendido, obteniendo buenos resultados en tareas de completar y generar formas en 3D.

Este enfoque es una aportación importante a la morfogénesis computacional, ya que muestra que las reglas locales que se autoorganizan pueden aplicarse a estructuras 3D complejas utilizando redes neuronales adaptadas.

En [93], se muestra que la autoorganización local se puede escalar a tres dimensiones. Por otro lado, Shyam Sudhakaran et al. [94] amplían el trabajo de [9] a tres dimensiones, usando el juego de Minecraft como espacio celular[3]. El autómata celular neuronal entrenado puede crear estructuras de hasta 3584 bloques utilizando 50 bloques diferentes, manteniendo las propiedades de regeneración de su versión en 2D.

Una entidad en Minecraft se representa como una red 3D donde cada elemento es un vector que almacena (1) el tipo de bloque (incrustado en un vector *one-hot*), (2) su estatus (replicando el comportamiento del canal α de [9]) y (3) estados ocultos.

Se utilizan convoluciones para permitir la generación y manipulación de estructuras 3D. Además, se sustituye el uso de filtros de Sobel por una red de percepción

[3]En Minecraft, se pueden crear estructuras estáticas (como árboles) y dinámicas (como máquinas voladoras)

aprendible con un kernel de $3 \times 3 \times 3$, *stride*[4] de 1 y canales de salida equivalentes a tres veces el número de canales del estado celular. Esta red permite que el sistema aprenda activamente a percibir su entorno local, adaptando sus filtros durante el entrenamiento y capturando patrones espaciales relevantes de manera dinámica.

Se utiliza una actualización estocástica y un umbral de activación similar a [9]. Finalmente, todos los modelos se inicializan con una distribución normal estándar, con media cero y desviación estándar en el rango de 0.0001 a 0.1, lo que proporciona un punto de partida estable para el entrenamiento y la evolución del sistema. Este sistema hereda las propiedades regenerativas de su versión 2D.

Finalmente, Horibe et al. [95] llevan las propiedades regenerativas al mundo físico. Se implementa el enfoque regenerativo en robots blandos mediante autómatas celulares neuronales. Cada robot tiene un conjunto de volúmenes de reposo para cada vóxel[5]. Estos volúmenes se activan con una señal global que tiene forma de onda sinusoidal, lo que causa que se expandan y se contraigan. Esta dinámica permite simular locomoción a través de ciclos de actuación, evaluando el desplazamiento del centro de masa como métrica de fitness. Los robots blandos se componen de distintos tipos de vóxeles: músculos en fase y contrafase, huesos rígidos y flexibles, cada uno con un rol en la locomoción.

El modelo entrenado demuestra que los robots blandos pueden recuperar parcialmente su morfología original tras sufrir daño y restaurar su capacidad de movimiento sin intervención externa. Este comportamiento surge de la dinámica autoorganizada del autómata celular neuronal, que codifica de manera implícita la forma objetivo y su capacidad de reparación.

Para evaluar la capacidad de locomoción y regeneración de los robots blandos generados mediante autómatas celulares neuronales, se emplea el simulador físico Voxelyze. Los experimentos se realizan en configuraciones 2D y 3D, utilizando rejillas de 7×7 y $9 \times 9 \times 9$ vóxeles, respectivamente. Las condiciones de frontera tipo Dirichlet limitan el crecimiento morfológico a regiones internas (5×5 en 2D y $7 \times 7 \times 7$ en 3D).

Este entorno permite comprobar cómo los autómatas celulares neuronales se autoorganizan para crear formas que son funcionales y resistentes. Además, proporciona una base cuantitativa para comparar la eficiencia locomotora antes y después de procesos de regeneración. La estructura en vóxeles ayuda a combinar reglas locales y programación distribuida, y se relaciona con modelos inspirados

[4]Cuántas posiciones se desplaza el *kernel* sobre la entrada en cada operación de convolución

[5]Unidad básica de un espacio tridimensional, similar a un píxel en imágenes 2D, pero con volumen

en la biología sobre cómo se forma la estructura.

En conjunto, estos trabajos evidencian cómo los autómatas celulares neuronales pueden extender sus principios regenerativos y de autoorganización a dominios tridimensionales cada vez más complejos. Mientras Autómatas celulares generativos establecen un marco generativo para formas 3D a partir de reglas locales aprendidas, los experimentos en Minecraft muestran que dichas propiedades pueden preservarse incluso en entornos con alta variabilidad estructural. Finalmente, la aplicación en robots blandos confirma el potencial bioinspirado de los autómatas celulares neuronales: no solo permiten generar morfologías funcionales, sino que además dotan a los sistemas físicos de resiliencia ante daños, abriendo camino hacia formas de robótica autorreparable.

Otra forma de implementar los autómatas celulares neuronales en 3D es mediante planos, mallas y texturas autoorganizadas que cubren la superficie de un objeto tridimensional.

5.1.2. Autómata celular neuronal en malla

Niklasson et al. [96] y Mordvintsev et al. [97] [98] introducen el concepto de texturas autoorganizadas, cuyo objetivo es reproducir la apariencia global de una textura de referencia sin generar copias exactas. Para esto, entrenan un autómata celular neuronal utilizando la misma estructura neuronal que se propuso en [9], para que pueda reconocer y crear patrones distintivos como tableros de ajedrez, líneas o mosaicos. Aunque este enfoque tiene un carácter demostrativo, constituye una evidencia significativa de que los autómatas celulares neuronales pueden aprender y generalizar comportamientos visuales locales en lugar de memorizar imágenes específicas.

Niklasson et al. proponen los autómatas celulares neuronales en mallas [99], que es un método creado para crear texturas en movimiento en superficies en 3D. A diferencia de los experimentos en cuadrículas de dos dimensiones, los autómatas celulares neuronales en mallas aplican el mecanismo de autoorganización a mallas de cualquier forma, incluso si están en movimiento. El modelo es consistente en diferentes situaciones, es resistente a cambios externos y permite ajustar características como la densidad, la dirección y la rotación de los patrones que aparecen. Además, varios autómatas celulares neuronales en redes pueden combinarse, creando cambios suaves entre texturas sin interrupciones.

Sin embargo, este enfoque presenta un reto de escalabilidad. El número de

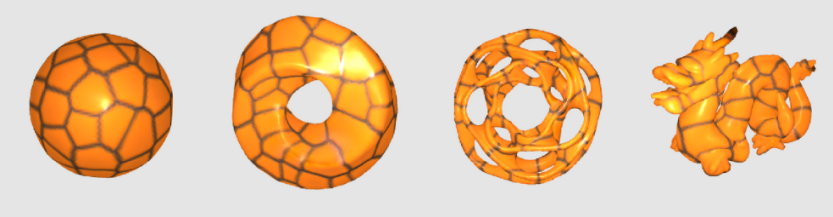


Figura 5.3: El entrenamiento se realiza sobre un poliedro simétrico y el autómata celular neuronal puede ser extrapolado a cualquier objeto tridimensional [9].

células involucradas suele encontrarse en el rango de $10^4 \sim 10^5$, lo que limita la resolución alcanzable a entre 64×64 y 256×256 píxeles. Como el tiempo de entrenamiento escala de forma cuadrática con respecto al número de celdas, el aumento de resolución conlleva un cuello de botella en la propagación de información a lo largo de la malla.

Para solucionar esta limitación, Niklasson et al. sugieren en [100] un modelo híbrido que une autómatas celulares neuronales con representaciones neuronales implícitas. La idea central es desacoplar el tamaño de la cuadrícula de los autómatas celulares neuronales de la resolución final, delegando el renderizado detallado a un decodificador ligero denominado red local de producción de patrones. De este modo, el autómata celular neuronal mantiene una cuadrícula explícita y relativamente pequeña que evoluciona de manera local, mientras que la red local de producción de patrones se encarga de producir el color en cualquier punto del espacio continuo a partir de información interpolada de las celdas vecinas.

En términos formales, para cada punto de muestreo p, la red local de producción de patrones recibe dos entradas: Una es un estado celular promedio $s(p)$, obtenido mediante interpolación de las celdas cercanas en un primitivo geométrico; la otra es un vector de coordenadas locales $u(p)$, que codifica la posición relativa de p dentro de dicho primitivo. Dependiendo de cómo se divida el espacio, estas formas básicas pueden ser rectángulos, cubos, triángulos o polígonos en general. Estas formas están definidas por coordenadas $\lambda_i(p)$ que deben cumplir con ciertas propiedades, como ser una partición unitaria y no ser negativas. Para asegurar una continuidad en el espacio, se cambian las coordenadas usando funciones sinusoidales en cuadrículas cartesianas o en redes triangulares, creando así campos posicionales suaves.

El resultado es un sistema que conserva las propiedades autoorganizativas de los autómatas celulares neuronales al tiempo que permite alcanzar mayor resolución sin un aumento proporcional del costo computacional. Desde una perspectiva biológica, el autómata celular neuronal puede interpretarse como un plano morfo-

genético que define la organización global, mientras que la red local de producción de patrones añade los detalles finos sin necesidad de simular interacciones locales a gran escala. Este paralelismo, aunque principalmente ilustrativo, refuerza la idea de que los autómatas celulares neuronales pueden actuar como generadores de estructuras robustas y adaptables, mientras que módulos complementarios aportan precisión y escalabilidad.

A pesar de estas mejoras, una limitación persistente es que cada autómata celular neuronal debe entrenarse de manera específica para una textura o patrón dado. Incluso en escenarios donde múltiples autómatas celulares neuronales coexisten, cada modelo requiere su propio entrenamiento independiente. Esto ha motivado nuevas propuestas orientadas a la construcción de espacios compartidos de autómatas celulares neuronales, capaces de generalizar a múltiples texturas dentro de un mismo marco de entrenamiento.

5.1.3. Modelos generativos

Cada imagen creada con un autómata celular neuronal tiene una red neuronal artificial que ha sido entrenada específicamente para copiar ese comportamiento. Alejandro Hernández et al. [101] proponen unas redes neuronales artificiales de mayor capacidad que permitan crecer múltiples imágenes, a lo que llamaron colector de autómatas celulares neuronales.

El colector de autómatas celulares neuronales usa un modelo de convoluciones dinámicas para modelar un espacio de dichos autómatas. Utiliza un *autoencoder* que se compone de un codificador y un decodificador: El codificador transforma la entrada en una representación continua y luego categórica, que el decodificador reconstruye en el espacio original.

Un elemento destacable de la arquitectura propuesta es un parámetro entrenable llamado *Leak Factor* (LF) que mejora la estabilidad durante el entrenamiento. Este parámetro regula la cantidad de información que se filtra en cada paso del modelo; por ejemplo, un valor bajo retiene el conocimiento acumulado. Este valor se incrementa durante el entrenamiento a medida que mejora el rendimiento.

La arquitectura también emplea convoluciones dinámicas. Los *kernels* se generan a partir del *embedding* de cada instancia, permitiendo adaptabilidad sin modificar la estructura del autómata celular neuronal. El colector de autómatas celulares neuronales inicia desde una imagen semilla y actualiza cada celda mediante percepciones locales (filtros Sobel + normalización) y capas densas por píxel,

moduladas por el LF. Se exploran actualizaciones estocásticas y sincrónicas, siendo esta última más precisa y estable, tal como en [87].

Los experimentos muestran que la codificación y la actualización simultáneamente mejoran la calidad de la reconstrucción, incluso sin mecanismos de enmascaramiento. También se observa que aumentar la dimensionalidad del *embedding* mejora significativamente los resultados.

Por otro lado, Kevin Frans en [102] propone un modelo llamado autómatas celulares neuronales condicionales, cuya semilla recibe un *embedding*, el cual define la imagen en la cual va a crecer.

Dado un diseño deseado, una red de codificador lo comprime en un *embedding*, el cual se utiliza como condición inicial en el autómata celular neuronal, que luego itera para crecer en la figura objetivo; esto permite crear nuevas configuraciones.

Utilizando un *dataset* con aproximadamente 3000 *emojis* de 32×32 píxeles, fue capaz de entrenar a un autómata celular neuronal condicional utilizando el mismo método de [9] para poder dibujar todos los *emojis* con un único autómata celular neuronal.

Para generar números de MNIST, se adopta un enfoque tipo red adversarial generativa, donde el objetivo es crecer dígitos desde ruido aleatorio. Dado que entrenar directamente un autómata celular neuronal como generador de red adversarial generativa resultó inestable, se utilizó una estrategia en dos etapas: (1) Se entrena primero un generador tradicional (*feedforward*) para mapear vectores aleatorios a dígitos MNIST y (2) el autómata celular neuronal se entrena para imitar la salida de ese generador, logrando una generación más estable.

Este enfoque permite que el autómata celular neuronal genere dígitos sin necesidad de un *dataset* explícito, aprendiendo directamente a partir del objetivo de la red adversarial generativa. Además, se observa un patrón de crecimiento distintivo: mientras los *emojis* crecen en ondas desde el centro, los dígitos MNIST se desarrollan siguiendo la forma del trazo.

Mingxiang Chen et al. [103] combinan los principios de codificador automático variacional con autómata celular neuronal. Utilizan un *encoder* que transforma una imagen en una distribución de probabilidad, las cuales forman un *embedding*, el cual funciona como condición inicial del autómata celular neuronal. Después, Rasmus Berg Palm et al. [104] llaman a este modelo "autómatas celulares neuronales variacionales". También afirman que ninguno de los otros modelos mencionados ([9], [101], [93], [95] y [102]) es realmente generativo.

El autómata celular neuronal variacional aprende una distribución de probabilidad que codifica los datos observados. A partir de esta representación, el *encoder* genera imágenes mediante un autómata celular neuronal que simula el crecimiento espacial desde una semilla. A diferencia del codificador automático variacional, que decodifica la imagen directamente, el autómata celular neuronal variacional utiliza el *embedding* como configuración inicial de una cuadrícula de celdas que evolucionan siguiendo las reglas definidas por una red neuronal convolucional.

Por otra parte, este modelo incorpora mecanismos de duplicación celular inspirados en la mitosis. Cada M pasos, las celdas se duplican hacia la derecha y hacia abajo, permitiendo a la imagen crecer de manera autoorganizada.

Los experimentos realizados en [104] demuestran que el autómata celular neuronal variacional es capaz de aprender procesos generativos autoorganizados tanto en dominios simples como MNIST, como en conjuntos más complejos como CelebA[6]. En MNIST, el modelo logra reconstrucciones precisas y muestras plausibles desde el espacio latente, con una log-verosimilitud de $\log p(x) \geq -84.23$ nats[7]; su rendimiento es comparable a modelos autorregresivos como PixelCNN. Además, el análisis del espacio latente muestra una estructura más clara y organizada en comparación con los decodificadores convolucionales tradicionales. Esto indica que la codificación es más semántica y eficiente.

En CelebA, aunque las muestras generadas directamente no alcanzan calidad visual comparable a modelos de última generación, el autómata celular neuronal variacional muestra capacidad de reconstrucción robusta y mejora visual significativa al ajustar el parámetro β, que regula el equilibrio entre fidelidad de reconstrucción y calidad de generación.

Finalmente, se explora la resiliencia de los autómatas celulares neuronales variacionales ante daño estructural durante el crecimiento. Al eliminar las capas de duplicación y entrenar con un esquema de recuperación de daño, el modelo aprende dinámicas estables capaces de restaurar imágenes deterioradas casi por completo, incluso tras múltiples pasos de evolución. Este comportamiento sugiere que el autómata celular neuronal variacional no solo codifica la imagen, sino que también es un proceso de reconstrucción en el estado celular.

[6] *Dataset* de imágenes de celebridades

[7] Unidad de medida utilizada en aprendizaje automático y estadística para expresar la log-verosimilitud o la entropía de un modelo

5.1.4. Representaciones y aplicaciones adicionales

Hasta ahora, muchos de los estudios sobre autómatas celulares neuronales se han enfocado en la morfogénesis, que es el proceso de formación de formas, y en su habilidad para crear estructuras visuales que se organizan solas. Sin embargo, también han surgido enfoques que amplían el marco conceptual hacia nuevas formas de representación y aplicaciones en dominios específicos. A continuación, se presentan algunas de estas variantes.

Mattie Tesfaldet et al. [105] extienden los autómatas celulares neuronales mediante mecanismos de atención inspirados en arquitecturas de *transformers*, dando lugar a autómatas celulares con *transformers* de visión. Esta arquitectura híbrida permite que las interacciones locales tengan acceso contextual a regiones más amplias a través de *self-attention*, lo que habilita organización global sin incurrir en la complejidad cuadrática típica de la atención completa. Los autómatas celulares con *transformers* de visión adaptan el proceso de los *transformers* de visión tradicionales, que dividen una imagen en parches y los convierten en secuencias de *tokens*, al formato de una cuadrícula de celdas, donde cada celda almacena información visual, posicional y oculta. En lugar de aplicar atención global, restringe el cálculo a vecindarios locales, manteniendo así la naturaleza distribuida del sistema y reduciendo el costo computacional. El entrenamiento se realiza de forma iterativa, alternando entre inyectar entradas corruptas y actualizar parcialmente los estados celulares hasta reconstruir la imagen. De este modo, los autómatas celulares con *transformers* de visión combinan eficiencia, capacidad de autoorganización y organización espacial coherente.

En un ámbito distinto, Jian Tang et al. [106] emplean una red neuronal convolucional para simular el desarrollo de microestructuras en la solidificación de metales. Cada celda del dominio contiene un estado de fase $P \in [0, 1]$ (donde $p = 0$ representa líquido y $p = 1$ sólido), tres ángulos de Euler $\theta = (\alpha_1, \alpha_2, \alpha_3)$ que definen su orientación cristalográfica y la magnitud de subenfriamiento $\Delta T = T_{melting} - T$. La evolución se modela con un autómata celular neuronal que aplica recurrentemente una red neuronal convolucional para predecir los incrementos en fase, orientación y estados ocultos. Estos canales ocultos facilitan el aprendizaje de patrones temporales complejos sin necesidad de reglas explícitas. Con ello, el modelo reproduce fenómenos como crecimiento competitivo o direcciones preferentes, con una eficiencia computacional sustancialmente mayor que la de muchos autómatas celulares tradicionales y con capacidad de generalización fuera del rango de entrenamiento.

Por otro lado, Gordon H. Y. Li et al. [107] proponen el modelo de autómatas celulares neuronales fotónicos, que aprovecha la velocidad y la interconexión masiva

de la fotónica para implementar autómata celular neuronal en hardware óptico. A diferencia de arquitecturas convencionales basadas en perceptrón multicapa o red neuronal convolucional, difíciles de trasladar a entornos fotónicos por sus conexiones densas, el autómata celular neuronal fotónico utiliza reglas locales aprendidas mediante técnicas de aprendizaje profundo. Esto permite realizar clasificación de imágenes en sistemas ópticos, donde la actualización celular se implementa mediante interferencia lineal de luz y activaciones ópticas no lineales. Los autores demuestran clasificación binaria con alta precisión utilizando un número muy reducido de parámetros fotónicos programables, mostrando además robustez frente a datos fuera de distribución. El autómata celular neuronal fotónico representa una vía prometedora para llevar el aprendizaje autoorganizado a plataformas fotónicas escalables.

En el ámbito de la biología computacional, Caitlin Grasso y Josh Bongard [108] introducen los autómatas celulares neuronales potenciados. Esta es una extensión que incorpora el concepto de *empowerment*, la cantidad de control que una célula puede ejercer sobre su entorno, formalizado mediante medidas de información mutua entre acciones pasadas y estados sensoriales futuros. Incluir *empowerment* como un objetivo adicional en las tareas de morfogénesis mejora la capacidad de evolución y genera patrones de crecimiento más fuertes y adaptables. Esto permite que cada célula actúe como un agente independiente dentro de un sistema distribuido y facilita el estudio de la coordinación y comunicación celular desde una perspectiva computacional.

Finalmente, mientras que modelos como autómatas celulares con *transformers* de visión, autómata celular neuronal fotónico o autómatas celulares neuronales potenciados amplían las capacidades arquitectónicas y las plataformas de aplicación de los autómatas celulares neuronales, Richardson et al. [109] utilizan los autómatas celulares neuronales como herramienta para modelar fenómenos dinámicos complejos. Entrenando autómata celular neuronal mediante optimización basada en gradientes para reproducir trayectorias espacio-temporales generadas por ecuaciones diferenciales parciales (por ejemplo, Gray–Scott), este enfoque captura tanto estructuras transitorias como patrones estacionarios, incluyendo la formación de patrones tipo Turing, y aborda el problema inverso de inferir reglas locales a partir de comportamiento emergente. Con una complejidad de red mínima, los autores muestran capacidad de generalización, respeto por simetrías y robustez frente a perturbaciones, posicionando a los autómatas celulares neuronales como un marco explicativo y eficiente para sistemas biológicos y físicos.

Se observa que los autómatas celulares neuronales han evolucionado desde sus primeras aplicaciones en morfogénesis, donde se exploró su capacidad de generar

estructuras complejas partiendo de una única célula activa, hacia modelos que buscan una mayor versatilidad generativa, capaces de producir dinámicas visuales, texturas, comportamientos adaptativos e incluso manipular estrategias, muchos de ellos inspirados en la definición de Mordvintsev. Además, se han desarrollado arquitecturas híbridas que no generan un autómata celular neuronal individual, sino que modelan un espacio de autómatas semejantes, así como configuraciones generativas y bioinspiradas.

En resumen, estos estudios muestran que los autómatas celulares neuronales no solo son un concepto interesante desde el punto de vista teórico, sino que también son útiles en inteligencia artificial, ciencias naturales e ingeniería. Esta diversidad de enfoques abre la puerta a explorar nuevas adaptaciones y propuestas, especialmente en áreas de aprendizaje profundo, autoorganización y simulación de procesos dinámicos.

Capítulo 6

Marco metodológico

El marco metodológico se constituye como el resultado de la aplicación sistemática y lógica de los conceptos y fundamentos abordados en el marco teórico. Azuero et al. [110] lo definen como un conjunto de acciones destinadas a describir y analizar el fondo del problema planteado, mediante procedimientos específicos que incluyen técnicas de observación y recolección de datos, los cuales determinarán el cómo se llevará a cabo la investigación.

Este capítulo se dividirá en secciones que definan la estructura del marco metodológico, con el fin de sustentar los métodos y recursos elegidos para el desarrollo de la investigación. Se asegurará que cada fase esté alineada con los principios teóricos fundamentales y que los resultados obtenidos sean fiables, replicables y efectivos en el tratamiento de la problemática planteada.

6.1. Diseño de la investigación

6.1.1. Diseño experimental

El diseño experimental es un método sistemático para planificar, ejecutar y analizar experimentos, con el objetivo de identificar relaciones causales entre variables. En otras palabras, se manipula una variable independiente para observar su efecto sobre la variable dependiente [111].

De acuerdo con su nivel de control, los diseños experimentales se clasifican en tres tipos: preexperimentos, cuasiexperimentos y experimentos puros. En el pre-

sente trabajo se adopta un diseño cuasiexperimental por las características propias del entorno de simulación y la naturaleza del objeto de estudio.

El diseño experimental resulta apropiado en este contexto. Se manipula una variable independiente (la definición de la función de transición mediante una red neuronal) para observar sus efectos sobre el comportamiento del sistema simulado. Esto se hace sin que exista una asignación aleatoria ni un grupo control en un sentido estricto.

6.1.1.1. Diseño cuasiexperimental

El diseño cuasiexperimental busca establecer relaciones causales sin la necesidad de asignar aleatoriamente a los participantes o unidades experimentales a los grupos de estudio. Se caracteriza por la existencia de una intervención deliberada sobre una variable independiente, mientras que los grupos de comparación se definen con base en condiciones preexistentes [112].

En el caso de esta investigación, el estudio se desarrolla en un entorno computacional controlado, donde las condiciones experimentales se reproducen mediante simulaciones. El objetivo principal es estudiar cómo diferentes factores, como la forma de la red neuronal o la configuración del autómata, afectan los patrones que aparecen en los autómatas celulares neuronales. Este enfoque permite observar efectos y comportamientos resultantes, manteniendo un control suficiente sobre las variables involucradas sin requerir un esquema de aleatorización.

6.2. Enfoque de la investigación

Este proyecto utiliza un enfoque cuantitativo, que se basa en recoger y analizar datos numéricos para describir fenómenos a través de métodos estadísticos y matemáticos [113]. Este enfoque prioriza la objetividad y medición precisa para relacionar causales entre las variables observadas.

A diferencia de los enfoques cualitativos o mixtos, este no requiere de estudiar percepciones, significados ni experiencias humanas, por lo que es adecuado para investigaciones donde las variables pueden ser definidas, controladas y medidas [114].

El enfoque cuantitativo se justifica porque el análisis se realiza a partir de datos generados mediante simulación computacional. En este proceso no hay intervención

directa con humanos y los resultados obtenidos no están sujetos a interpretaciones. Este enfoque permite comparar escenarios, desempeño de modelos e influencia de parámetros sobre la dinámica emergente del sistema.

6.3. Población

La población de estudio se define como un conjunto cerrado, finito y accesible de casos (personas, animales, expedientes, objetos, etc.), los cuales cumplen con un criterio de inclusión y que sirven como base para la investigación [115]. En otras palabras, es un conjunto conformado por todos los elementos a los que se pretende generalizar los resultados del estudio.

En este trabajo, la población que se estudia está formada por todos los autómatas celulares finitos que se encuentran en un espacio bidimensional con una distribución de cuadrícula regular. Se consideran condiciones de frontera un conjunto de estados finito y cualquier función que pueda mapear de $\mathbb{R}^{|\mathcal{N}|} \to \mathbb{R}$.

6.4. Muestra y muestreo

La muestra corresponde a un subconjunto de la población de estudio y consiste en el número de casos que realmente serán estudiados. Esta debe ser representativa para garantizar la validez de los resultados. Por otro lado, el muestreo se refiere al proceso aplicado por el investigador para seleccionar a los participantes dentro de la población [116].

Existe el muestreo probabilístico, donde cada miembro de la población tiene una probabilidad de ser escogido; y el no probabilístico, donde la selección se basa en la conveniencia, el juicio del investigador u otros criterios [116].

Para esta investigación se realiza un muestreo no probabilístico intencional, donde los elementos seleccionados para la muestra son elegidos por el criterio del investigador. Esto se justifica, ya que, al existir una infinitud de autómatas celulares dadas las restricciones en la población, es necesario delimitar y tratar de replicar los comportamientos. Además, dado que cada autómata celular resuelve un problema, es necesario identificar de forma precisa el problema que se está resolviendo [117].

6.5. Tipo de investigación

Con base en los objetivos, esta investigación se considera aplicada. La investigación aplicada, también conocida como práctica o empírica, se caracteriza porque busca poner en práctica o utilizar los conocimientos adquiridos. En este tipo, las consecuencias prácticas son un interés primordial del investigador [118].

Además, se utiliza un diseño cuasiexperimental, porque se cambian variables independientes para ver cómo afectan los resultados de la simulación, sin tener un grupo control o asignar condiciones experimentales de forma aleatoria.

Este tipo de investigación es útil porque ayuda a encontrar relaciones de causa y efecto entre los elementos del modelo y los comportamientos que surgen en el sistema, mientras se mantiene un entorno computacional que es controlado y se puede repetir.

6.6. Técnicas e instrumentos de recolección de datos

La recolección de datos se realiza a partir de las simulaciones computacionales realizadas. Dado que se trata de enfoque cuantitativo, no es necesario usar técnicas de observación como encuestas, sino procedimientos de registro automatizado para su posterior análisis numérico.

6.6.1. Técnicas de recolección de datos

- Simulación computacional: Ejecución del modelo del autómata celular bajo diferentes configuraciones de vecindad, estados y arquitectura de la red neuronal artificial.
- Registro automático de datos: Datos generados por la simulación. Como el historial evolutivo, patrones y métricas de estabilidad, error y desempeño.
- Análisis estadístico: Se aplican métricas cuantitativas que permiten comparar configuraciones para evaluar la efectividad del modelo. Métricas como precisión, convergencia, tasa de error y comportamiento temporal de los patrones.

6.6.2. Instrumentos de recolección de datos

Se consideran los siguientes instrumentos para obtener y analizar los datos:

- Simulador desarrollado: Herramienta computacional implementada en Python que permite ejecutar y visualizar el comportamiento de los autómatas celulares neuronales.
- Códigos de registro y procesamiento: Conjunto de programas auxiliares que capturan las métricas numéricas de cada simulación y las almacenan en archivos.
- Herramientas de análisis estadístico y visualización: Librerías y herramientas que permiten obtener representaciones gráficas con los datos obtenidos.

6.7. Procedimiento

El desarrollo metodológico de la investigación se llevará a cabo en diversas fases secuenciales. Estas abarcan desde el diseño e implementación del simulador hasta la evaluación del modelo de autómatas celulares neuronales. Cada fase está orientada a cumplir los objetivos planteados y a generar datos cuantitativos que permitan analizar el comportamiento de los sistemas simulados.

- **Diseño, desarrollo e implementación del simulador**: Se construye una herramienta computacional capaz de simular autómatas celulares bidimensionales. Aunque el fin último es estudiar los autómatas celulares neuronales, este simulador debe poder aceptar configuraciones clásicas. Esto debido a que, para las siguientes fases, es necesario conocer el comportamiento de configuraciones conocidas.
- **Diseño, desarrollo e integración del módulo red neuronal artificial**: Se implementa un módulo adicional que permite entrenar una red neuronal artificial para aprender el comportamiento de autómatas celulares existentes. La salida de este módulo es la función de transición aprendida. Además, este módulo se integra dentro del simulador principal para permitir la experimentación directa.
- **Imitación de patrones conocidos**: Se diseñan y ejecutan experimentos para evaluar la capacidad del modelo neuronal de imitar el comportamiento

de autómatas celulares clásicos, como el juego de la vida. Se busca, además, medir la precisión del aprendizaje y la estabilidad de las configuraciones generadas.

- **Simulación de otros sistemas complejos:** Una vez validado el funcionamiento del modelo neuronal, se aplican las mismas técnicas a simulaciones que representen otros sistemas complejos. Se busca probar diferentes configuraciones iniciales y parámetros para analizar la capacidad de generalizar la aparición de nuevos patrones.

Cada una de estas etapas genera resultados que ayudan a mejorar el proceso de desarrollo. Esto permite ajustar tanto la estructura de la red neuronal como las condiciones del simulador.

6.8. Técnicas y herramientas de procesamiento de datos

Una vez recolectados los datos provenientes de las simulaciones computacionales, es fundamental aplicar técnicas adecuadas de procesamiento y análisis para garantizar la validez de los resultados y su correcta interpretación. Dado que el enfoque de la investigación es cuantitativo, el procesamiento se centra en el análisis numérico de los datos generados por el simulador, permitiendo identificar patrones, tendencias y relaciones entre las variables del sistema.

6.8.1. Procesamiento de datos cuantitativos

Los datos cuantitativos se obtienen de las ejecuciones del simulador, donde se registran variables como número de iteraciones, estados promedio de las celdas, métricas de error y estabilidad de los patrones emergentes durante un periodo determinado. Posteriormente, estos datos se organizan, procesan y analizan mediante herramientas estadísticas y de visualización.

- **Análisis estadístico descriptivo:** Se calculan medidas de tendencia central y de dispersión con el fin de describir cuantitativamente el comportamiento de las variables analizadas y detectar posibles irregularidades o tendencias [119].

- **Visualización de datos:** Se elaboran gráficos de barras, histogramas y diagramas de dispersión para representar visualmente los resultados obtenidos, facilitando la comprensión de la dinámica del sistema y la comparación entre diferentes configuraciones experimentales.

El procesamiento y análisis de los datos se lleva a cabo utilizando herramientas computacionales. Los resultados obtenidos servirán como base para la interpretación y discusión en las etapas posteriores de la investigación.

6.9. Contextualización de la tecnología

El desarrollo de un simulador de sistemas complejos mediante autómatas celulares neuronales implica la integración de múltiples tecnologías, seleccionadas estratégicamente para satisfacer los requerimientos funcionales y técnicos del proyecto. Esto incluye crear una interfaz gráfica de usuario fácil de usar, simular autómatas celulares en dos dimensiones con diferentes estados, vecindades y reglas de transición, personalizar las condiciones iniciales y entrenar sistemas para que imiten comportamientos que el usuario proporciona.

La elección de estas tecnologías responde a criterios de compatibilidad, escalabilidad, facilidad de implementación y disposición de herramientas específicas para el procesamiento de datos e inteligencia artificial.

Con el fin de proporcionar una visión clara y comprensiva de las bases técnicas del proyecto, a continuación se presentan tanto el área de estudio, elementos clave para orientar el desarrollo del simulador y validarlo, como los fundamentos y el contexto de las tecnologías utilizadas, así como los entornos de trabajo que hacen posible este trabajo.

6.9.1. Fundamentos y contexto de las tecnologías

Un lenguaje de programación es una herramienta que permite el desarrollo de software y programas para computadora. Son empleados para diseñar programas encargados de definir el comportamiento físico y lógico de una computadora [120].

Debido a que la elección del entorno y lenguaje de programación resulta crítica para el éxito de cualquier proyecto, es importante considerar las metas y alcances

del mismo, puesto que no todos los lenguajes son óptimos para resolver ciertos tipos de problemas [121].

Entre los factores clave para la elección de un lenguaje de programación se encuentran la escalabilidad, el rendimiento, el costo de mantenimiento, la compatibilidad y el sistema de herramientas, entre otros [121].

Es por esto que, como lenguajes de programación, se decidió utilizar C++/CUDA para el núcleo de simulación y cálculos pesados en paralelo, y Python para la interfaz, visualización, control de simulación y diseño de redes neuronales.

A continuación se presentan las definiciones pertinentes de los lenguajes de programación, frameworks y librerías utilizadas para el desarrollo de la aplicación móvil.

6.9.1.1. C++

C++ es un lenguaje de programación compilado y multiparadigma desarrollado como una extensión de C [122]. Se considera un lenguaje de bajo nivel, ya que permite manipular directamente la memoria, lo que lo convierte en un lenguaje versátil. Es orientado a objetos, permite la creación de clases y objetos y es ampliamente utilizado para aplicaciones que requieren un alto rendimiento.

Es utilizado en motores de videojuegos, software de sistemas, aplicaciones en tiempo real, simulaciones científicas y cálculo numérico [123]. Para esta investigación, se utilizarán las siguientes bibliotecas de C++:

- **pybind11**: facilita la integración de código C++ con Python, permitiendo exponer funciones y clases para análisis y visualización [124].
- **iostream**: Declara objetos que controlan la lectura y escritura de datos de un programa en C++ [125].
- **algorithm**: Contiene funciones que realizan algoritmos comunes [126].
- **cstring**: Funciones que permiten la manipulación de memoria [127].

6.9.1.2. CUDA

CUDA es una plataforma de computación paralela y un modelo de programación desarrollado por NVIDIA que permite utilizar la GPU para cálculos de

propósito general. Se emplea para acelerar operaciones matemáticas y simulaciones que requieren grandes cantidades de procesamiento paralelo [128], como el entrenamiento de redes neuronales o la simulación de autómatas celulares.

Se requiere de la librería **cuda_runtime** la cual le permite a un programa escrito en C/C++ usar los núcleos CUDA de una tarjeta grafica de NVIDIA [129]. Esto permite gestionar la memoria, controlar los núcleos, manejar errores y dispositivos.

6.9.1.3. Python

De acuerdo con la página oficial de la documentación de Python [130], es un lenguaje de programación potente y fácil de aprender. Tiene estructuras de datos de alto nivel, sintaxis elemental y elegante, un tipado dinámico, naturaleza interpretada, etc.

Además de ser gratuito, se puede integrar bien en todos los equipos y sistemas. En adición a esto, [131] señala que entre los beneficios que Python incluye se encuentran los siguientes:

- Los programas son fáciles de leer y comprender.
- Mayor productividad al escribir un programa con menos líneas de código en comparación con muchos otros lenguajes de programación.
- Se puede usar fácilmente con otros lenguajes de programación.
- Soporte rápido de la comunidad.
- Se puede trasladar a través de diferentes sistemas operativos de computadora.

En adición a lo anterior, cuenta con una amplia colección de librerías y *frameworks* diseñados para tareas que resultaron muy convenientes en el desarrollo de simuladores. Se detallan tanto librerías como *frameworks* a utilizar a continuación.

- **NumPy** es una librería de Python especializada en el cálculo numérico y el análisis de datos. Es ideal para el manejo de grandes volúmenes de datos [132].

  Puede hacer operaciones hasta 50 veces más rápido que las listas normales usando un objeto llamado arreglo que permite representar colecciones de datos en varias dimensiones [132].

- **PyTorch** es una biblioteca de tensores optimizada para el aprendizaje profundo utilizando GPU y CPU [133]. Se emplea para crear redes neuronales, combinando la biblioteca de ML de Torch con una API de alto nivel basada en Python.

  Es uno de los marcos de ML destinados a comunidades académicas y de investigación por su flexibilidad y sencillez de uso, además de admitir una amplia variedad de estructuras de redes neuronales [134].

- **PyQt5 / PySide6** son herramientas que permiten usar Python como un lenguaje alternativo de desarrollo de aplicaciones a C++ en todas las plataformas soportadas [135]. PyQt5 también puede integrarse en aplicaciones basadas en C++ para permitir a los usuarios de dichas software configurar o mejorar la funcionalidad de las mismas, mientras que PySide6 ofrece compatibilidad oficial con las bibliotecas Qt desarrolladas por The Qt Company [136].

- **pybind11** es una biblioteca de C++ diseñada para exponer tipos y funciones de C++ en Python y viceversa, facilitando la creación de extensiones de Python basadas en código C++ [124]. Puede generar enlaces entre ambos lenguajes de manera compacta, eficiente y con un código más simple, posibilitando así integrar cálculos de alto rendimiento escritos en C++ dentro de entornos Python.

- **Matplotlib** De acuerdo con [137], Matplotlib es una librería de Python especializada en la creación de gráficos en dos dimensiones.

  Además, mencionan que permite crear y personalizar los gráficos más comunes, entre los que se encuentran:

  - Diagramas de barras.
  - Histograma.
  - Diagramas de sectores.
  - Diagramas de caja y bigotes.

- **Scikit-learn** es una librería que cuenta con algoritmos de clasificación, regresión, clustering y reducción de dimensionalidad, etc. Es compatible con otras librerías de Python [138].

6.9.1.4. Wolfram Mathematica

Es un sistema de computación cientifica que sigue el paradigma de programación simbolica. Fue desarrollado por Wolfram Research y se especializa en manejar

matemáticas complejos, algoritmos de visualización y manipulación de datos de forma nativa [139]. Algunas de sus funcionalidades son:

- Visualización: Genera gráficos en 2D y 3D, mapas y diagramas dinámicos.
- Procesamiento de Datos: Herramientas para aprendizaje automático, procesamiento de imágenes y señales.
- Documentos Interactivos: Se trabaja en cuadernos, los cuales mezclan código ejecutable, texto enriquecido, imágenes y modelos interactivos.

Adicionalmente, en Wolfram Mathematica existen funciones nativas para construir, evolucionar, manipular y analizar autómatas celulares. Por lo que su uso como medio de referencia en la construcción del simulador y como herramienta de análisis de datos resulta fundamental (Wolfram Mathematica no tiene interacción directa con el simulador, unicamente se usa para el estudio de los resultados obtenidos mediante el simulador).

6.9.2. Entornos de trabajo

El desarrollo del presente proyecto requiere de un entorno flexible y bien estructurado que facilite tanto la construcción como la validación de los distintos componentes del sistema. Para ello, se han seleccionado diversas plataformas que permiten organizar tareas, escribir y probar código, almacenar datos, así como documentar y presentar avances de manera clara y profesional.

6.9.2.1. Plataformas de desarrollo

Con el fin de cubrir las distintas etapas del proyecto, desde la implementación hasta la comunicación de resultados. Las siguientes plataformas han sido clave para mantener un flujo de trabajo continuo y bien coordinado.

- **Visual Studio Code** Es un editor de código gratuito, de código abierto y compatible con múltiples sistemas operativos. Fue creado por Microsoft, empresa con amplia trayectoria en el desarrollo de entornos integrados (IDE), lo que se refleja en la eficiencia y funcionalidad que ofrece esta herramienta, muy valorada por la comunidad de desarrolladores [140].

Aunque su nombre puede sugerir una relación directa con Visual Studio, en realidad Visual Studio Code es una aplicación independiente, construida desde cero con tecnologías diferentes. Mientras Visual Studio se orienta principalmente a tecnologías propias de Microsoft, VSCode destaca por su flexibilidad y capacidad de adaptarse a prácticamente cualquier lenguaje de programación [141].

- **Github** Es una plataforma basada en la nube donde puedes almacenar, compartir, trabajar y versionar junto con otros usuarios para escribir código [142]. En adición a lo anterior, también permite realizar lo siguiente:

  - Presentar o compartir el trabajo.
  - Seguir y administrar los cambios en el código a lo largo del tiempo.
  - Dejar que otros usuarios revisen el código y realicen sugerencias para mejorarlo.
  - Colaborar en un proyecto compartido, sin preocuparse de que los cambios afectarán al trabajo de los colaboradores antes de que esté listo para integrarlos.

  El versionamiento, una de las características fundamentales de GitHub, es posible gracias al software de código abierto Git, en el que se basa GitHub [142].

- **Canva** fue lanzada en 2013, Canva es una herramienta de diseño y publicación online que tiene como misión poner el poder del diseño al alcance de todo el mundo, para que cualquier persona pueda crear lo que quiera y publicarlo donde quiera [143].

- **TexStudio** es un entorno de escritura integrado para la creación de documentos LaTeX. Cuenta con numerosas características como resaltado de sintaxis, visor integrado, comprobación de referencias y varios asistentes. TeXstudio es de código abierto y está disponible para todos los principales sistemas operativos [144].

- **draw.io** es una herramienta de software gratuita diseñada para la creación de diagramas. Se utiliza para modelado UML, documentación de proyectos y gestión de flujos [145].

Habiendo definido el enfoque metodológico, justificado las decisiones técnicas del estudio y contextualizado la tecnología utilizada, se cuenta con una base sólida para la implementación y ejecución de los experimentos. Esto permite garantizar

que las simulaciones se realicen bajo condiciones controladas, que los datos recolectados sean confiables y que los análisis posteriores aporten resultados válidos y pertinentes para responder a las preguntas de investigación planteadas.

Capítulo 7

Análisis de software

Hasta el momento conocemos la metodología utilizada para la investigación completa; sin embargo, es necesario especificar el desarrollo del software, ya que es necesario garantizar el correcto diseño e implementación del simulador para poder realizar las pruebas adecuadas con el menor número de errores. Es por esto que en este capítulo se detalla el proceso de desarrollo del simulador, lo que incluye la especificación de la metodología utilizada, requerimientos del sistema y modelado, diseño e implementación de cada una de las entregas.

7.1. Metodología de desarrollo

Para el desarrollo de software se utiliza un modelo de entrega por etapas. Este modelo, como se observa en la figura 7.1, realiza las primeras etapas de forma secuencial y permite diseñar e implementar los componentes del proyecto por separado. Seguir este modelo permite entregar partes del proyecto y tener visibilidad del progreso.

Estas son algunas de las razones por las cuales se opta por este modelo:

- El tamaño del equipo de desarrollo es reducido: Al tener un número de integrantes reducido, resulta inviable desarrollar múltiples componentes de forma paralela.
- No existe un cliente: El desarrollo de este producto de software es únicamente un medio para el proceso experimental.

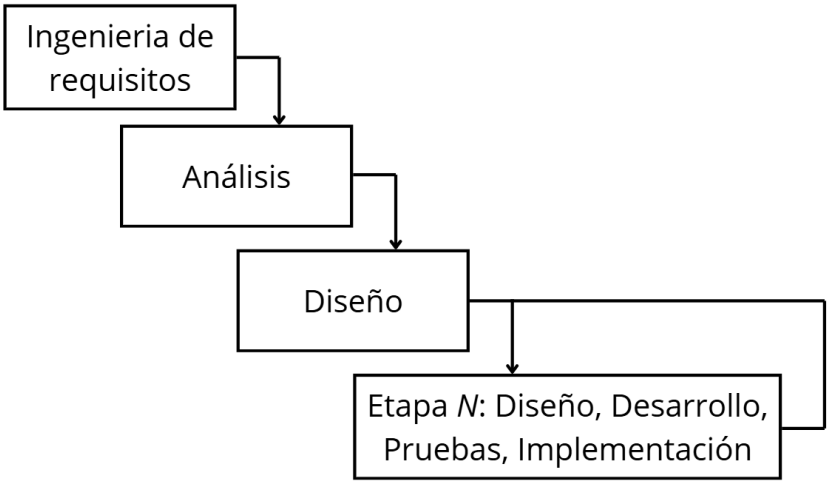


Figura 7.1: Representación gráfica del modelo de entrega por etapas para el desarrollo del software

- No es un producto comercial: No se planea monetizar dado que no existe un cliente. Por lo que en la ingeniería de requisitos se consideran los necesarios propuestos por el equipo de investigación para satisfacer las necesidades tecnológicas.
- No hay procesos iterativos: Al no tener cliente, no es necesario tener una validación externa, ni cambios significativos en los requerimientos del software.

- Simulador funcional del cualquier autómata celular en dos dimensiones en un espacio celular finito.
- Personalización de la vecindad, conjunto de estados, condición de frontera y función de transición.
- Personalización del diseño de estados e interacción en tiempo real con el espacio celular.
- Controles de simulación para avanzar, retroceder, reiniciar y manipular la velocidad de la animación.

Para mayor detalle de esta primer etapa, mas adelante se especifican los casos de uso.

7.2. Reglas de negocio

RN1 El estado de una celda en el tiempo $t + 1$ depende únicamente de su estado actual y de los estados de sus vecinos.

RN2 Para iniciar la simulación es necesario que el conjunto de estados, la topología de la vecindad, la condición de frontera y la función de transición hayan sido definidas.

RN3 Cuando la simulación está en curso, no se puede cambiar el conjunto de estados, la topología de la vecindad ni la función de transición.

RN4 El usuario puede manipular el estado de una célula y la condición de frontera en tiempo de ejecución.

RN5 Los datos importados deben cumplir con el formato definido por el sistema para garantizar su correcta interpretación.

RN6 Los datos exportados deben conservar el mismo formato que los datos importados.

RN7 El usuario puede pausar, detener, reanudar o retroceder la simulación sin pérdida de información.

RN8 Solo se permite una simulación activa por sesión.

7.3. Requerimientos

7.3.1. Requerimientos funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe ofrecer al usuario para cumplir con los objetivos establecidos.

RF1 El sistema debe mostrar una cuadrícula regular y finita que represente el espacio celular.

RF2 El sistema debe permitir al usuario interactuar con el espacio celular, asignando o modificando los estados de las celdas.

RF3 El sistema debe permitir al usuario definir el conjunto de estados, la topología de la vecindad local y las condiciones de frontera del espacio celular.

RF4 El sistema debe permitir la evolución del espacio celular mediante un único paso o de forma continua, ajustando la velocidad de actualización según las capacidades computacionales del sistema.

RF5 El sistema debe permitir la importación y exportación de configuraciones y evoluciones del espacio celular.

RF6 El sistema debe permitir al usuario establecer la función de transición de manera manual.

RF7 El sistema debe permitir configurar la red neuronal para imitar el comportamiento de una evolución señalada.

RF8 El sistema debe entrenar una red neuronal dados los parámetros del usuario y la evolución que desea replicar.

7.3.2. Requerimientos no funcionales

Los requerimientos no funcionales especifican las condiciones de calidad, desempeño y compatibilidad bajo las cuales el sistema debe operar.

RNF1. El sistema debe poder ejecutarse en Fedora 42 sin requerir modificaciones en el código fuente.

RNF2. El sistema debe ser capaz de procesar al menos 10,000 celdas o registros por segundo utilizando una GPU NVIDIA RTX 5080.

RNF3. El sistema debe garantizar disponibilidad continua durante la ejecución, sin fallos o interrupciones críticas.

RNF4. La interfaz del sistema debe tener mensajes claros y funcionalidades bien definidas.

7.3.3. Requerimientos de sistema

7.3.3.1. Recomendado

Los siguientes requerimientos describen el entorno principal y recomendado de ejecución, en el cual el sistema garantiza un rendimiento óptimo y estabilidad durante los procesos de simulación:

- **Procesador:** Intel Core i9-14900K × 32 núcleos.
- **GPU:** NVIDIA GeForce RTX 5080.
- **Sistema operativo:** Fedora Linux 42 (Workstation Edition).
- **Entorno de escritorio:** GNOME 48.
- **Sistema de ventaneo:** Wayland.

7.3.3.2. Mínimo

También se establecen los requisitos mínimos necesarios para usar el simulador, teniendo en cuenta que el rendimiento y la velocidad de procesamiento dependerán de las capacidades del hardware disponible y que el entrenamiento podría no ser posible.

- **Procesador:** Intel Core i7-1185G7 × 4 núcleos.
- **GPU:** Intel Corporation Raptor Lake-S GT1 [UHD Graphics 770].
- **Sistema operativo:** Fedora Linux 42 (Workstation Edition).
- **Entorno de escritorio:** GNOME 48.
- **Sistema de ventaneo:** Wayland.

7.4. Análisis de riesgos

El análisis de riesgos se realiza tomando en cuenta el tiempo estimado de desarrollo de aproximadamente 6 meses, los recursos disponibles para el desarrollo tecnológico y las habilidades técnicas para el diseño de simuladores de autómatas celulares, cómputo paralelo, redes neuronales y programación orientada a eventos.

7.4.1. Identificación de riesgos

7.4.1.1. Riesgos técnicos

RT1 Incompatibilidad entre diferentes lenguajes de programación: los módulos desarrollados en Python, C++ y CUDA generan errores de integración al

intercambiar información y sincronizar procesos.

RT2 Indisponibilidad de hardware: La tarjeta gráfica no tiene el rendimiento suficiente.

RT3 Incompatibilidad entre tecnologías: El sistema operativo, los lenguajes de programación y los elementos de hardware no son compatibles para el desarrollo y ejecución.

RT4 Dependencias de librerías: La instalación y configuración de diferentes librerías varía según el sistema operativo.

RT5 Mala visualización: El renderizado de los componentes gráficos es incorrecto o está mal optimizado.

7.4.1.2. Riesgos operativos

RO1 Documentación deficiente: La documentación no permite mantener ni ampliar el proyecto a futuro.

RO2 Problemas personales: Falta de motivación o problemas de salud física o mental se presentan durante el desarrollo.

RO3 Curva de aprendizaje elevada: Se encuentra una curva de aprendizaje elevada y se carece de experiencias con la tecnología a utilizar.

7.4.1.3. Riesgos de requerimientos

RR1 Requerimientos ambiguos: La definición incompleta o ambigua de los requerimientos puede provocar cambios en etapas avanzadas del desarrollo.

7.4.1.4. Riesgos de estimación

RE1 Subestimación del tiempo de desarrollo: La complejidad de enlazar diferentes tecnologías puede extender los plazos planeados.

7.4.1.5. Riesgos de alcance

RA1 Limitación del alcance funcional: Las restricciones de tiempo pueden dar pie a limitar el número de funcionalidades requeridas.

RA2 Carga de trabajo imprevista: El poder de cómputo resulta insuficiente, por lo que se reduce la complejidad de las simulaciones.

7.4.2. Probabilidad e impacto

Siguiendo la tabla 7.1, a continuación se enlistan los niveles de riesgo para cada riesgo identificado.

		Probabilidad				
		Nada probable 10 %	Levemente probable 30 %	Poco probable 50 %	Probable 70 %	Muy probable 90 %
Severidad	Nula	Bajo	Bajo	Bajo	Medio	Medio
	Menor	Bajo	Bajo	Medio	Medio	Medio
	Moderado	Medio	Medio	Medio	Alto	Alto
	Crítico	Medio	Medio	Alto	Alto	Muy alto
	Fatal	Medio	Alto	Alto	Muy alto	Muy alto

Tabla 7.1: Matriz de riesgos [146]

Riesgo técnico	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Incompatibilidad entre diferentes lenguajes de programación	Poco probable	Crítico	Alto
Indisponibilidad de hardware	Levemente probable	Moderado	Medio
Incompatibilidad entre tecnologías	Probable	Fatal	Muy alto
Dependencias de librerías	Levemente probable	Menor	Bajo
Mala visualización	Poco probable	Menor	Medio

Tabla 7.2: Nivel de riesgos técnicos

Riesgo operativo	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Documentación deficiente	Probable	Moderado	Alto
Problemas personales	Poco probable	Crítico	Alto
Curva de aprendizaje elevada	Muy probable	Moderado	Alto

Tabla 7.3: Nivel de riesgos operativos

Riesgo de requerimientos	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Requerimientos ambiguos	Levemente probable	Menor	Bajo

Tabla 7.4: Nivel de riesgos de requerimientos

Riesgo de estimación	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Subestimación del tiempo de desarrollo	Probable	Fatal	Muy alto

Tabla 7.5: Nivel de riesgos de estimación

Riesgo de alcance	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Limitación del alcance funcional	Poco probable	Moderado	Medio
Carga de trabajo imprevista	Probable	Crítico	Alto

Tabla 7.6: Nivel de riesgos de alcance

7.4.3. Plan de mitigación

Riesgos técnicos	
Incompatibilidad entre diferentes lenguajes	Verificar la documentación de compatibilidad entre los lenguajes de programación y realizar pruebas de transferencia de datos y sincronización previas al desarrollo.
Indisponibilidad de hardware	Asegurar la disponibilidad de equipos de respaldo o entornos virtuales alternativos para continuar el desarrollo en caso de fallos de hardware.
Incompatibilidad entre tecnologías	Realizar pruebas de compatibilidad del hardware y software en diferentes sistemas operativos y versiones antes de la integración.
Dependencias de librerías	Documentar y verificar las versiones de librerías utilizadas, además de considerar alternativas equivalentes que garanticen continuidad en caso de incompatibilidad.
Mala visualización	Realizar pruebas de usabilidad y ajustar la interfaz de acuerdo con las resoluciones de pantalla y dispositivos objetivo.

Tabla 7.7: Plan de mitigación de riesgos técnicos

Riesgos operativos	
Documentación deficiente	Establecer un estándar de documentación y realizar revisiones periódicas del avance para garantizar la calidad y completitud de los registros técnicos.
Problemas personales	Definir roles de respaldo o distribuir responsabilidades para minimizar el impacto de ausencias o contratiempos personales.
Curva de aprendizaje elevada	Programar capacitaciones iniciales y establecer un periodo de adaptación con apoyo técnico y materiales de referencia.

Tabla 7.8: Plan de mitigación de riesgos operativos

Riesgos de requerimientos	
Requerimientos ambiguos	Realizar reuniones periódicas con los usuarios o stakeholders para aclarar requerimientos y validar el entendimiento del equipo de desarrollo.

Tabla 7.9: Plan de mitigación de riesgos de requerimientos

Riesgos de estimación	
Subestimación del tiempo de desarrollo	Dividir el proyecto en fases pequeñas y manejables, aplicando seguimiento semanal y reajuste de cronograma según los resultados obtenidos.

Tabla 7.10: Plan de mitigación de riesgos de estimación

Riesgos de alcance	
Limitación del alcance funcional	Definir claramente los objetivos y alcances desde el inicio, con aprobación formal de los responsables del proyecto antes de iniciar nuevas funcionalidades.
Carga de trabajo imprevista	Implementar revisiones periódicas del plan de trabajo y ajustar el alcance en función de los recursos disponibles y los resultados obtenidos.

Tabla 7.11: Plan de mitigación de riesgos de alcance

7.4.4. Plan de contingencia

A continuación se describen los planes de contingencia para los riesgos de nivel fatal:

RT3 Incompatibilidad entre tecnologías.

RE1 Subestimación del tiempo de desarrollo.

Nombre del riesgo	**Incompatibilidad entre tecnologías**
Probabilidad	Probable
Impacto	Fatal
Plan de evasión	Realizar pruebas de integración tempranas entre las distintas tecnologías implicadas. Documentar las versiones, dependencias y configuraciones utilizadas para garantizar compatibilidad. Establecer un entorno de desarrollo unificado que minimice diferencias entre plataformas.
Mitigación	Sustituir o actualizar los componentes que generen incompatibilidad. Implementar adaptadores o módulos intermedios que permitan la interoperabilidad entre sistemas. Reasignar recursos técnicos para resolver los conflictos de integración y evitar retrasos prolongados.

Tabla 7.12: Plan de contingencia para incompatibilidad entre tecnologías

Nombre del riesgo	**Subestimación del tiempo de desarrollo**
Probabilidad	Probable
Impacto	Fatal
Plan de evasión	Realizar estimaciones realistas basadas en experiencias previas y tareas medibles. Incluir márgenes de tiempo de reserva en el cronograma para absorber imprevistos. Efectuar reuniones de seguimiento semanales para detectar desviaciones tempranas.
Mitigación	Reasignar recursos o personal adicional para recuperar el tiempo perdido. Reducir temporalmente el alcance funcional o posponer características no críticas. Reajustar el cronograma de entregas y comunicar los cambios al equipo y partes interesadas.

Tabla 7.13: Plan de contingencia para subestimación del tiempo de desarrollo

7.5. Estudio de factibilidad

El proyecto cuenta con los recursos de hardware necesarios para llevar a cabo simulaciones de alto nivel computacional. El equipo de cómputo disponible permite implementar técnicas de paralelización, asegurando la capacidad de realizar múltiples inferencias en tiempos razonables. Adicionalmente, se dispone de herramientas tecnológicas de alto rendimiento que brindan la ventaja computacional requerida para ejecutar la simulación de manera eficiente.

En cuanto a la factibilidad económica, la mayor parte del software utilizado es de libre acceso, incluyendo Python, C++, Visual Studio Code y TeXstudio. Se utiliza también Wolfram Mathematica, que requiere una licencia anual de 195 dólares, la cual se encuentra disponible de manera gratuita para estudiantes del IPN, garantizando así la viabilidad financiera del proyecto.

Desde el punto de vista operativo, el diseño de la interfaz facilita la definición de reglas y la ejecución de simulaciones, optimizando el flujo de trabajo del grupo de investigadores. Esto permite acelerar significativamente el proceso de experimentación y análisis de resultados, maximizando la eficiencia del equipo durante el desarrollo del proyecto.

Finalmente, se cumple la factibilidad temporal, ya que el cronograma de desarrollo estimado permite realizar todas las simulaciones necesarias para el análisis posterior, asegurando que el proyecto se pueda completar dentro del tiempo previsto.

7.6. Casos de uso

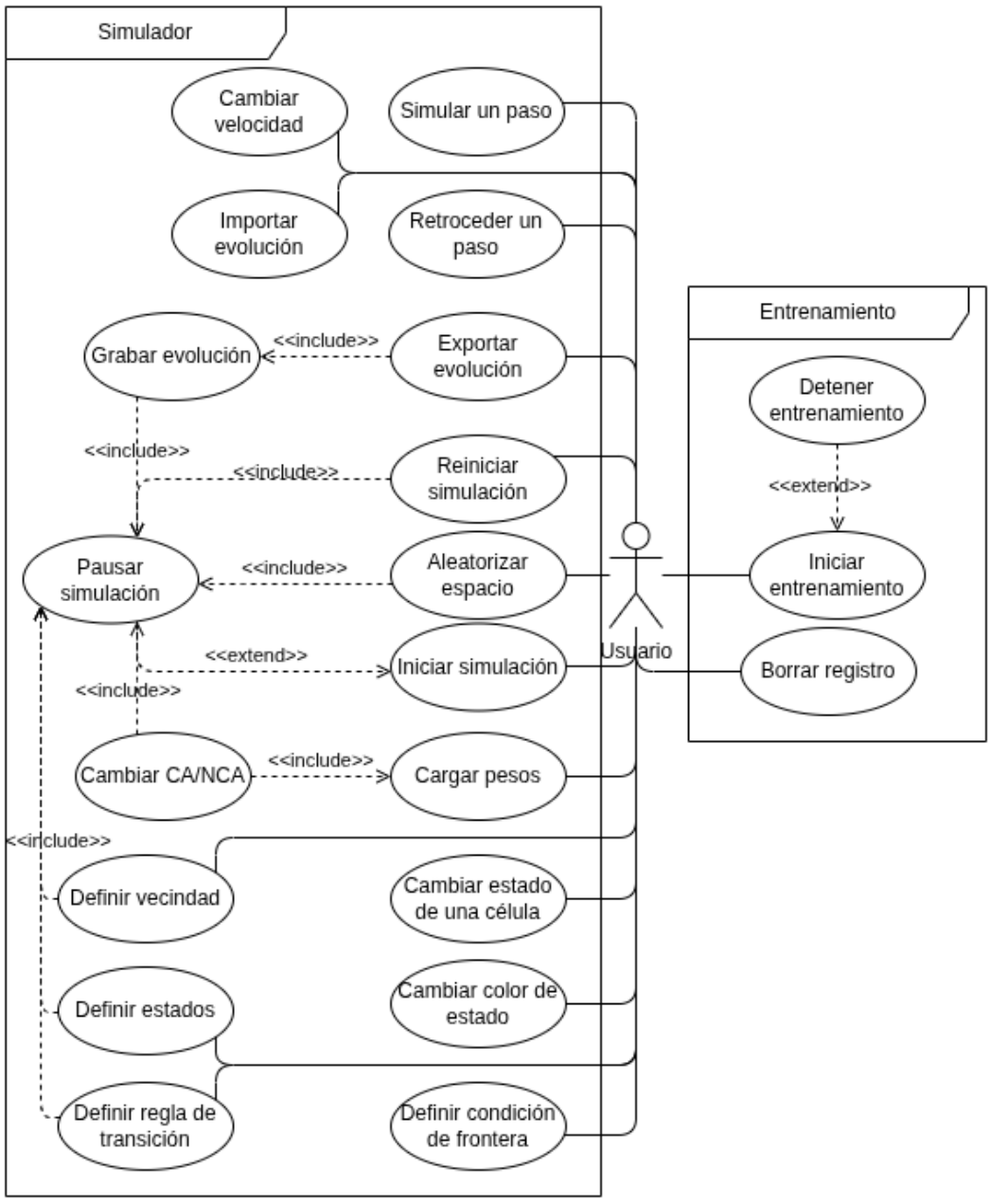


Figura 7.2: Diagrama de casos de uso

7.7. Diccionario

7.7.1. Interfaces gráficas

- UI1 Control: Elemento que contiene los botones de control sobre la simulación del espacio celular.
- UI2 Simulación: Elemento que contiene la cuadrícula del espacio celular y muestra la evolución del autómata celular.
- UI3 Configuración: Elemento que permite realizar la configuración de la vecindad, condición de frontera, estados y función de transición.
- UI4 Entrenamiento: Contiene la personalización del entrenamiento del autómata celular neuronal.

7.7.2. Entradas

- Controles de simulación:
  - BTN1 Iniciar simulación
  - BTN2 Pausar simulación
  - BTN3 Simular un paso
  - BTN4 Retroceder un paso
  - BTN5 Reiniciar simulación
  - BTN6 Aleatorizar espacio
  - BTN7 Importar evolución
  - BTN8 Grabar evolución
  - BTN9 Exportar evolución
  - BTN10 Cargar pesos
  - BTN11 Cambiar CA/NCA
  - SLD1 Cambiar velocidad
  - SLD2 Eligir paso
  - IN1 Numero de pasos
  - MSG1 Formato de archivo incorrecto
  - MSG2 Tamaño incorrecto

    - MSG3 Sin evolución

- Configuración del autómata celular:
    - BTN12 Definir vecindad
    - BTN13 Añadir grupo
    - BTN14 Eliminar grupo
    - BTN15 Definir estados
    - BTN16 Estado n-esimo.
    - BTN17 Definir condición de frontera
    - BTN18 Definir regla
    - IN2 Grupo de vecindad
    - IN3 Espacio vecindad
    - IN4 Estados
    - IN5 Regla
    - IN6 Regla (Código)
- Espacio de simulación:
    - IN7 Espacio
- Entrenamiento del automata celular neuronal:
    - BTN19 Cargar evolución
    - BTN20 Definir ruta de guardado
    - BTN21 Iniciar entrenamiento
    - BTN22 Detener entrenamiento
    - IN8 Vecindad
    - IN9 Capas ocultas
    - IN10 Neuronas
    - IN11 Activación ReLU
    - IN12 Activación sigmoide
    - IN13 Activación Tanh
    - IN14 Activación lineal
    - IN15 Capa de salida

- IN16 Ultima función de activación
- IN17 Epocas
- IN18 Tamaño de lotes
- IN19 Tasa de aprendizaje
- IN20 Perdida objetivo
- IN21 Imprimir cada N
- IN22 Borrar consola
- MSG4 Entrenamiento incompleto
- OUT1 Ruta de evolución
- OUT2 Ruta de guardado
- OUT3 Mapa de vecindad
- OUT4 Consola

7.8. Especificaciones de casos de uso

7.8.1. Controles de simulación

Los siguientes casos de uso permiten al usuario controlar la evolución del autómata celular. Esto implica evolucionar la sistema hacia adelante, hacia atrás y definiendo velocidades.

UC01 Iniciar simulación

Actor	Usuario.
Descripción	El sistema inicia la evolución en pasos de tiempo constante.
Tipo	Primario.
Entradas	▪ Desde mouse BTN1 Iniciar simulación.
Salidas	Ninguna.
Precondición	▪ Simulación detenida. ▪ Botón BTN1 Iniciar simulación habilitado.
Postcondición	▪ Simulación en ejecución. ▪ Botón BTN1 Iniciar simulación deshabilitado. ▪ Botón BTN2 Pausar simulación habilitado.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN1 Iniciar simulación desde UI1 Control.	2. Deshabilita el botón BTN1 Iniciar simulación. 3. Habilita el botón BTN2 Pausar simulación. 4. Inicia la simulación. Cada paso evolutivo se realiza a una velocidad constante. 5. Precarga evoluciones futuras en una cola. 6. Actualiza de forma constante UI2 Simulación con la evolución del sistema.

UC02 Pausar simulación

Actor	Usuario.
Descripción	Se detiene la simulación.
Tipo	Secundario.
Entradas	▪ Desde mouse BTN2 Pausar simulación.
Salidas	Ninguna.
Precondición	▪ Simulación en ejecución. ▪ Botón BTN2 Pausar simulación habilitado.
Postcondición	▪ Simulación detenida. ▪ Botón BTN2 Pausar simulación deshabilitado. ▪ Botón BTN1 Iniciar simulación habilitado.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN2 Pausar simulación desde UI1 Control.	2. Detiene la simulación en UI2 Simulación. 3. Deshabilita el botón BTN2 Pausar simulación. 4. Habilita el botón BTN1 Iniciar simulación.

UC03 Cambiar velocidad

Actor	Usuario.
Descripción	Se cambia la velocidad de actualización del autómata celular.
Tipo	Secundario.
Entradas	▪ Desde mouse SLD1 Cambiar velocidad.
Salidas	Ninguna
Precondición	Ninguna
Postcondición	▪ Cambia la velocidad de simulación.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona SLD1 Cambiar velocidad desde UI1 Control y desliza a la velocidad deseada.	2. Cambia la velocidad de la simulación a la velocidad seleccionada.

UC04 Simular un paso

Actor	Usuario.
Descripción	Se realiza la simulación del CA, pero únicamente en un paso.
Tipo	Primario.
Entradas	▪ Desde mouse BTN3 Simular un paso.
Salidas	Ninguna.
Precondición	Ninguna.
Postcondición	▪ Evoluciona el sistema.
Errores	▪ Velocidad máxima: Se intenta dar un paso cuando la simulación está activa en la velocidad máxima.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN3 Simular un paso desde UI1 Control.	2. La simulación avanza un paso. 3. Actualiza UI2 Simulación.

Flujo alterno 1: Velocidad máxima	
Actor	**Sistema**
1.1. Presiona el botón BTN3 Simular un paso desde UI1 Control cuando la simulación esta activa y la velocidad esta en la velocidad máxima.	1.2. Agrega la simulación a una cola y mantiene la velocidad actual. 1.3. Actualiza UI2 Simulación.

UC05 Retroceder un paso

Actor	Usuario.
Descripción	El sistema restaura el estado anterior de la simulación.
Tipo	Secundario.
Entradas	▪ Desde mouse BTN4 Retroceder un paso.
Salidas	Ninguna.
Precondición	▪ Existe un estado previo.
Postcondición	▪ El sistema restaura la evolución pasada.
Errores	▪ Configuración inicial: El usuario intenta retroceder la simulación de la configuración inicial.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN4 Retroceder un paso desde UI1 Control.	2. La simulación retrocede un paso. 3. Actualiza UI2 Simulación.

Flujo alterno 1: Configuración inicial	
Actor	**Sistema**
1.1. Presiona el botón BTN4 Retroceder un paso desde UI1 Control estando en la configuración inicial (no hay más configuraciones antes).	1.2. No realiza ningún cambio.

UC06 Reiniciar simulación

Actor	Usuario.
Descripción	Restablece la simulación a su estado inicial (todas las celdas vacías).
Tipo	Secundario.
Entradas	▪ Desde mouse BTN5 Reiniciar simulación.
Salidas	Ninguna.
Precondición	Ninguna.
Postcondición	▪ El sistema adquiere la configuración nula.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN5 Reiniciar simulación desde UI1 Control.	2. Regresa la configuración a la última versión modificada mientras estaba en pausa. 3. Elimina el historial de evoluciones. 4. Actualiza UI2 Simulación.

UC07 Aleatorizar espacio

Actor	Usuario.
Descripción	Establece una configuración aleatoria en el espacio.
Tipo	Secundario.
Entradas	▪ Desde mouse BTN6 Aleatorizar espacio.
Salidas	Ninguna.
Precondición	Ninguna.
Postcondición	▪ El sistema adquiere una configuración aleatoria.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN6 Aleatorizar espacio desde UI1 Control.	2. La configuración del espacio celular adquiere valores aleatorios según el numero de estados definidos. 3. Elimina el historial de evoluciones. 4. Actualiza UI2 Simulación.

UC08 Importar evolución

Actor	Usuario.
Descripción	Importa una evolución guardada en formato "npy".
Tipo	Secundario.
Entradas	▪ Desde mouse BTN7 Importar evolución.
Salidas	▪ MSG1 Formato de archivo incorrecto. ▪ MSG2 Tamaño incorrecto.
Precondición	Ninguna.
Postcondición	▪ El sistema carga la simulación y la muestra en UI2 Simulación.
Errores	▪ Formato invalido: El archivo no coincide. ▪ Tamaño no compatible: El tamaño no coincide.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN7 Importar evolución desde UI1 Control. 3. Selecciona el archivo "npy" que contenga una evolución.	2. Abre el explorador de archivos. 4. Carga la evolución. 5. Actualiza UI2 Simulación.

Flujo alterno 1: Formato invalido	
Actor	**Sistema**
1.1 Presiona el botón BTN7 Importar evolución desde UI1 Control. 1.3. Selecciona un archivo "npy" invalido u otro tipo de archivo.	1.2. Abre el explorador de archivos. 1.4. No realiza ningún cambia y muestra MSG1 Formato de archivo incorrecto.

Flujo alterno 2: Tamaño no compatible	
Actor	**Sistema**
2.1. Presiona el botón BTN7 Importar evolución desde UI1 Control. 2.3. Selecciona el archivo "npy" que contenga una evolución con un tamaño diferentes a UI2 Simulación.	2.2. Abre el explorador de archivos. 2.4. Recorta o centra la evolución y muestra el mensaje MSG2 Tamaño incorrecto. 2.5. Actualiza UI2 Simulación.

UC09 Grabar evolución

Actor	Usuario.
Descripción	Evoluciona el sistema sin visualización y guarda todos los pasos.
Tipo	Secundario.
Entradas	▪ Desde mouse BTN8 Grabar evolución. ▪ Desde teclado IN1 Numero de pasos
Salidas	Ninguna.
Precondición	Ninguna.
Postcondición	▪ El sistema evoluciona los pasos seleccionados.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN8 Grabar evolución desde UI1 Control. 3. Elige un numero con IN1 Numero de pasos.	2. Abre una ventana preguntando el numero de pasos a evolucionar. 4. Evoluciona en sistema el numero de pasos. 5. Muestra en UI2 Simulación el ultimo paso.

UC10 Exportar evolución

Actor	Usuario.
Descripción	Exporta la evolución en formato “npy” y “mat”.
Tipo	Primario.
Entradas	▪ Desde mouse BTN9 Exportar evolución.
Salidas	▪ Evolución en formato “npy”. ▪ Evolución en formato “mat”.
Precondición	UC09 Grabar evolución.
Postcondición	Ninguna.
Errores	▪ Sin evolución: No se ha grabado ninguna evolución.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN9 Exportar evolución desde UI1 Control. 3. Selecciona la ruta de guardado archivo.	2. Abre el explorador de archivos. 4. Crea los archivos de la evolución en la ruta especificada.

Flujo alterno 1: Sin evolución	
Actor	**Sistema**
1.1. Presiona el botón BTN9 Exportar evolución desde UI1 Control sin haber una evolución.	1.2. No realiza ningún cambio y muestra MSG3 Sin evolución.

UC11 Cargar pesos

Actor	Usuario.
Descripción	Carga los pesos de la red neuronal entrenada en el autómata celular neuronal.
Tipo	Primario.
Entradas	▪ Desde mouse BTN10 Cargar pesos.
Salidas	Ninguna.
Precondición	Ninguna.
Postcondición	▪ Habilita el botón BTN11 Cambiar CA/NCA.
Errores	▪ Formato invalido: El archivo no coincide.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN10 Cargar pesos desde UI1 Control. 3. Selecciona los pesos del archivo "npz".	2. Abre el explorador de archivos. 4. Carga los pesos del autómata celular neuronal. 5. habilita el botón BTN11 Cambiar CA/NCA.

Flujo alterno 1: Formato invalido	
Actor	**Sistema**
1.1. Presiona el botón BTN10 Cargar pesos desde UI1 Control. 1.3. Selecciona un archivo "npz" invalido u otro tipo de archivo.	1.2. Abre el explorador de archivos. 1.4. No realiza ningún cambia y muestra MSG1 Formato de archivo incorrecto.

UC12 Cambiar CA/NCA

Actor	Usuario.
Descripción	Cambia la evolución del sistema entre el autómata celular y el autómata celular neuronal.
Tipo	Secundario.
Entradas	▪ Desde mouse BTN11 Cambiar CA/NCA.
Salidas	Ninguna.
Precondición	UC11 Cargar pesos.
Postcondición	Ninguna.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN11 Cambiar CA/NCA desde UI1 Control.	2. Cambia entre autómata celular y autómata celular neuronal.

7.8.2. Configuración del autómata celular

Estos casos de uso definen las características generales del autómata celular, tales como el numero de estados, vecindad, comportamiento de frontera y la función de transición.

UC13 Definir vecindad

Actor	Usuario.
Descripción	Se define el conjunto de posiciones relativas de cada célula que forman parte de la vecindad.
Tipo	Primario.
Entradas	▪ Desde mouse BTN12 Definir vecindad. ▪ Desde mouse BTN13 Añadir grupo. ▪ Desde mouse BTN14 Eliminar grupo. ▪ Desde mouse IN2 Grupo de vecindad. ▪ Desde mouse IN3 Espacio vecindad.
Salidas	▪ Se actualiza IN6 Regla (Código).
Precondición	Ninguna.
Postcondición	▪ Vecindad actualizada en el sistema. ▪ Simulación en pausa.
Errores	▪ Vecindad vaciá: No se selecciono ningún elemento en IN3 Espacio vecindad.

Flujo básico	
Actor	**Sistema**
1. Presiona el IN2 Grupo de vecindad en UI3 Configuración. 2. Selecciona múltiples elementos del IN3 Espacio vecindad. 4. Presiona el botón BTN12 Definir vecindad.	3. Asigna el grupo seleccionado al elemento de IN3 Espacio vecindad. 5. Detiene la simulación. 6. Asigna la nueva vecindad y actualiza el tamaño de reglas permitidas. 7. Actualiza IN6 Regla (Código).

Flujo alterno 1: Vecindad vacía	
Actor	**Sistema**
1.2. Selecciona el grupo vació. 1.4. Presiona el botón BTN12 Definir vecindad.	1.3. Asigna el grupo a todos los elementos de IN3 Espacio vecindad. 1.5. No detiene la animación ni actualiza la vecindad.

Flujo alterno 2: Añadir grupo	
Actor	**Sistema**
2.1 Presiona el botón BTN13 Añadir grupo.	2.2 Añade un nuevo IN2 Grupo de vecindad.

Flujo alterno 3: Eliminar grupo	
Actor	**Sistema**
3.1 Presiona el botón BTN14 Eliminar grupo.	3.2. Borra todas las celdas en IN3 Espacio vecindad asignadas al ultimo IN**??** . 3.3. Elimina el ultimo IN2 Grupo de vecindad.

UC14 Definir estados

Actor	Usuario.
Descripción	Se define el numero de estados.
Tipo	Secundario.
Entradas	▪ Desde teclado IN4 Estados. ▪ Desde mouse BTN15 Definir estados.
Salidas	▪ Se añaden BTN16 Estado n-esimo.. ▪ Se actualiza IN6 Regla (Código).
Precondición	Ninguna.
Postcondición	▪ Estados actualizados en el sistema. ▪ Simulación en pausa. ▪ Botones de estados añadidos.
Errores	▪ Invalido: El numero de estados ingresados es invalido.

Flujo básico	
Actor	**Sistema**
1. Ingresa el numero de estados en IN4 Estados en UI3 Configuración y presiona el botón BTN15 Definir estados.	2. Se detiene la simulación. 3. Se añaden BTN16 Estado n-esimo. definidos. 4. Actualiza el tamaño de las reglas permitidas. 5. Actualiza IN6 Regla (Código).

Flujo alterno 1: invalido	
Actor	**Sistema**
1.1. Ingresa el numero de estados en IN4 Estados invalido.	1.2. Corrige el numero al valor mas cercano posible.

UC15 Definir condición de frontera

Actor	Usuario.
Descripción	Se selecciona entre diferentes opciones de condiciones de frontera.
Tipo	Secundario.
Entradas	▪ Desde mouse BTN17 Definir condición de frontera.
Salidas	Ninguna.
Precondición	Ninguna.
Postcondición	▪ Condición actualizada en el sistema.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN17 Definir condición de frontera desde UI3 Configuración.	2. Actualiza la condición de frontera a la siguiente configuración.

UC16 Definir regla de transición

Actor	Usuario.
Descripción	Se configura la regla que define el comportamiento del CA.
Tipo	Primario.
Entradas	▪ Desde mouse BTN18 Definir regla. ▪ Desde teclado IN5 Regla. ▪ Desde mouse IN6 Regla (Código).
Salidas	▪ Se actualiza IN6 Regla (Código). ▪ Se actualiza IN5 Regla.
Precondición	Ninguna.
Postcondición	▪ Regla actualizada en el sistema. ▪ Simulación en pausa.
Errores	▪ Invalido: La regla establecida está fuera del rango.

Flujo básico	
Actor	**Sistema**
1. Ingresa la regla en el campo IN5 Regla en UI3 Configuración. 2. Presiona el botón BTN18 Definir regla.	3. Actualiza la regla. 4. Actualiza IN6 Regla (Código).

Flujo alterno 1	
Actor	**Sistema**
1.1. Ingresa la regla seleccionando IN6 Regla (Código) en UI3 Configuración. 1.2. Presiona el botón BTN18 Definir regla.	1.3. Actualiza la regla. 1.4. Actualiza IN5 Regla.

Flujo alterno 2: Invalido	
Actor	**Sistema**
2.1. Ingresa la regla fuera del rango de reglas en el campo IN5 Regla en UI3 Configuración. 2.2. Presiona el botón BTN18 Definir regla mientras la simulación está activa.	2.3. La regla no es compatible, se ajusta a la configuración del CA. 2.4. Actualiza la regla.

UC17 Cambiar color de estado

Actor	Usuario.
Descripción	Se define un color asociado a cada estado.
Tipo	Secundario.
Entradas	▪ Desde mouse BTN16 Estado n-esimo..
Salidas	▪ Interfaz UI2 Simulación actualizada.
Precondición	▪ BTN16 Estado n-esimo. definido por los estados.
Postcondición	▪ Se le asigna un color al estado seleccionado. ▪ Todas las células con ese estado actualizan su color en UI2 Simulación. ▪ La configuración de color queda almacenada.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona dos veces un BTN16 Estado n-esimo. en UI3 Configuración. 2. Selecciona un color.	3. Abre un panel de colores. 4. Asigna ese color a ese estado. 5. Cambia el color de todas las células con ese estado en UI2 Simulación.

7.8.3. Interacción con el espacio celular

El usuario puede interactuar con el espacio celular para cambiar valores, colores o vecindades. Los siguientes casos de uso están diseñado para mejorar la visibilidad del espacio celular.

UC18 Cambiar estado de una célula

Actor	Usuario.
Descripción	Se cambia manualmente la configuración del espacio celular, seleccionando el estado de cada célula.
Tipo	Secundario.
Entradas	1. Desde mouse BTN16 Estado n-esimo.. 2. Desde mouse IN7 Espacio.
Salidas	1. Actualización del espacio UI2 Simulación.
Precondición	1. Haber seleccionado un BTN16 Estado n-esimo..
Postcondición	1. La célula cambia de estado.
Errores	1. Sin estado: No se ha seleccionado un estado.

Flujo básico	
Actor	**Sistema**
1. Selecciona un estado BTN16 Estado n-esimo. desde UI3 Configuración. 2. Selecciona una célula desde UI2 Simulación.	3. La célula se le asigna el estado seleccionado. 4. Se actualiza UI2 Simulación.

Flujo alterno 1: Sin estado	
Actor	**Sistema**
1.1 Selecciona una célula desde UI2 Simulación sin haber seleccionado un BTN16 Estado n-esimo..	1.2. La célula cambia al estado siguiente. 1.3. Se actualiza UI2 Simulación.

7.8.4. Entrenamiento

Los casos de uso siguientes permiten entrenar y definir la red neuronal del autómata celular neuronal. Se realiza el entrenamiento y la exportación de los pesos neuronales.

UC19 Iniciar entrenamiento

Actor	Usuario.
Descripción	El usuario carga la evolución, define la vecindad, la configuración de la red neuronal, los hiperparametros de entrenamiento y la ruta de guardado.
Tipo	Primario.
Entradas	1. Desde mouse BTN19 Cargar evolución 2. Desde mouse BTN20 Definir ruta de guardado 3. Desde mouse IN8 Vecindad 4. Desde teclado IN9 Capas ocultas 5. Desde teclado IN10 Neuronas 6. Desde teclado IN11 Activación ReLU 7. Desde teclado IN12 Activación sigmoide 8. Desde teclado IN13 Activación Tanh 9. Desde teclado IN14 Activación lineal 10. Desde teclado IN15 Capa de salida 11. Desde teclado IN16 Ultima función de activación 12. Desde teclado IN17 Epocas 13. Desde teclado IN18 Tamaño de lotes 14. Desde teclado IN19 Tasa de aprendizaje 15. Desde teclado IN20 Perdida objetivo 16. Desde teclado IN21 Imprimir cada N 17. Desde mouse BTN21 Iniciar entrenamiento
Salidas	1. OUT1 Ruta de evolución 2. OUT2 Ruta de guardado 3. OUT3 Mapa de vecindad 4. OUT4 Consola
Precondición	Ninguna.
Postcondición	Ninguna.
Errores	1. Formato invalido: El archivo de evolución no coincide. 2. Entrenamiento fallido: El entrenamiento no se completo debido a un error de ejecución.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN19 Cargar evolución desde UI4 Entrenamiento. 3. Selecciona la evolución en el archivo “npy”. 5. Presiona el botón BTN20 Definir ruta de guardado. 6. Selecciona la ruta de guardado de los pesos “npz”. 8. Selecciona la vecindad con IN8 Vecindad 9. Define la configuración de la red neuronal con las siguientes entradas: IN9 Capas ocultas, IN10 Neuronas, IN11 Activación ReLU, IN12 Activación sigmoide, IN13 Activación Tanh, IN14 Activación lineal, IN15 Capa de salida y IN16 Ultima función de activación. 10. Define los hiperparametros de entrenamiento: IN17 Epocas, IN18 Tamaño de lotes, IN19 Tasa de aprendizaje, IN20 Perdida objetivo, IN21 Imprimir cada N. 11. Presiona el botón BTN21 Iniciar entrenamiento.	2. Abre el explorador de archivos. 4. Muestra la ruta en OUT1 Ruta de evolución 5. Abre el explorador de archivos. 7. Muestra la ruta en OUT2 Ruta de guardado. 12. Borra el contenido de OUT4 Consola. 13. Llama la función de entrenamiento con todos los parámetros definidos y muestra los resultados en OUT4 Consola. 14. Guarda la evolución en la ruta definida en OUT2 Ruta de guardado.

Flujo alterno 1: Formato invalido	
Actor	**Sistema**
1.1. Presiona el botón BTN19 Cargar evolución. 1.3. Selecciona un archivo incorrecto o con otro formato. 1.5. Presiona el botón BTN20 Definir ruta de guardado. 1.6. Selecciona la ruta de guardado de los pesos “npz”.	1.2. Abre el explorador de archivos. 1.4. Muestra el mensaje MSG1 Formato de archivo incorrecto.

Flujo alterno 2: Entrenamiento fallido	
Actor	**Sistema**
2.1. Presiona el botón BTN19 Cargar evolución. 2.3. Selecciona la evolución en el archivo “npy”. 2.5. Presiona el botón BTN20 Definir ruta de guardado. 2.6. Selecciona la ruta de guardado de los pesos “npz”. 2.8. Selecciona la vecindad con IN8 Vecindad 2.9. Define la configuración de la red neuronal con las siguientes entradas: IN9 Capas ocultas, IN10 Neuronas, IN11 Activación ReLU, IN12 Activación sigmoide, IN13 Activación Tanh, IN14 Activación lineal, IN15 Capa de salida y IN16 Ultima función de activación. 2.10. Define los hiperparametros de entrenamiento: IN17 Epocas, IN18 Tamaño de lotes, IN19 Tasa de aprendizaje, IN20 Perdida objetivo, IN21 Imprimir cada N. 2.11. Presiona el botón BTN21 Iniciar entrenamiento.	2.2. Abre el explorador de archivos. 2.4. Muestra la ruta en OUT1 Ruta de evolución 2.5. Abre el explorador de archivos. 2.7. Muestra la ruta en OUT2 Ruta de guardado. 2.12. Borra el contenido de OUT4 Consola. 2.13. Llama la función de entrenamiento con todos los parámetros definidos y muestra los resultados en OUT4 Consola. 2.14. Algún elemento de la configuración, la cantidad de datos usados o limitaciones de hardware impidieron que se completara el entrenamiento. Muestra el mensaje MSG4 Entrenamiento incompleto en OUT4 Consola y no guarda nada.

UC20 Detener entrenamiento

Actor	Usuario.
Descripción	El usuario detiene el entrenamiento de la red neuronal.
Tipo	Secundario.
Entradas	1. Desde mouse BTN22 Detener entrenamiento.
Salidas	1. OUT4 Consola.
Precondición	1. UC18 Iniciar entrenamiento.
Postcondición	Ninguna.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN22 Detener entrenamiento desde UI4 Entrenamiento.	2. Interrumpe el entrenamiento en curso. 3. Muestra el mensaje MSG**??** en OUT4 Consola. 4. Habilita nuevamente los controles de UI4 Entrenamiento.

UC21 Borrar registro

Actor	Usuario.
Descripción	El usuario elimina la salida de la consola.
Tipo	Secundario.
Entradas	1. Desde mouse BTN22 .
Salidas	Ninguna.
Precondición	Ninguna.
Postcondición	Ninguna.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN22 desde UI4 Entrenamiento.	2. Elimina los mensajes de OUT4 Consola.

7.9. Diagramas de secuencia

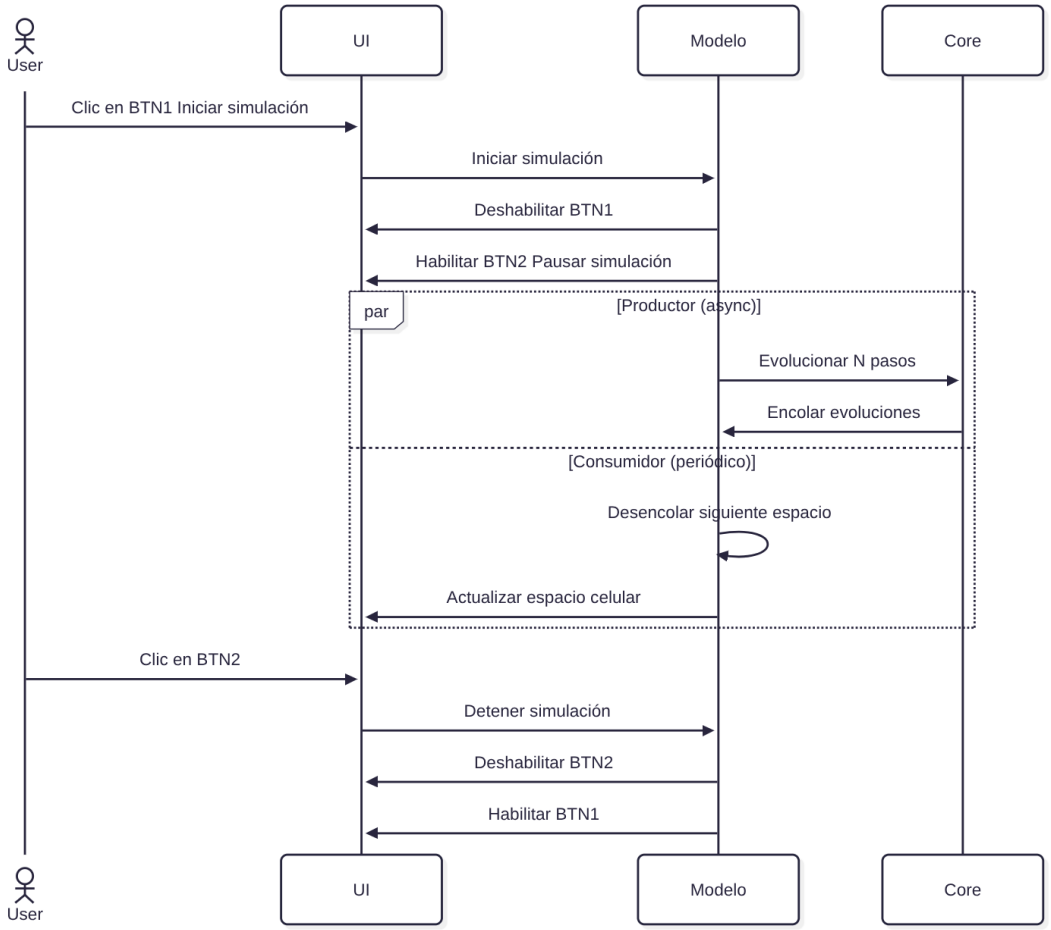


Figura 7.3: Diagrama de secuencia: UC01 Iniciar simulación.

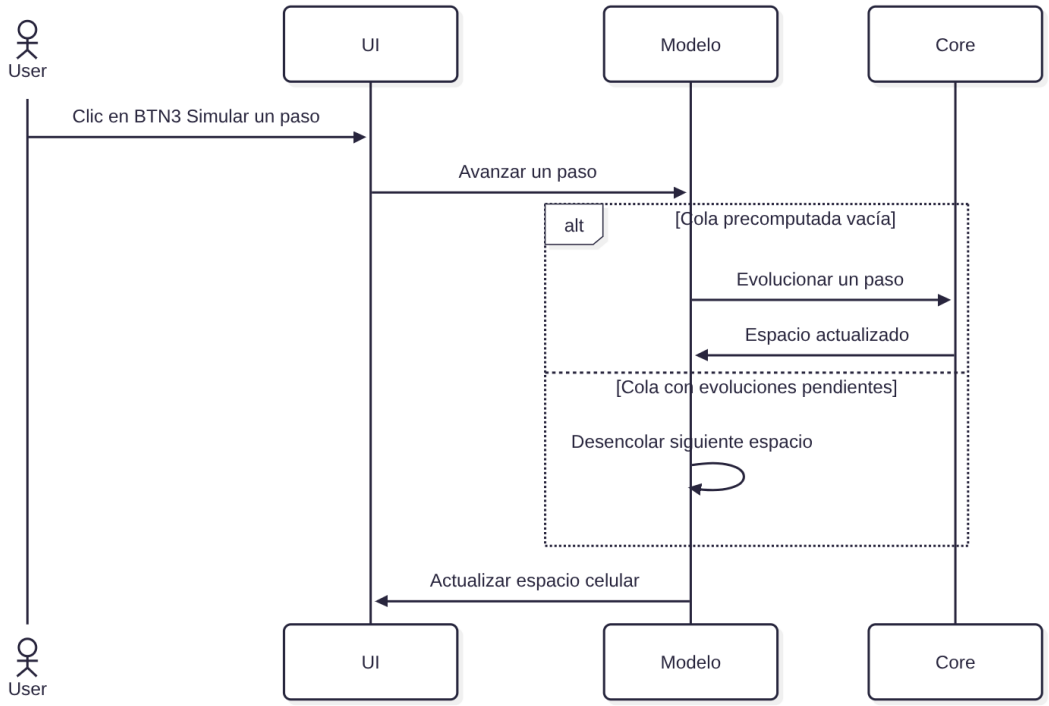


Figura 7.4: Diagrama de secuencia: UC04 Simular un paso.

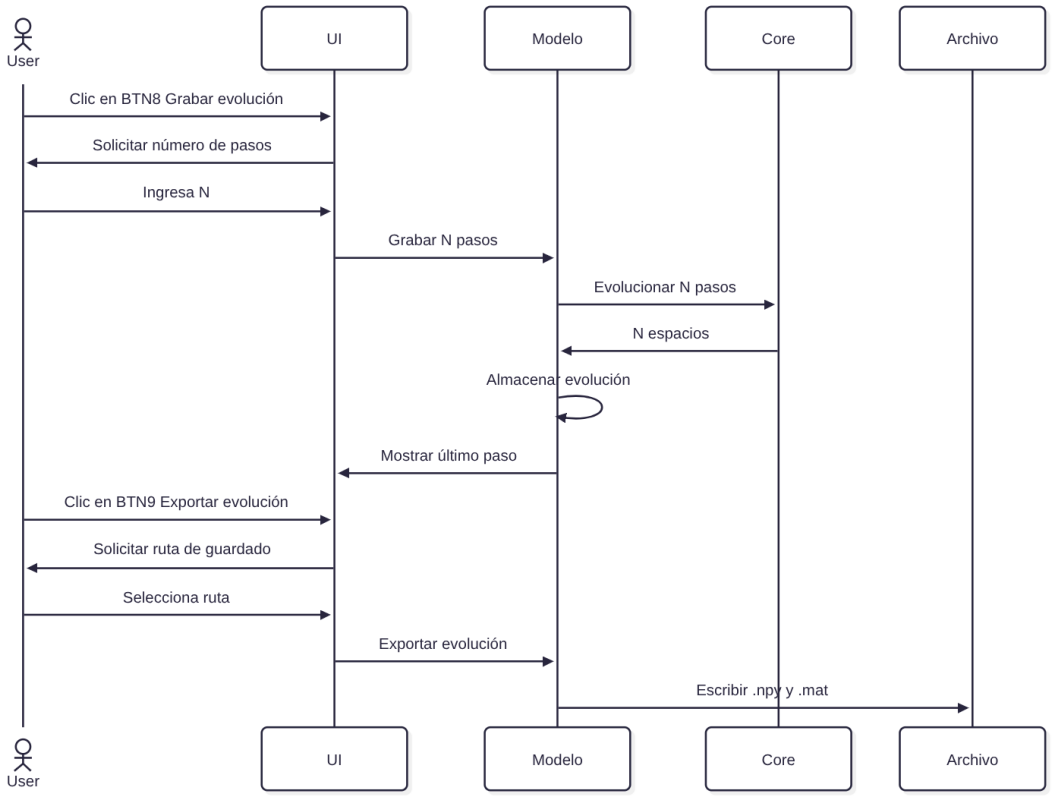


Figura 7.5: Diagrama de secuencia: UC09 Grabar evolución y UC10 Exportar evolución.

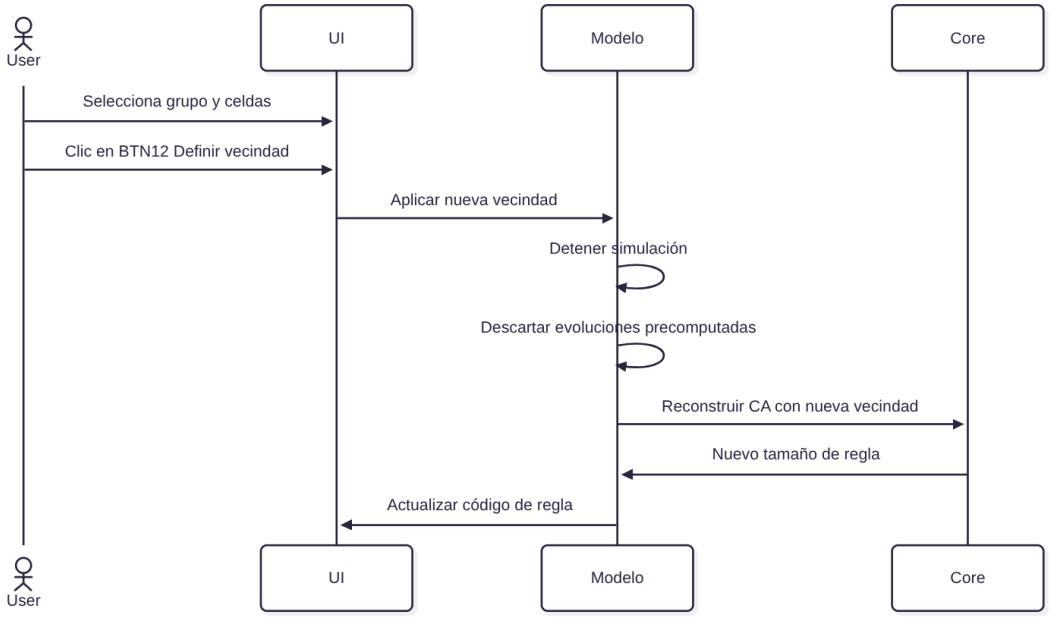


Figura 7.6: Diagrama de secuencia: UC13 Definir vecindad.

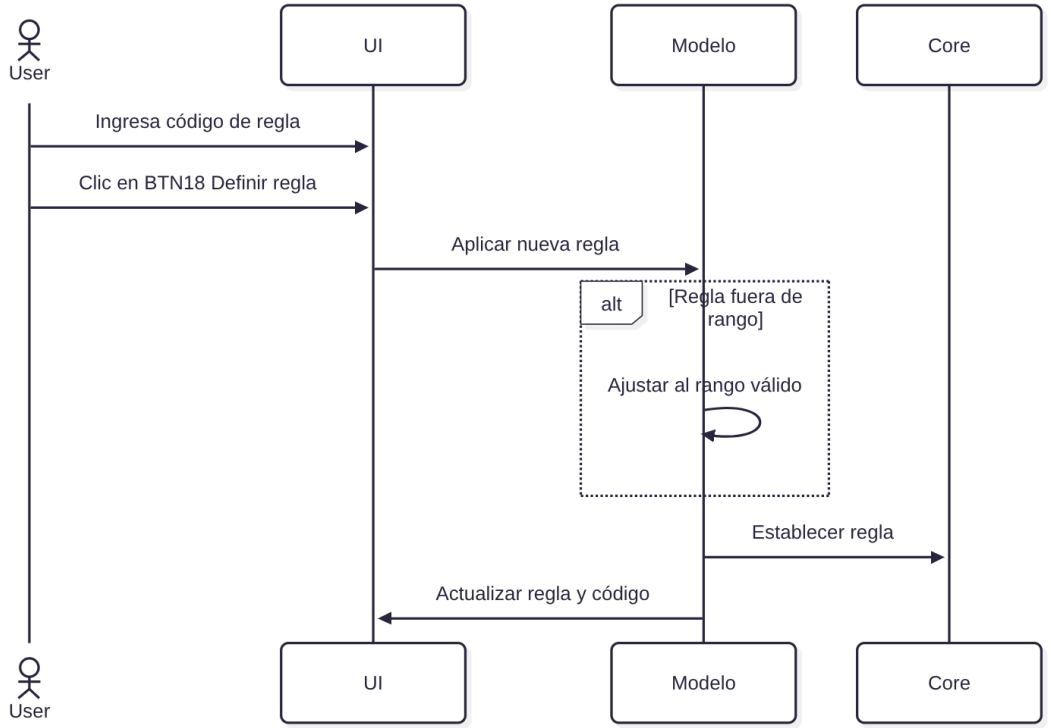


Figura 7.7: Diagrama de secuencia: UC16 Definir regla de transición.

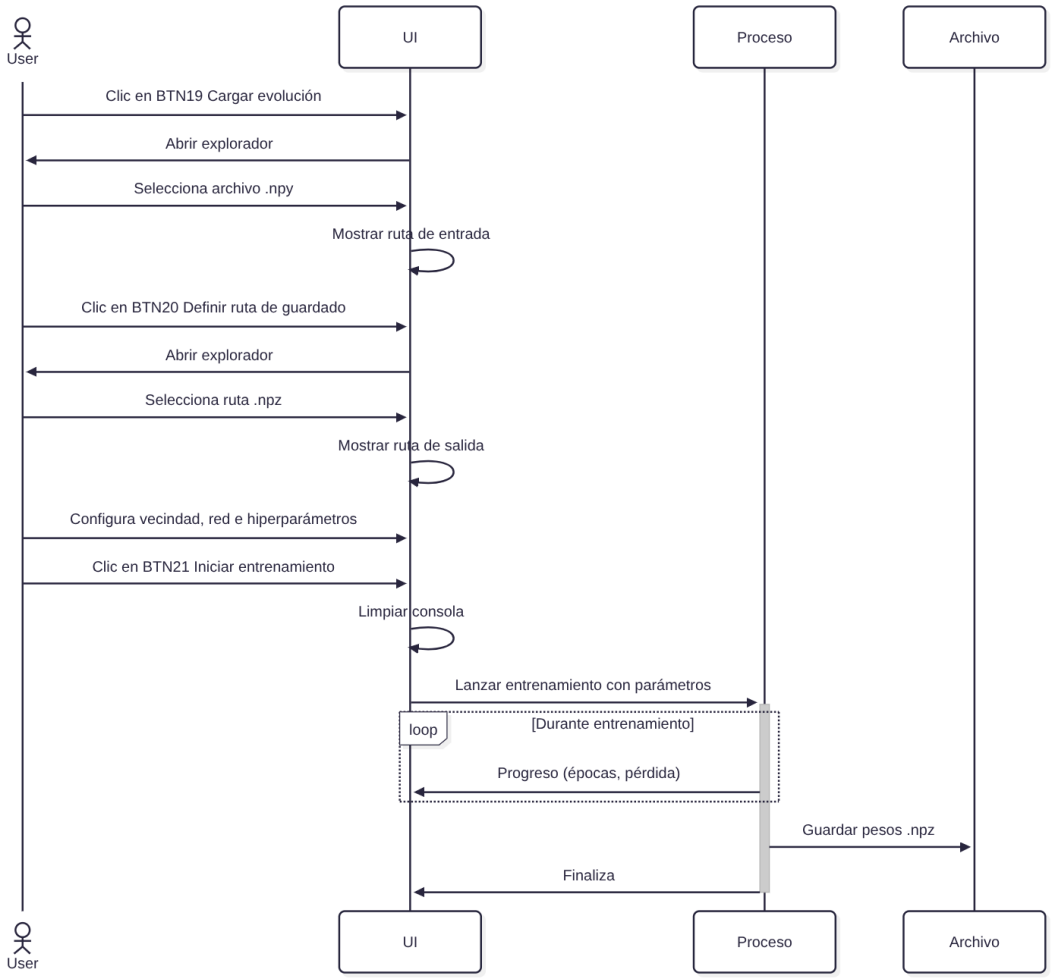


Figura 7.8: Diagrama de secuencia: UC19 Iniciar entrenamiento.

Capítulo 8

Diseño de software

8.1. Arquitectura

La arquitectura del sistema se basa en el patrón de diseño modelo vista controlador, el cual resulta adecuado para aplicaciones que combinan lógica de negocio, manipulación de datos y una interfaz gráfica interactiva [147]. Este patrón favorece la separación de responsabilidades, la mantenibilidad y la extensibilidad del sistema.

- **Modelo:** Gestiona los datos y la lógica principal del autómata celular, incluyendo la definición de estados, reglas, vecindad y condiciones de frontera.
- **Vista:** Se encarga de la presentación visual de la información y la interacción con el usuario mediante la interfaz gráfica.
- **Controlador:** Actúa como intermediario entre el modelo y la vista, gestionando los eventos de usuario y actualizando el modelo o la interfaz según corresponda.

8.1.1. Vista

La vista está implementada en *Python* utilizando el framework **PyQt5**. Esta capa proporciona la interfaz gráfica principal del simulador, permitiendo visualizar y controlar el espacio celular y entrenar el autómata celular neuronal.

8.1.2. Modelo

El modelo encapsula la lógica del autómata celular. Esta capa está implementada en *C++* y CUDA con el fin de optimizar el rendimiento en las operaciones de actualización de celdas y el procesamiento paralelo. El modelo gestiona la estructura del espacio celular, los estados, la topología de la vecindad, las condiciones de frontera y la regla de transición.

8.1.3. Controlador

El **controlador** coordina la comunicación entre la vista y el modelo. Implementado en Python, este componente recibe los eventos generados por la interfaz, actualiza el modelo y refleja los resultados en la interfaz gráfica. Además, sincroniza los procesos de simulación y representación visual.

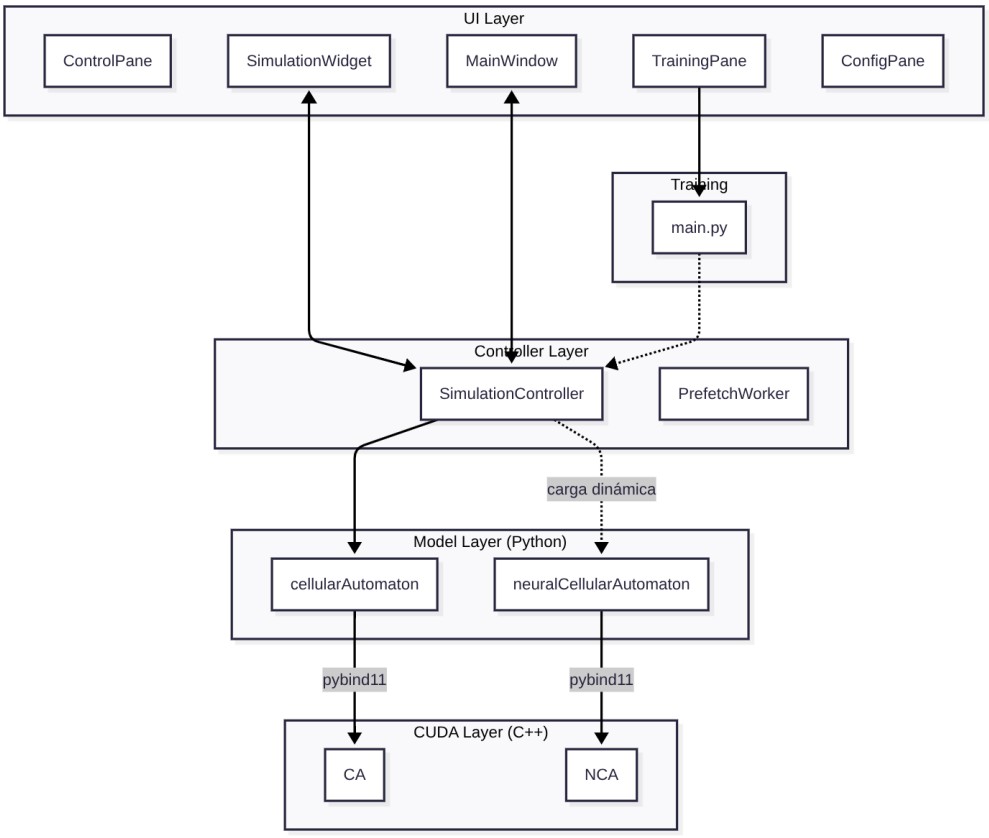


Figura 8.1: Diagrama de arquitectura del sistema.

8.2. Diagrama de clases

El diagrama de clases UML sirve para mostrar una vista estática de las clases de un sistema, sus atributos, métodos y relaciones entre objetos. Sirve para proporcionar una información general sobre la aplicación [148].

El diagrama de clases se compone de tres partes:

- **CUDA**: La figura 8.2 muestra la definición de las clases del autómata celular y el autómata celular neuronal.
- **Modelos** Son los modelos construidos con Pybind11 los cuales son manipulados por "SimulationController", como se muestra en la figura 8.3.
- **UI** Elementos y controladores de la interfaz grafica en PyQt5. Ver figura 8.4.

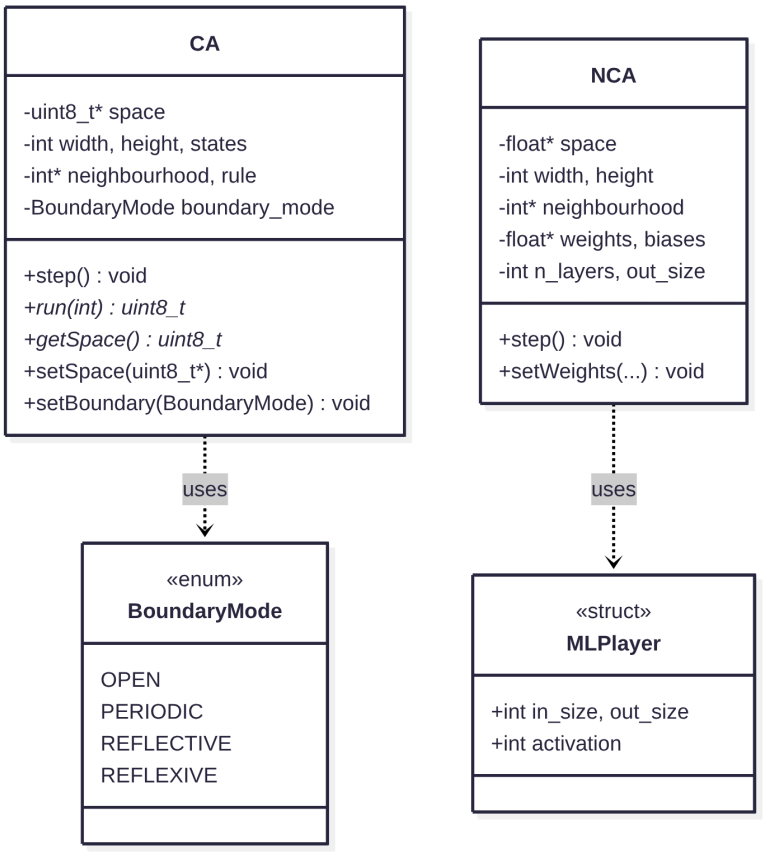


Figura 8.2: Diagrama de clases: Capa CUDA

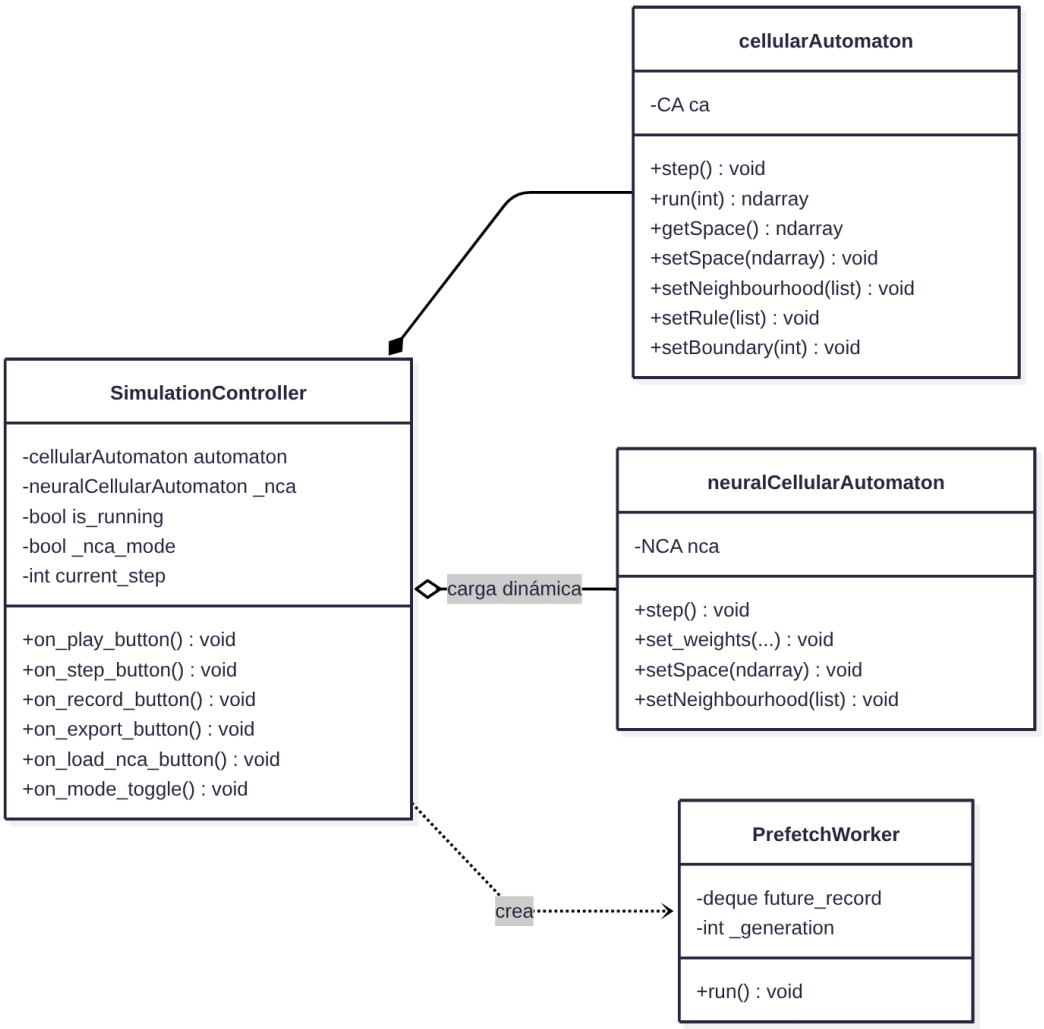


Figura 8.3: Diagrama de clases: Capa de modelos

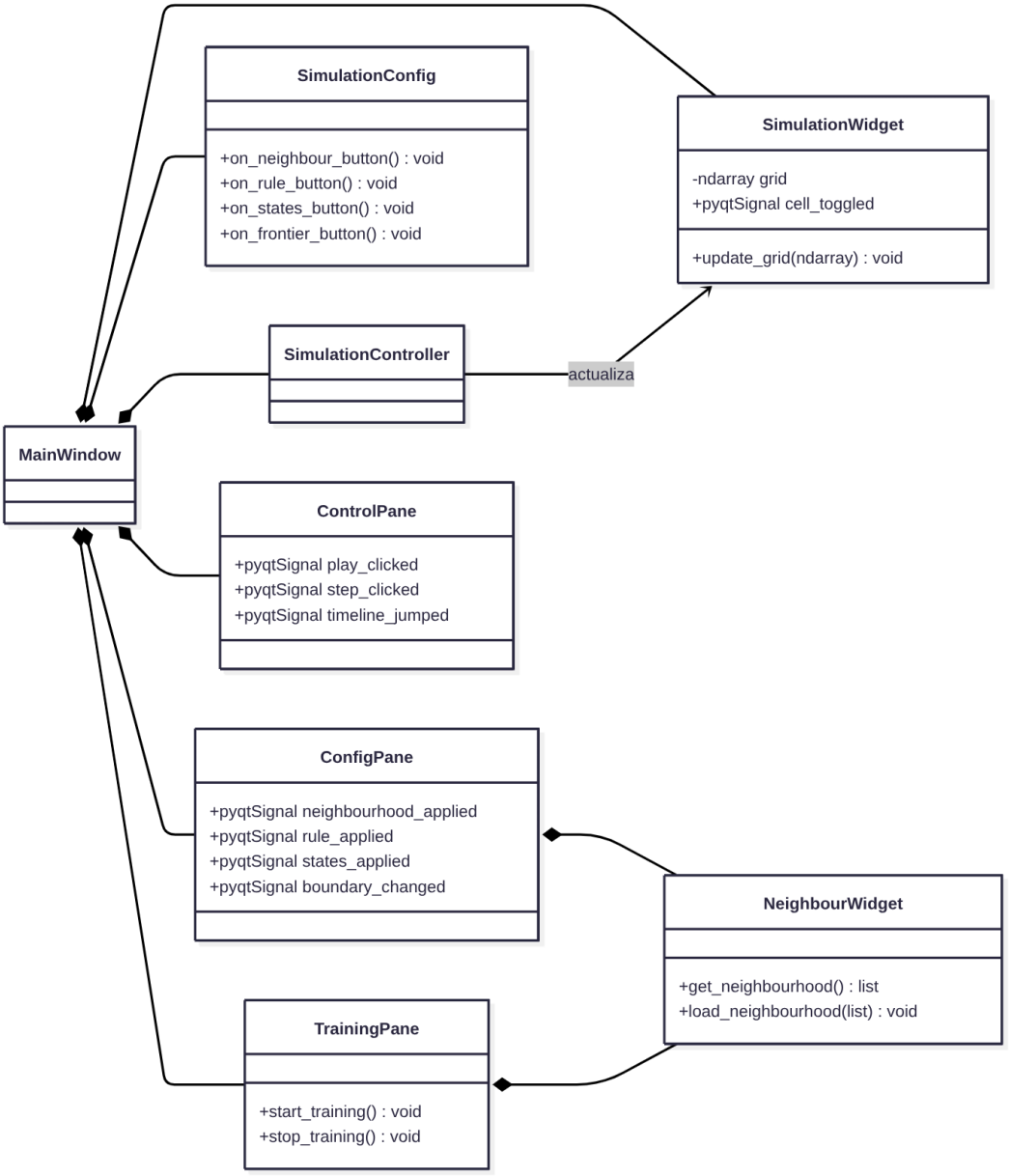


Figura 8.4: Diagrama de clases: Capa UI

8.3. Diseño de Datos

El simulador maneja seis estructuras de datos principales: el espacio celular, la vecindad, la regla de transición, la configuración del autómata celular, la configura-

ción del autómata celular neuronal y el historial de las evoluciones. A continuación se definen estas variables y cómo evolucionan en las diferentes capas del sistema.

La información dentro de CUDA, también llamado *device*, no puede almacenar arreglos multidimensionales. Por esta razón, almacena matrices y tensores como un único arreglo unidimensional continuo y se accede a los indices de forma manual. Resulta necesario hacer un mapeo entre Python, también *host*, con el *device*.

8.3.1. Espacio Celular

Se define como una matriz bidimensional de dimensiones $H \times W$ con valores enteros de 8 bits, permitiendo representar hasta 256 estados distintos por celda.

- **Host**: Implementado como un `numpy.ndarray` de *shape* (H, W) y `dtype=uint8`. Para maximizar el rendimiento, este arreglo se comparte mediante punteros, sin realizar copias, entre el *widget* de renderizado, el controlador de simulación y el *wrapper* de CUDA.
- **Device**: Utiliza dos búferes (`A_d` y `B_d`) de tipo `uint8_t*` en la memoria global. Tienen un esquema de *ping-pong buffering*, donde tras cada iteración del kernel, se intercambian los punteros de lectura y escritura, eliminando la necesidad de copiar datos entre pasos de evolución.
- **Sincronización perezosa**: El *host* no se sincroniza tras cada paso del kernel. Una bandera `host_dirty` indica si el búfer del *host* está desactualizado respecto al *device*; la copia *device-to-host* se ejecuta únicamente cuando se invoca una operación de lectura (`getSpace`, `getCell`) o al exportar.

8.3.2. Vecindad

La vecindad define el conjunto de celdas que influyen en la transición de una celda objetivo. Se modela bajo un esquema semitotalístico organizado en grupos de desplazamientos relativos.

- **Host**: Una estructura `list[list[list[int]]]` donde el nivel externo enumera los grupos, el intermedio las posiciones relativas, y el interno los componentes $[dx, dy]$ del desplazamiento.

- **Device**: La vecindad se define mediante dos arreglos: (1) `neighbourhood[]` que contiene los vectores de desplazamiento concatenados y (2) `group_sizes[]` que almacena los tamaños de los grupos.
- **Cotas técnicas**: Se establece un límite de $n_{\text{size}} \leq 64$ desplazamientos totales y $n_{\text{groups}} \leq 64$ grupos para cumplir con las restricciones de la memoria constante.

8.3.3. Regla de Transición (CA Clásico)

La regla es de naturaleza semitotalística: el nuevo estado depende del conteo de estados dentro de cada grupo de la vecindad. El tamaño total r_{size} se determina por el producto de las combinaciones posibles de cada grupo:

$$r_{\text{size}} = \prod_{g \in \text{grupos}} \big((\text{states} - 1) \cdot |g| + 1\big)$$

donde $|g|$ es la cardinalidad del grupo y `states` el número de estados permitidos.

El acceso a la regla se realiza mediante una suma posicional de las contribuciones de cada grupo de vecindad, calculando un índice lineal único para cada configuración de conteos.

- **Host**: `numpy.ndarray` de `uint8`, tamaño r_{size}.
- **Device**: Debido a que r_{size} puede alcanzar hasta $16,777,216$ entradas, la tabla reside en la memoria global del dispositivo (`rule_d`).

8.3.4. Configuración del Autómata Celular (CA)

Esta estructura encapsula los parámetros estáticos que definen el comportamiento de un autómata clásico:

Campo	Tipo	Descripción
`dimensions`	`(int, int)`	Dimensiones $H \times W$ del espacio.
`states`	`uint8`	Estados permitidos (1–256).
`neighbourhood`	`list[list[list[int]]]`	Definición de grupos de vecindad.
`rule`	`uint8[]`	Tabla indexada de la sección 3.3.
`boundary_mode`	`enum`	Modos: `OPEN`, `PERIODIC`, `REFLECTIVE`, `REFLEXIVE`.
`colors`	`list[QColor]`	Mapeo de estados a colores para el renderizado.

8.3.5. Configuración del Autómata Celular Neuronal (NCA)

El modelo neuronal sustituye la tabla de reglas por un Perceptrón Multicapa (MLP) que procesa la información de la vecindad.

- **Topología del MLP**: Se define mediante `layer_in` (entradas según vecindad), `layer_out` (neuronas de salida), `hidden_layers[]` (arquitectura oculta) y `activations[]` (códigos: `0=ReLU`, `1=Sigmoid`, `2=Tanh`, `3=Linear`).
- **Pesos y Sesgos**: Se organizan en un arreglo plano de tipo `float` con un vector de desplazamientos (*offsets*). Esta estructura lineal favorece el acceso coalescente en la GPU y minimiza la fragmentación de memoria.
- **Formato de Intercambio**: La persistencia se realiza en archivos `.npz` (NumPy) que contienen los tensores de pesos, sesgos y los metadatos de la topología, permitiendo una carga directa en la interfaz gráfica.

8.3.6. Historial de Evoluciones

El historial gestiona el estado temporal de la simulación mediante búferes en memoria y persistencia en disco.

- **Gestión de Memoria**:
  - `past_record`: Lista de instantáneas `ndarray` para permitir el retroceso cronológico.

  - `future_record`: Un `collections.deque` que almacena estados precomputados asíncronamente por un `PrefetchWorker`.

- **Grabación de Alto Rendimiento**: Se utiliza memoria *pinned* (fija) reservada con `cudaHostAlloc`. Esto permite que el flujo de datos dispositivo–anfitrión sea asíncrono y no se vea penalizado por la paginación del sistema operativo.
- **Formatos de Salida**: Los datos se exportan simultáneamente en `.npy` (para re-entrenamiento en Python) y `.mat` (para análisis y validación en Wolfram Mathematica).

8.4. Diseño de Algoritmos

Dentro del simulador existen seis algoritmos no triviales que determinan el correcto funcionamiento de todos los modulos. estos algoritmos son: indexación de la regla semitotalística, el paso de evolución del autómata celular clásico, el manejo de las condiciones de frontera, el *forward pass* del autómata celular neuronal, el entrenamiento del modelo neuronal y el prefetch asíncrono. En esta sección se presentan los pseudocódigos de dichos algoritmos junto con sus decisiones de diseño.

Para homogeneizar los algoritmos, todos se rigen bajo las siguientes convenciones:

- Los arreglos se indexan desde cero.
- `H` y `W` son alto y ancho del espacio celular.
- Para acceder a la posición de espacio celular en su forma unidimensional se realiza mediante el siguiente calculo: `A[y · W + x]`.
- `states` es el número de estados permitidos por celda.
- `n_groups` y `group_sizes[]` describen la partición en grupos de la vecindad.

8.4.1. Indexación de la regla semitotalística

La tabla de regla `rule[]` contiene una entrada por cada combinación posible de conteos por grupo. Para acceder a la entrada correcta durante la evaluación de

una celda, se requiere convertir el vector de conteos en un índice lineal único. Este algoritmo se invoca $H \times W$ veces por cada paso evolutivo.

Cada grupo g aporta una contribución igual a la suma de los estados de las celdas que lo integran. Esa suma puede tomar valores en $\{0, 1, \ldots, (\text{states}-1)\cdot|g|\}$, es decir, $((\text{states} - 1) \cdot |g| + 1)$ valores distintos. El índice global se construye en base mixta posicional, multiplicando cada contribución por el producto acumulado de los tamaños de los grupos previos.

Algorithm 1 IndiceRegla: vector de conteos → índice lineal

Require: `sums[n_groups]`, `group_sizes[n_groups]`, `states`
Ensure: `idx` $\in [0, r_{\text{size}})$
1: `idx` $\leftarrow 0$
2: `mult` $\leftarrow 1$
3: **for** $g \leftarrow 0$ **to** `n_groups` $- 1$ **do**
4: `idx` $\leftarrow$ `idx` $+$ `sums`$[g] \cdot$ `mult`
5: `mult` $\leftarrow$ `mult` $\cdot ((\texttt{states} - 1) \cdot \texttt{group_sizes}[g] + 1)$
6: **end for**
7: **return** `idx`

Complejidad: $\mathcal{O}$(`n_groups`) por celda.

En el juego de la vida existen dos grupos: los 8 vecinos de Moore y la célula central, y dos estados (vivo/muerto). El tamaño de tabla resulta $r_{\text{size}} = \big((2-1)\cdot 8 + 1\big) \cdot \big((2-1)\cdot 1 + 1\big) = 9 \cdot 2 = 18$ entradas, indexadas por el conteo de vecinos vivos, lo que corresponde a la formulación clásica B3/S23.

8.4.2. Paso de evolución del autómata celular clásico

En cada paso evolutivo el kernel CUDA gestiona los hilos responsables de calcular el nuevo estado de exactamente todas las celdas. La grilla se mapea a una topología bidimensional de bloques: cada bloque cubre 16×16 celdas, y los bloques se organizan en un grid de tamaño $\lceil W/16 \rceil \times \lceil H/16 \rceil$.

Para cada celda, el hilo recorre la vecindad agrupada, acumula la suma de estados por grupo, calcula el índice de la regla y escribe el resultado en el búfer destino. Tras el kernel, los punteros `A_d` y `B_d` se intercambian (*ping-pong*), evitando copias entre pasos.

Algorithm 2 stepKernel⟨`BoundaryMode`⟩: un paso de evolución del autómata celular

Require: búfer origen `A[H·W]`, búfer destino `B[H·W]`, `neighbourhood[n_size]`, `group_sizes[n_groups]`, `rule[r_size]`

1: $(x, y) \leftarrow$ coordenadas del hilo en el grid
2: **if** $x \geq W$ **or** $y \geq H$ **then**
3: **return**
4: **end if**
5: `offset` $\leftarrow 0$; `idx` $\leftarrow 0$; `mult` $\leftarrow 1$
6: **for** $g \leftarrow 0$ **to** `n_groups` $- 1$ **do**
7: `sum` $\leftarrow 0$
8: **for** $i \leftarrow 0$ **to** `group_sizes`$[g] - 1$ **do**
9: $(dx, dy) \leftarrow$ `neighbourhood`[`offset` $+ i$]
10: $(x', y',$ `valid`$) \leftarrow$ ApplyBoundary⟨`BoundaryMode`⟩$(x + dx, y + dy, W, H)$
11: **if** `valid` **then**
12: `sum` $\leftarrow$ `sum` $+$ `A`$[y' \cdot W + x']$
13: **end if**
14: **end for**
15: `idx` $\leftarrow$ `idx` $+$ `sum` $\cdot$ `mult`
16: `mult` $\leftarrow$ `mult` $\cdot ((\text{states} - 1) \cdot$ `group_sizes`$[g] + 1)$
17: `offset` $\leftarrow$ `offset` $+$ `group_sizes`$[g]$
18: **end for**
19: `B`$[y \cdot W + x] \leftarrow$ `rule`[`idx`]

El parámetro `BoundaryMode` es un argumento de plantilla, no un parámetro en tiempo de ejecución. El compilador genera cuatro versiones del kernel (una por modo de frontera) y el `if constexpr` dentro de ApplyBoundary elimina toda ramificación condicional en el GPU lo que evita la divergencia de *warps* en celdas cercanas al borde.

Complejidad: $\mathcal{O}$(`n_size`) por celda, $\mathcal{O}(H \cdot W \cdot$ `n_size`$)$ por paso.

8.4.3. Manejo de condiciones de frontera

Las cuatro condiciones de frontera modifican la coordenada de un vecino que cae fuera del dominio antes de leerlo. Cada modo se implementa como una función separada pero con la misma firma.

Algorithm 3 ApplyBoundary: cuatro variantes especializadas en compilación

function ApplyBoundary(OPEN)x, y, W, H
  if $x < 0$ **or** $x \geq W$ **or** $y < 0$ **or** $y \geq H$ **then**
    return $(0, 0, \texttt{false})$
  end if
  return $(x, y, \texttt{true})$
end function

function ApplyBoundary(PERIODIC)x, y, W, H
  return $((x \bmod W + W) \bmod W,\ (y \bmod H + H) \bmod H,\ \texttt{true})$
end function

function ApplyBoundary(REFLECTIVE)x, y, W, H ▷ La frontera actúa como un espejo
  if $x < 0$ **then**
    $x \leftarrow -x - 1$
  end if
  if $x \geq W$ **then**
    $x \leftarrow 2W - x - 1$
  end if
  if $y < 0$ **then**
    $y \leftarrow -y - 1$
  end if
  if $y \geq H$ **then**
    $y \leftarrow 2H - y - 1$
  end if
  return $(x, y, \texttt{true})$
end function

function ApplyBoundary(REFLEXIVE)x, y, W, H ▷ El borde se trata como una celda real
  if $x < 0$ **then**
    $x \leftarrow -x$
  end if
  if $x \geq W$ **then**
    $x \leftarrow 2W - x - 2$
  end if
  if $y < 0$ **then**
    $y \leftarrow -y$
  end if
  if $y \geq H$ **then**
    $y \leftarrow 2H - y - 2$
  end if
  return $(x, y, \texttt{true})$
end function

Complejidad: $\mathcal{O}(1)$.

8.4.4. Forward pass del autómata celular neuronal

El autómata celular neuronal, a diferencia del clasico utiliza un perceptrón multicapa que se evalúa de forma independiente sobre cada celda. El kernel CUDA mantiene la misma estructura que el autómata celular ejecutando un hilo por cada celda. Sin embargo, en lugar de un acceso a la tabla, se ejecuta una secuencia de productos matriz-vector con activaciones intercaladas.

Los pesos y sesgos de las capas residen en un dos arreglos planos `weights[]` y `biases[]`, cada uno con su vector `w_offsets[]` que apunta al inicio de cada matriz $W^{(l)}$.

Algorithm 4 ncaKernel: *forward pass* del MLP por celda

Require: `A`, `B`, `neighbourhood[n_size]`, `weights[]`, `biases[]`, `w_offsets[]`, `b_offsets[]`, `layer_sizes[n_layers+1]`, `activations[n_layers]`

1: $(x, y) \leftarrow$ coordenadas del hilo
2: **if** $x \geq W$ **or** $y \geq H$ **then**
3: **return**
4: **end if**
5: `input`$[$`n_size`$] \leftarrow$ vector de tamaño `n_size`
6: **for** $i \leftarrow 0$ **to** `n_size` $- 1$ **do**
7: $(dx, dy) \leftarrow$ `neighbourhood`$[i]$
8: $(x', y', _) \leftarrow$ ApplyBoundary$(x + dx, y + dy, W, H)$
9: `input`$[i] \leftarrow$ `A`$[y' \cdot W + x']$
10: **end for**
11: `current` $\leftarrow$ `input`
12: **for** $\ell \leftarrow 0$ **to** `n_layers` $- 1$ **do**
13: $W_\ell \leftarrow$ `weights` $+$ `w_offsets`$[\ell]$
14: $b_\ell \leftarrow$ `biases` $+$ `b_offsets`$[\ell]$
15: $n_{\text{in}} \leftarrow$ `layer_sizes`$[\ell]$; $\quad n_{\text{out}} \leftarrow$ `layer_sizes`$[\ell + 1]$
16: **for** $j \leftarrow 0$ **to** $n_{\text{out}} - 1$ **do**
17: $s \leftarrow b_\ell[j]$
18: **for** $k \leftarrow 0$ **to** $n_{\text{in}} - 1$ **do**
19: $s \leftarrow s + W_\ell[j \cdot n_{\text{in}} + k] \cdot$ `current`$[k]$
20: **end for**
21: `next`$[j] \leftarrow$ Activar$(s,$ `activations`$[\ell])$
22: **end for**
23: `current` $\leftarrow$ `next`
24: **end for**
25: `B`$[y \cdot W + x] \leftarrow$ `current`$[0]$

Se evita el uso de `std::function` u otros punteros a función, ya que no son compatibles con kernels CUDA.

Complejidad: $\mathcal{O}\left(N_{size} + \sum_{l=0}^{L-1}(n_l \cdot n_{l+1})\right)$ por celda.

8.4.5. Entrenamiento del autómata celular neuronal

El entrenamiento parte de una evolución registrada en formato (T, H, W) y produce los pesos del perceptrón multicapa que mejor reproducen la transición observada. El procedimiento se ejecuta en PyTorch fuera del módulo CUDA, en CPU o GPU según disponibilidad.

A diferencia del aprendizaje supervisado tradicional sobre imágenes, aquí cada par de entrenamiento corresponde a una celda individual: la entrada es el vector de estados en su vecindad en el instante t, y el objetivo es el estado de la misma celda en el instante $t + 1$.

Algorithm 5 EntrenarNCA: ajuste del MLP a partir de una evolución registrada

Require: `evolucion`$\in \mathbb{R}^{T\times H\times W}$, `vecindad`, `arq[]` (tamaños de capa), `act[]` (activaciones), `lr`, `epochs_max`, `batch`, `loss_target`
Ensure: pesos y sesgos del MLP

$X \leftarrow [\,]$; $Y \leftarrow [\,]$ ▷ Extracción de pares (vecindad, objetivo)
for $t \leftarrow 0$ **to** $T-2$ **do**
  for $y \leftarrow 0$ **to** $H-1$ **do**
    for $x \leftarrow 0$ **to** $W-1$ **do**
      $v \leftarrow [\,]$
      for all $(dx, dy) \in$ `vecindad` **do**
        v.APPEND(`evolucion`$[t,\ (y+dy) \bmod H,\ (x+dx) \bmod W]$)
      end for
      X.APPEND(v)
      Y.APPEND(`evolucion`$[t+1, y, x]$)
    end for
  end for
end for

`modelo` $\leftarrow$ CONSTRUIRMLP(`arq`, `act`)
`optim` $\leftarrow$ ADAM(`modelo.parametros`, `lr`)
`criterio` $\leftarrow$ MSE

for $e \leftarrow 0$ **to** `epochs_max` -1 **do**
  $L_{\text{acum}} \leftarrow 0$
  for all `(X_b, Y_b)` en lotes de tamaño `batch` **do**
    $\hat{Y} \leftarrow$ `modelo(X_b)`
    $L \leftarrow$ `criterio`$(\hat{Y}$, `Y_b`$)$
    `optim.zero_grad()`
    L`.backward()`
    `optim.step()`
    $L_{\text{acum}} \leftarrow L_{\text{acum}} + L$
  end for
  if $L_{\text{acum}} <$ `loss_target` **then**
    break
  end if
end for
return pesos y sesgos del `modelo`

Además del límite `epochs_max`, se detiene el entrenamiento si la pérdida acumulada cae por debajo de `loss_target`.

Complejidad: La primera fase $\mathcal{O}(T \cdot H \cdot W \cdot N_{vec})$. La segunda fase $\mathcal{O}(E \cdot T \cdot H \cdot W \cdot P)$, donde E es el número de épocas y P representa la cantidad total de parámetros del percentrón muticapa ($\sum n_l \cdot n_{l+1}$).

8.4.6. Prefetch asíncrono con invalidación por generación

La GUI debe sostener una reproducción visualmente fluida sin bloquear el hilo principal de Qt, incluso cuando el cómputo de un paso es computacionalmente complejo (grillas grandes o autómatas celulares neuronales profundos). Para ello, un *worker* en un hilo del `QThreadPool` precomputa hasta `PREFETCH_SIZE` frames y los entrega al `future_record` del controlador.

Si el usuario cambia la regla, la vecindad o el estado mientras el *worker* está corriendo, los frames precomputados ya no corresponden a la configuración activa y deben descartarse. La solución es un contador entero `_generation` que el controlador incrementa ante cualquier cambio; el *worker* captura su valor al lanzarse y verifica coincidencia antes de publicar resultados.

Algorithm 6 PrefetchWorker.run: producción asíncrona de frames

Require: `automata`, `gen` (generación capturada al instanciar), `controller`, $N =$ `PREFETCH_SIZE`

`frames` $\leftarrow [\,]$
for $i \leftarrow 0$ **to** $N - 1$ **do**
    if `controller.generation` $\neq$ `gen` **then**
        return ▷ Configuración cambió; descartar trabajo
    end if
    `automata.step()`
    `frames.append(automata.getSpace().copy())`
end for

// Postear de regreso al hilo de Qt vía señal
if `controller.generation` $=$ `gen` **then**
    `controller.future_record.extend(frames)`
else
    descartar `frames`
end if

La comprobación dentro del bucle permite abortar tempranamente y liberar el hilo del pool; la comprobación final antes de publicar evita que la generación cambie justo después del último `step()` pero antes de la respuesta.

Cuando el usuario retrocede, los frames se consumen desde `past_record`; cuando avanza más allá de lo registrado, se consumen desde `future_record`. Si este último se vacía, se dispara un nuevo `PrefetchWorker` con la generación actual.

Complejidad: $\mathcal{O}(N \cdot H \cdot W \cdot c)$, donde $N =$ `PREFETCH_SIZE` y c es el costo por celda del paso evolutivo subyacente (`n_size` para CA, $N_{size} + \sum_{\ell} n_{\ell} n_{\ell+1}$ para NCA).

8.5. Interfaces de usuarios

A continuación se muestran las interfaces desarrolladas en PyQt5. Se busca mantener una estructura intuitiva y modular que facilite la configuración, ejecución y visualización de los autómatas celulares.

- **Pestaña de simulación**

  La pestaña de simulación permite configurar y controlar la evolución de los autómatas celulares como se observa en la figura 8.5. Se divide en tres secciones:

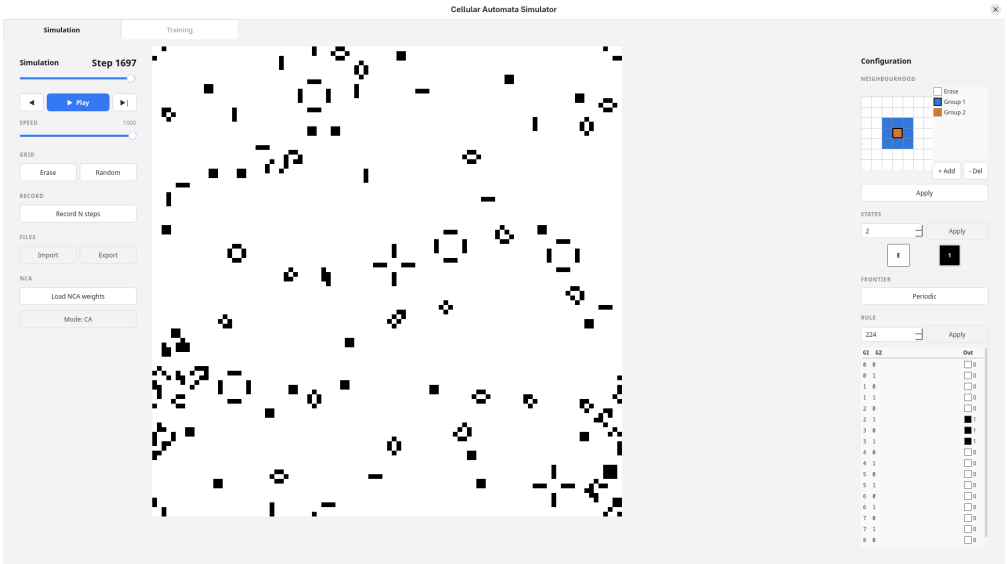
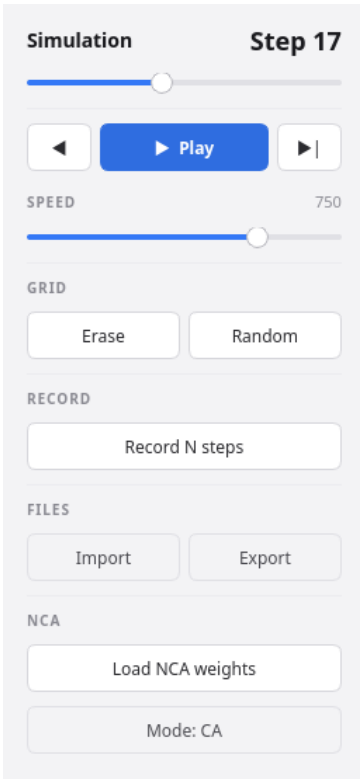
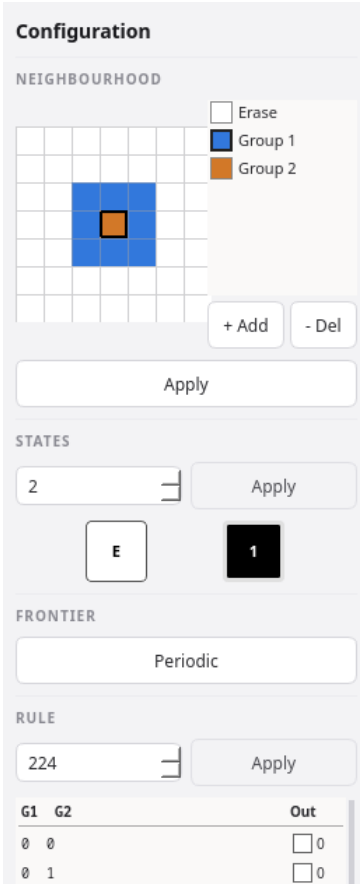


Figura 8.5: Interfaz de usuario. Pestaña de simulación.

1. Los controles de evolución, situados del lado izquierdo (ver figura 8.6a), manipula la evolución del autómata celular (iniciar, pausar, reiniciar, avanzar, retroceder o cambiar velocidad), ademas, permite importar y exportar evoluciones y cargar modelos del autómata celular neuronal.
2. El área central es el espacio de evolución definido por una red regular cuadrada.
3. La configuración del autómata celular, localizado del lado derecho (ver figura 8.6b), define la vecindad, los estados, la condición de frontera y la regla de transición.



(a) Controles de evolución.



(b) Controles de configuración.

- **Pestaña de entrenamiento**

  Esta pestaña permite al usuario entrenar al autómata celular neuronal dado un conjunto de datos, tal como se muestra en la figura 8.7.

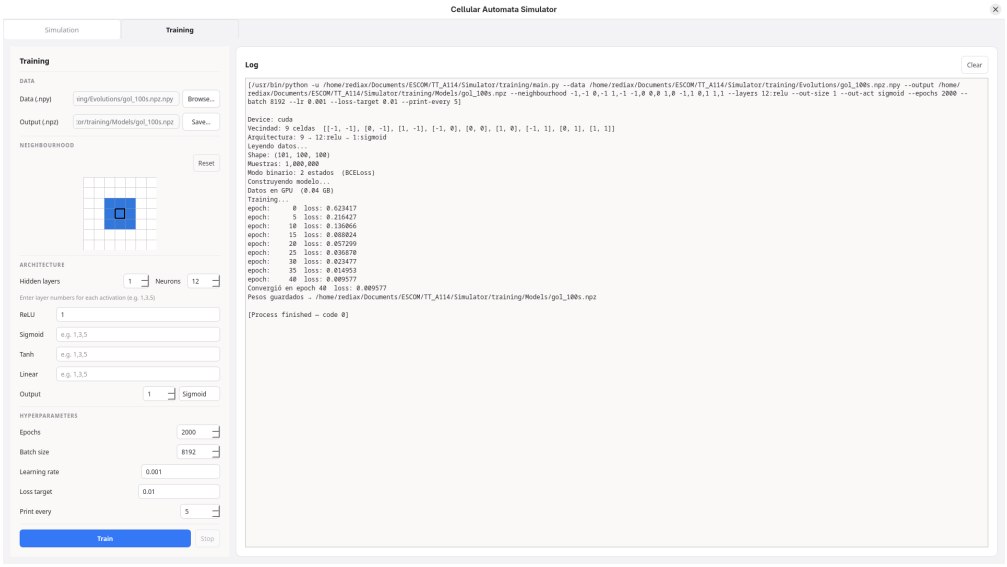


Figura 8.7: Interfaz de usuario. Pestaña de entrenamiento .

La primer sección permite realizar la configuración, la cual incluye la vecindad, numero de capas ocultas y funciones de activación en cada capa. así como definición de hiperparametros como numero de épocas, tasa de aprendizaje y perdida optima (ver figura 8.8). La segunda parte muestra unicamente los registros del entrenamiento.

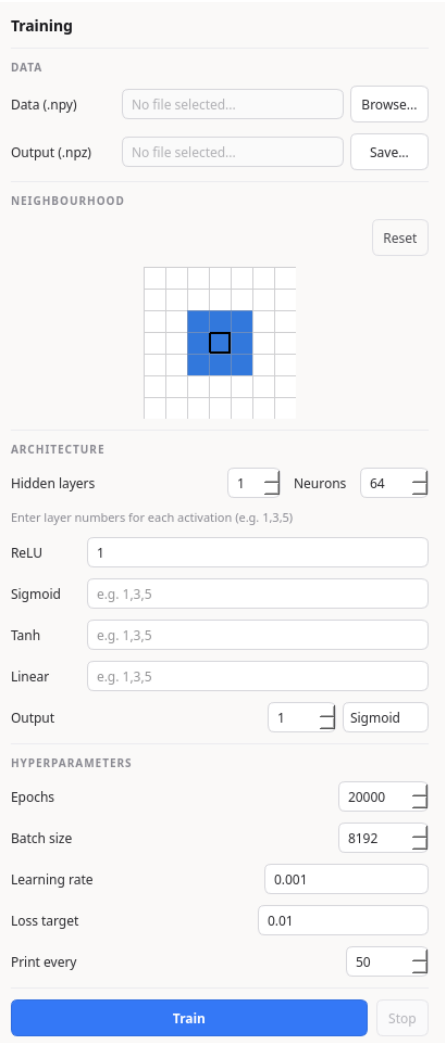


Figura 8.8: Controles de definición de entrenamiento.

Capítulo 9

Implementación

Durante el siguiente capítulo se explora el proceso de implementación de software. Se describe como fueron usadas las tecnologías, las versiones requeridas de las herramientas utilizadas, la estructura del proyecto, el proceso de entrenamiento de la red neuronal y termina con la validación del software, analizando las evoluciones obtenidas.

9.1. Entorno de desarrollo

El simulador se construyó mediante tres entornos de ejecución independientes; cada uno está asociado a una parte fundamental del proyecto o a una herramienta específica. Estos entornos son:

- **core**: Implementa el cómputo intensivo para la simulación del autómata celular y su versión neuronal, lo que requiere cálculos matriciales complejos que fueron delegados a CUDA por su alto poder computacional paralelo.
- **GUI**: La interfaz gráfica construida en Python utilizando PyQt5 que tiene instancia con el *binding* del core.
- **training**: Módulo de entrenamiento neuronal ejecutado en PyTorch como un proceso desacoplado.

El sistema fue desarrollado y probado en *Fedora Linux 44 Workstation Edition* con una GPU *NVIDIA* compatible con CUDA. Adicionalmente, en la tabla 9.1 se describen las versiones requeridas de las herramientas utilizadas.

Herramienta	**Versión**
Sistema operativo	Fedora Linux 44
Compilador C++	GCC 15 (*host compiler* para `nvcc`)
CUDA Toolkit	13.2
CMake	$\geq$ 3.15
Python	3.14
Pybind11	última versión estable
PyQt5	5.15
NumPy	$\geq$ 1.24
SciPy	$\geq$ 1.10
PyTorch	2.x con soporte CUDA

Tabla 9.1: Versiones de las herramientas del entorno global.

9.1.1. core

El módulo de *core* implementa ambos autómatas celulares en C++17 y CUDA. Estos se exponen a Python mediante *bindings*. Se compilan y generan los archivos `CA.cpython-*.so` y `NCA.cpython-*.so` que posteriormente son importados por GUI como un módulo nativo.

- **Lenguajes**: C++17 para la lógica y los *bindings*. CUDA C++17 para los *kernels* del dispositivo.
- **Sistema de *build***: CMake (El compilador para `nvcc` se fija a GCC 15, ya que HCC 16 no es compatible con CUDA 13.2).
- **Dependencias**: `pybind11` (instalado en un entorno virtual local), `CUDAToolkit` y encabezados de Python.
- **Arquitectura GPU**: la opción `CMAKE_CUDA_ARCHITECTURES native` permite que el compilador detecte la *Compute Capability* de la GPU presente al momento de la compilación.

El proceso de compilación y exportación del binding al directorio GUI se automatiza mediante el código `core/bind.sh`. Este *script* activa el entorno virtual, ejecuta CMake, invoca `make` y copia el artefacto en el directorio de la GUI.

9.1.2. GUI

La interfaz gráfica se desarrolla en Python sobre PyQt5. Es el único medio de interacción con el usuario.

- **Lenguaje**: Python 3.14.
- ***Framework* gráfico**: PyQt5 (*bindings* de Qt 5.15). Elegido por su modelo de señales y su soporte estable de `QImage` para el renderizado del espacio celular.
- **Dependencias numéricas**: NumPy para el manejo del arreglo del espacio celular; SciPy para la exportación al formato `.mat`.
- **Módulo nativo**: `CA.cpython-*.so` y `NCA.cpython-*.so`.

9.1.3. training

El entorno de entrenamiento es independiente de la GUI y del módulo nativo. No requiere CUDA en tiempo de compilación, ya que PyTorch gestiona el acceso a la GPU en tiempo de ejecución.

- **Lenguaje**: Python 3.x.
- ***Framework* de aprendizaje profundo**: PyTorch 2.x con soporte CUDA. La elección frente a TensorFlow o JAX se justifica por la facilidad de prototipado y el control explícito sobre el entrenamiento.
- **Invocación**: El *script* principal `training/main.py` se ejecuta de forma independiente o como subproceso lanzado por la pestaña de entrenamiento de la GUI.

9.2. Estructura del proyecto

El proyecto se organiza en tres directorios principales equivalentes a los tres entornos descritos en la sección anterior. Esto asegura el aislamiento de dependencias y la claridad en la lectura dentro del repositorio.

```
Simulator/
├── core/          # C++17/CUDA + bindings Pybind11
├── GUI/           # Aplicación PyQt5
└── training/      # Scripts de entrenamiento en PyTorch
```

9.2.1. Directorio core

Contiene la implementación de bajo nivel del autómata celular y del autómata celular neuronal.

Archivo	**Responsabilidad**
`ca.h` / `ca.cu`	Definición e implementación del autómata celular clásico en CUDA.
`nca.h` / `nca.cu`	Definición e implementación del autómata celular neuronal.
`bindings/python_module.cpp`	Capa Pybind11 que expone las clases `CA` y `NCA` al intérprete de Python.
`CMakeLists.txt`	Configuración del *build* con CMake.
`bind.sh`	*Script* de automatización.
`requirements.txt`	Dependencia única: `pybind11`.
`build/`	Directorio generado por CMake (no versionado).
`venv_bind/`	Entorno virtual local para el proceso de compilación.

Tabla 9.2: Contenido del directorio `core/`.

9.2.2. Directorio GUI

Implementa la aplicación gráfica completa siguiendo el patrón Modelo-Vista-Controlador descrito en el capítulo de Diseño.

Ruta	Responsabilidad
`main.py`	Instancia `QApplication` y `MainWindow`.
`models/ cellularAutomaton.py`	*Wrapper* de Python sobre la clase `CA` expuesta por Pybind11.
`models/ neuralCellularAutomaton.py`	*Wrapper* de Python sobre la clase `NCA`.
`controllers/ SimulationController.py`	Controlador principal. Coordina los *widgets*, el modelo y el *prefetch* asíncrono.
`controllers/ SimulationConfig.py`	Encapsulamiento de la configuración del autómata.
`ui/ MainWindow.py`	Ventana principal. Contiene un `QTabWidget` con las pestañas de simulación y entrenamiento.
`ui/ ControlPane.py`	Panel izquierdo de la pestaña de simulación: reproducción, paso, retroceso, borrado, aleatorización y velocidad.
`ui/ SimulationWidget.py`	*Widget* central. Renderiza la grilla con `QImage` en formato indexado y emite señales de *click*.
`ui/ ConfigPane.py`	Panel derecho. Configura vecindad, estados, frontera y regla de transición.
`ui/ NeighbourWidget.py`	Editor visual de la vecindad. Soporta múltiples grupos.
`ui/ TrainingPane.py`	Pestaña de entrenamiento. Captura hiperparámetros, lanza `QProcess` y muestra el *log* en tiempo real.

Tabla 9.3: Contenido del directorio `GUI/`.

9.2.3. Directorio training

Reúne los *scripts* de entrenamiento del autómata celular neuronal y las evoluciones de referencia utilizadas como conjuntos de datos.

Archivo / directorio	Responsabilidad
`main.py`	*Script* principal. Implementa la extracción de pares (vecindad, objetivo), la construcción dinámica del MLP en PyTorch y el ciclo de entrenamiento.
`test.py`	Implementación de referencia en NumPy y la compara contra la evolución esperada.
`png_to_evolution.py`	Utilidad para convertir secuencias de imágenes PNG segmentadas al formato `(T, H, W)` en `.npy`/`.mat`.
`Evolutions/`	Evoluciones registradas en formatos `.npy` (NumPy) y `.mat` (Wolfram Mathematica).
`Models/`	Pesos entrenados en formato `.npz` listos para ser cargados por la GUI.
`analysis.nb`	*Notebook* de Wolfram Mathematica utilizado para análisis y verificación visual.
`training_env/`	Entorno virtual con PyTorch y dependencias auxiliares.

Tabla 9.4: Contenido del directorio `training/`.

9.2.4. Flujo de artefactos entre directorios

La figura conceptual del flujo de artefactos es la siguiente:

1. El *build* en `core/build/` produce `CA.cpython-*.so`, que se copia al directorio `GUI/`.
2. La GUI registra evoluciones mediante el botón de grabación de `ControlPane` y las exporta a `.npy`/`.mat` en cualquier ubicación, típicamente `training/Evolutions/`.
3. El *script* de entrenamiento lee dichos archivos y produce un `.npz` con pesos, sesgos y metadatos.
4. La GUI carga el `.npz` desde `SimulationController` e intercambia al modo NCA para reproducir la evolución aprendida.

Este flujo unidireccional garantiza que los tres entornos puedan evolucionar de forma independiente: un cambio en el módulo nativo no requiere reentrenar, y un cambio en el entrenamiento no requiere recompilar.

9.3. Entrenamiento del modelo

El entrenamiento del autómata celular neuronal consiste en ajustar los pesos y sesgos del perceptrón multicapa para que, dado el vector de estados en la vecindad de una celda en el instante t, produzca como salida el estado de esa misma celda en el instante $t + 1$.

Esta sección describe la implementación concreta del pipeline de entrenamiento: la preparación del conjunto de datos, la construcción dinámica del modelo, el ciclo de optimización, la persistencia de los pesos y la integración con la interfaz gráfica.

9.3.1. Preparación del conjunto de datos

El entrenamiento parte de una evolución registrada previamente desde la GUI o generada por una utilidad externa. El archivo se almacena en formato NumPy con *shape* (T, H, W) y `dtype` entero, donde T es el número de pasos temporales registrados.

A partir de esta evolución se extraen pares de entrenamiento a nivel de celda, no a nivel de imagen. Para cada celda (x, y) en cada instante $t \in [0, T - 1)$ se construye:

- **Entrada** $X_{t,x,y}$: vector de tamaño N_{vec} que concatena el estado de las celdas indicadas por la vecindad relativa a (x, y) en el instante t. Las coordenadas se resuelven con condición de frontera periódica (módulo H y W).
- **Salida esperada** $Y_{t,x,y}$: estado de la celda (x, y) en el instante $t + 1$.

Esta vectorización transforma una evolución de T pasos sobre una grilla $H \times W$ en un conjunto de $T \cdot H \cdot W$ muestras independientes, lo que multiplica la cantidad de datos por celdas evaluadas y permite entrenar con *batches* grandes sin requerir múltiples evoluciones.

9.3.2. Construcción dinámica del modelo

La arquitectura del perceptron multicapa no está fijada en el código: se determina en tiempo de ejecución a partir de los argumentos de la línea de comandos (o

de la pestaña de entrenamiento de la GUI). El *script* `training/main.py` acepta los parámetros resumidos en la tabla 9.5.

Argumento	**Default**	**Descripción**
`-data`	—	Archivo `.npy` con la evolución de entrada.
`-output`	`weights.npz`	Archivo de salida con los pesos entrenados.
`-neighbourhood`	—	Lista de pares `"dx,dy"` separados por espacio.
`-radius`	1	Vecindad cuadrada $(2r+1) \times (2r+1)$ si no se especifica `-neighbourhood`.
`-layers`	—	Capas ocultas con activación, ejemplo `"64:relu 32:tanh"`.
`-hidden`	64	Tamaño de capa oculta única (si no se usa `-layers`).
`-out-size`	1	Neuronas de la capa de salida.
`-out-act`	`sigmoid`	Activación de la capa de salida.
`-epochs`	20000	Número máximo de épocas.
`-batch`	8192	Tamaño de *batch*.
`-lr`	0.001	Tasa de aprendizaje del optimizador.
`-loss-target`	0.01	Umbral de pérdida para detención temprana.
`-print-every`	50	Frecuencia de impresión del *log*.

Tabla 9.5: Argumentos CLI del *script* de entrenamiento.

A partir de estos argumentos se construye un `torch.nn.Sequential` concatenando capas `nn.Linear` con la activación correspondiente. Las activaciones soportadas son `ReLU`, `Sigmoid`, `Tanh` y `Linear` (identidad), las mismas que reconoce el *kernel* CUDA del NCA. La equivalencia de códigos numéricos (`0=ReLU, 1=Sigmoid, 2=Tanh, 3=Linear`) garantiza que los pesos entrenados sean directamente ejecutables en el dispositivo.

9.3.3. Ciclo de optimización

El entrenamiento sigue un esquema de optimización por descenso de gradiente estocástico con minilotes:

- **Optimizador**: `torch.optim.Adam` con la tasa de aprendizaje configurada. Se eligió *Adam* por su robustez frente a la elección inicial de hiperparámetros y por su comportamiento estable en presencia de gradientes ruidosos.
- **Función de pérdida**: error cuadrático medio (`MSELoss`). Aunque el objetivo es discreto en los autómatas celulares clásicos, el MLP produce salidas continuas y se interpreta el valor de salida mediante umbralización al momento de inferencia.
- **Detención temprana**: el ciclo se interrumpe cuando la pérdida acumulada por época cae por debajo de `loss_target`, evitando épocas adicionales innecesarias una vez que el modelo ha convergido.
- ***Logging***: cada `print_every` épocas se imprime el número de época, la pérdida acumulada. Este *log* se captura en tiempo real por la GUI cuando el entrenamiento se lanza desde `TrainingPane`.

9.3.4. Persistencia de los pesos

Al finalizar el entrenamiento, el modelo se almacena en un archivo `.npz` con la siguiente estructura:

- `weights_N` y `biases_N` por cada capa lineal (con N el índice de capa).
- `layer_in`, `layer_out`: tamaños de entrada y salida.
- `activations`: arreglo de códigos enteros para cada capa.
- `neighbourhood`: lista de desplazamientos relativos utilizada durante el entrenamiento.

Este formato es el que espera el método `on_load_nca_button` de `SimulationController` y es compatible con la firma del constructor de la clase `NCA` en el módulo nativo, sin necesidad de transformaciones intermedias.

9.3.5. Integración con la interfaz gráfica

Desde la pestaña de entrenamiento de la GUI, el usuario configura visualmente la vecindad, las capas ocultas, las activaciones y los hiperparámetros. Al pulsar el botón de entrenamiento, la GUI ensambla la línea de comandos equivalente e instancia un `QProcess` con la *flag* `-u` de Python, que desactiva el *buffering* de la salida estándar. La salida estándar y de error del subproceso se conecta mediante señales de Qt a un *widget* de *log* que se actualiza en tiempo real.

Esta arquitectura permite que el entrenamiento se ejecute en un proceso completamente independiente del de la GUI: una falla durante el entrenamiento no afecta a la interfaz, y el usuario puede continuar simulando o ajustando la configuración mientras el proceso avanza en segundo plano. Una vez terminado, el archivo `.npz` resultante puede cargarse desde la pestaña de simulación para intercambiar al modo NCA.

9.4. Validación y pruebas

La validación del simulador se realiza en tres partes: primero se comprueba el funcionamiento del autómata celular replicando reglas conocidas; después se realiza una verificación numérica del autómata celular neuronal y finalmente se realizan mediciones de rendimiento.

9.4.1. Validación funcional del autómata celular clásico

El autómata celular se valida reproduciendo reglas ampliamente documentadas en la literatura y comparando visual y numéricamente la evolución obtenida con la esperada.

Para esto se establece la misma condición inicial en dos diferentes simuladores y se evolucionan bajo las mismas condiciones. Si la diferencia de las evoluciones en cada paso evolutivo es 0, se infiere que el comportamiento es el mismo, pues sus trayectorias en el espacio de evoluciones son equivalentes.

9.4.2. Validación del autómata celular neuronal

El criterio de validación del entrenamiento es la capacidad del modelo entrenado para replicar la regla original. Para cada regla objetivo se realiza el siguiente procedimiento:

1. Generar una evolución de T pasos con el autómata celular clásico.
2. Entrenar el perceptrón multicapa con dicha evolución.
3. Cargar los pesos resultantes en el modo NCA del simulador.
4. Ejecutar una nueva evolución partiendo de la misma configuración inicial y comparar.

La métrica principal es el porcentaje de celdas coincidentes entre las dos evoluciones a lo largo de los T pasos. Una coincidencia superior al 99 % en una vecindad completa se interpreta como que el autómata celular neuronal fue capaz de replicar exitosamente la regla.

9.4.3. Pruebas de rendimiento

Para las pruebas de rendimiento en la simulación se utilizó la regla B3/S23 y con una población aleatoria con 50 % de células vivas. El rendimiento se analiza dentro del método `step()` que es quien ejecuta el paso evolutivo y se sincroniza con la GPU y el método `run()`, que ejecuta N pasos consecutivos.

n	`step()` (steps/s)	`run(100)` (steps/s)
100	345 000	102 800
500	313 000	6 500
1000	137 000	1 800
2000	55 400	520

Tabla 9.6: Velocidad de evolución medida sobre grillas $n \times n$. `step()` mide únicamente el *kernel* y el intercambio de buffers (sincronización perezosa). `run()` incluye la copia *device-to-host* por *frame*, que constituye el techo real para la reproducción visible.

La caída de `run()` respecto a `step()` cuantifica el costo de la transferencia *device-to-host*: en grillas de 2000 × 2000 la transferencia domina por encima del cómputo.

9.4.3.1. Velocidad de renderizado

El renderizado se mide en modo `offscreen` de *Qt Platform Abstraction* para aislar el costo de dibujar del costo del sistema de ventanas. Se contabiliza el tiempo medio entre `update()` y la finalización del *repaint*.

n	ms / *repaint*	fps
100	2.66	375
500	0.89	1 125
1000	2.83	353
2000	8.18	122

Tabla 9.7: Velocidad de renderizado para distintos tamaños de grilla.

Incluso a 2000×2000 celdas (4 millones de celdas), el renderizado supera los 100 *fps*, lo que confirma la viabilidad del enfoque basado en `QImage(Format_Indexed8)` con una sola llamada a `drawImage` por *frame*.

9.4.4. Pruebas de interfaz

Las pruebas de la interfaz gráfica se realizaron de forma manual siguiendo guiones derivados de los casos de uso. Los escenarios principales validados son:

- Configuración de una regla desde cero: definir estados, vecindad multigrupo y tabla de regla, y verificar la evolución resultante.
- Edición interactiva del espacio celular: pintar con el mouse, borrar y aleatorizar, comprobando que la grilla mostrada coincide con el estado interno del autómata.
- Importación y exportación de evoluciones: grabar una evolución, exportarla a `.npy` y `.mat`, reimportarla y verificar que la secuencia reproducida es idéntica.
- Carga de un modelo NCA entrenado y conmutación entre los modos CA y NCA durante la simulación.

- Lanzamiento del subproceso de entrenamiento desde la pestaña correspondiente, verificando la captura de la salida en tiempo real y la correcta finalización del archivo `.npz`.

9.4.5. Robustez y manejo de errores

A lo largo del desarrollo se identificaron y resolvieron algunos errores:

- **Errores de CUDA silenciosos**: todas las llamadas a la API de CUDA se envuelven en una macro `CUDA_CHECK` que verifica el código de retorno y aborta con un mensaje descriptivo, evitando segmentos de fallo retardados que dificultan la depuración. La macro se omite deliberadamente en los destructores para no propagar excepciones durante el desensamblado.
- **Tamaño de tabla excesivo**: cambios en `states` o en `group_sizes` pueden generar tablas de regla que excederían los límites técnicos del dispositivo. La capa de *bindings* prevalida $r_{\text{size}} \leq 16\,777\,216$ antes de aplicar cualquier cambio.
- **Invalidación del *prefetch***: el contador `_generation` en el controlador descarta de forma segura los *frames* precomputados cuyo trabajo ha quedado obsoleto, evitando inconsistencias visuales.
- **Mismatch entre el *driver* y la biblioteca de CUDA**: Incidentes documentados durante el desarrollo recomendaron reiniciar el sistema tras actualizaciones parciales del *driver NVIDIA*.

9.4.6. Cambios

Durante el desarrollo de la investigación, estos fueron los cambios requeridos en el software que no fueron contemplados:

- Se añadió un campo opcional en el modulo de entrenamiento que es la semilla, esto para poder replicar experimentos y ademas, para poder realizar multiples pruebas con diferentes semillas aleatorias.

Capítulo 10

Experimentación y resultados

Una vez construido, validado e implementado el simulador, es momento de la experimentación y del análisis de resultados. A lo largo de este capítulo se exponen los tres principales experimentos realizados utilizando el simulador para generar evoluciones con un autómata celular neuronal para analizarlas en Mathematica.

10.1. Duplicación de reglas del juego de la vida

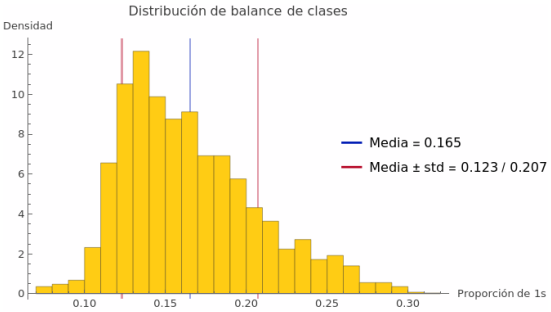
El objetivo de este primer experimento es replicar el comportamiento de evoluciones conocidas para identificar la capacidad del autómata celular neuronal de replicar comportamientos con base en su evolución y su vecindad.

Cada regla es un problema diferente. Por lo tanto, la generación de datos es única por configuración, es por esto que las siguientes pruebas están enfocadas únicamente en la regla B3/S23.

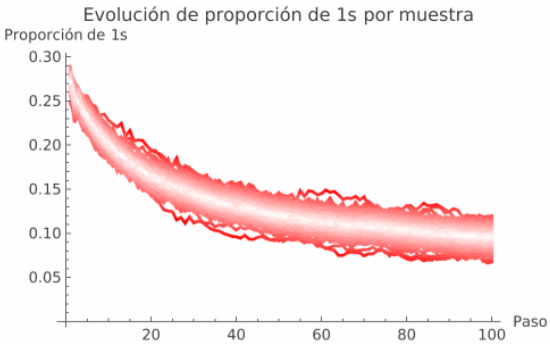
El entrenamiento de la red neuronal requiere una ingesta de datos balanceados, para realizar las mediciones, fueron evolucionadas 100 condiciones iniciales aleatorias en un espacio de 100×100 durante 100 pasos evolutivos.

De acuerdo con la figura 10.1a, la regla B3/S23 produce el estado 1 un 16.5 % de las veces. Teniendo en cuenta la desviación estándar, concluimos que en la mayoría de los escenarios este porcentaje se encuentra entre 12.3 % y 20.7 % del total de los datos, un desbalanceo moderado-bajo que resulta manejable por la red neuronal. La mayoría de los valores mayores a estos porcentajes son obtenidos por las condiciones iniciales que inician en un 50 %. Esto se demuestra en la figura

10.1b donde la proporción del estado 1 de todas las evoluciones a través del tiempo decrece hasta estabilizarse, comportamiento característico del atractor de dicha regla.



(a) Histograma de distribución de la clase 1. Media de 16.5 y desviación estándar de 4.2.



(b) Proporción de unos a lo largo del tiempo de todas las muestras.

Figura 10.1: Resultados estadísticos del conjunto de pruebas para la regla B3/S23.

Adicionalmente, dado que la salida de cada paso evolutivo es la entrada del siguiente, también se analiza el desbalanceo de entradas, es decir, la frecuencia con la que diferentes patrones aparecen. Para esto, utilizaremos la nomenclatura $CxNy$ para contar el número de unos que aparecen en el centro (C) y en el exterior (N).

En la figura 10.2 se muestran las frecuencias de aparición de todos los patrones en el mismo conjunto de datos. Se observa el desbalanceo y una propiedad de la regla B3/S23 de no generar mucha vida dentro de las comunidades. Los patrones con menor frecuencia de aparición son $C1N8$ y $C0N8$ sumando menos del 0.02 % del total de patrones, lo que causa un problema de representación o sesgo de muestra.

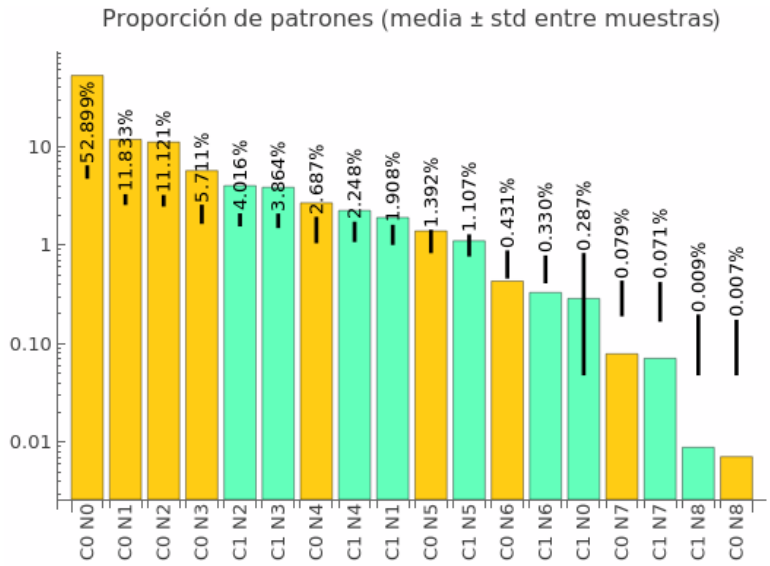


Figura 10.2: Distribuciones de probabilidad de los patrones en la regla B3/S23.

Para combatir esto, existen dos soluciones: (1) realizar un sobremuestreo de los datos menos frecuentes o (2) modificar la función de costo para penalizar en mayor medida los errores en patrones poco representados. Las próximas pruebas buscan disminuir la perdida objetivo con la intención de encontrar configuraciones capaces de replicar el comportamiento de forma exacta.

Considerando una red neuronal de 9 neuronas en la capa de entrada, se probaron 10 combinaciones de funciones de activación, de las cuales una sola converge en menos de 1000 épocas, tal como se observa en la tabla 10.1. La falta de convergencia puede deberse al tamaño de la red neuronal o la pérdida objetivo utilizada.

Si incrementamos el número de capas ocultas y de neuronas, se observa en la tabla 10.2 que existen cinco configuraciones que alcanzan la convergencia. Adicionalmente, en la figura 10.3 observamos que el estancamiento ocurre en las primeras 12 épocas, lo que sugiere una limitación de la tasa de aprendizaje.

Capa Oculta	Capa Salida	Épocas	Pérdida	Tiempo
1:4:ReLU	2:Linear	122	0.000098	4.64 s
1:4:TanH	1:Sigmoid	1000	0.052895	40.19 s
1:4:TanH	2:Linear	1000	0.079425	36.52 s
1:4:Sigmoid	2:Linear	1000	0.159343	36.71 s
1:4:Sigmoid	1:Sigmoid	1000	0.159343	39.37 s
1:4:ReLU	2:TanH	1000	0.179589	38.65 s
1:4:ReLU	1:Sigmoid	1000	0.269196	38.16 s
1:4:Linear	1:Sigmoid	1000	0.297529	39.12 s
1:4:Linear	2:Linear	1000	0.297529	34.75 s
1:4:ReLU	2:Sigmoid	1000	0.374756	37.55 s

Tabla 10.1: Comparativa de rendimiento con múltiples hiperparámetros de la regla B3/S23: Pérdida objetivo 0.0001, límite de épocas: 1000, tamaño del lote: 8192, tasa de aprendizaje: 0.001, semilla aleatoria: 42.

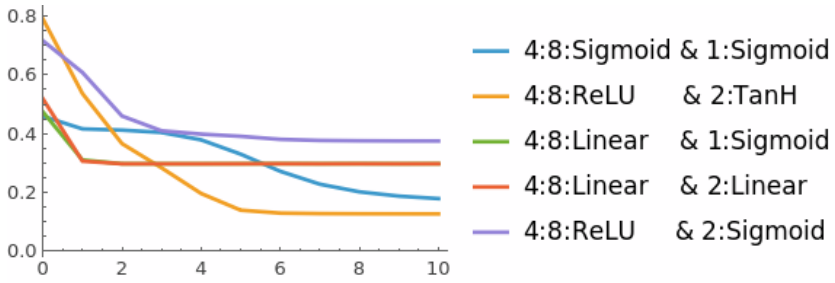


Figura 10.3: Registro de pérdida obtenida en los primeros 12 pasos de las configuraciones sin convergencia de la tabla 10.2.

Al haber encontrado configuraciones de la red neuronal que son capaces de converger con una pérdida de 0.0001. Queda demostrado que un autómata celular neuronal puede aprender el comportamiento de una regla ya conocida con un error mínimo. Para el objetivo de esta investigación, resulta innecesario experimentar con el resto de configuraciones no convergentes; las configuraciones descubiertas serán utilizadas para los próximos experimentos.

De acuerdo con la figura 10.2, la probabilidad de aparición de los patrones $C0N8$ y $C1N8$ son casi imperceptibles (menos del 0.02 %). Con las siguientes pruebas se busca identificar cómo afecta la variación de la pérdida objetivo con la regla aprendida. Para esto, entrenaremos la regla B3/S23 con diferentes perdidas objetivo e identificaremos si dicha probabilidad de aparición afecta en el

Capa Oculta	**Capa Salida**	**Épocas**	**Pérdida**
4:8:ReLU	2:Linear	18	0.000087
4:8:TanH	1:Sigmoid	51	0.000097
4:8:TanH	2:Linear	35	0.000097
4:8:Sigmoid	2:Linear	79	0.000095
4:8:Sigmoid	1:Sigmoid	1000	0.159342
4:8:ReLU	2:TanH	1000	0.126928
4:8:ReLU	1:Sigmoid	133	0.000096
4:8:Linear	1:Sigmoid	1000	0.297542
4:8:Linear	2:Linear	1000	0.297544
4:8:ReLU	2:Sigmoid	1000	0.374756

Tabla 10.2: Comparativa de rendimiento con múltiples hiperparámetros de la regla B3/S23: Pérdida objetivo 0.0001, límite de épocas: 1000, tamaño del lote: 8192, tasa de aprendizaje: 0.001, semilla aleatoria: 42.

aprendizaje.

Aunque las reglas del juego de la vida son semitotalísticas, cabe la posibilidad de que la regla aprendida no lo sea, es decir, el autómata celular neuronal puede hacer distinción entre dos patrones que tengan la misma forma. Por ejemplo, podemos generar de 8 formas diferentes el patrón $C0N1$:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \cdots = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

pero a falta de exposición de todas las posibles configuraciones, es posible que la red neuronal marque una distinción en uno o varios de esos elementos, convirtiendo a la regla en una tradicional.

Como se mencionó en el capítulo 3, las reglas tradicionales y las semitotalísticas son subconjuntos mutuos. Para verificar que la regla aprendida coincida con todas las posibles configuraciones de los patrones, se diseñó el espacio de la figura 10.4, se evoluciona un paso y se comparan los centros de cada permutación.

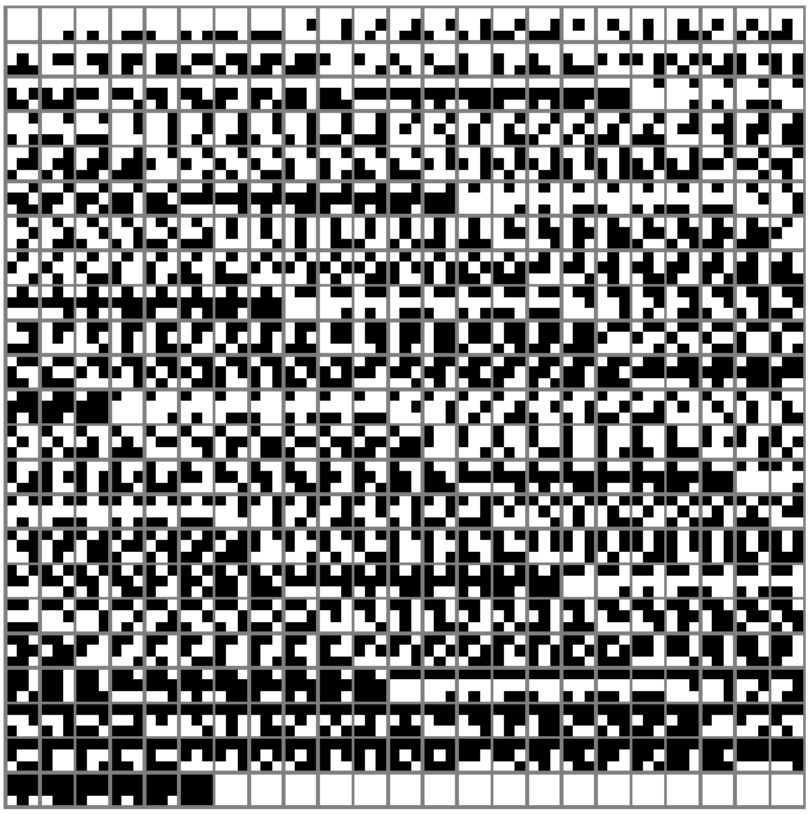


Figura 10.4: Espacio que contiene todas las permutaciones posibles de 2 estados en 9 células.

De acuerdo con la figura 10.5, la pérdida objetivo de la regla B3/S23 tiene que ser menor o igual a 0.1.

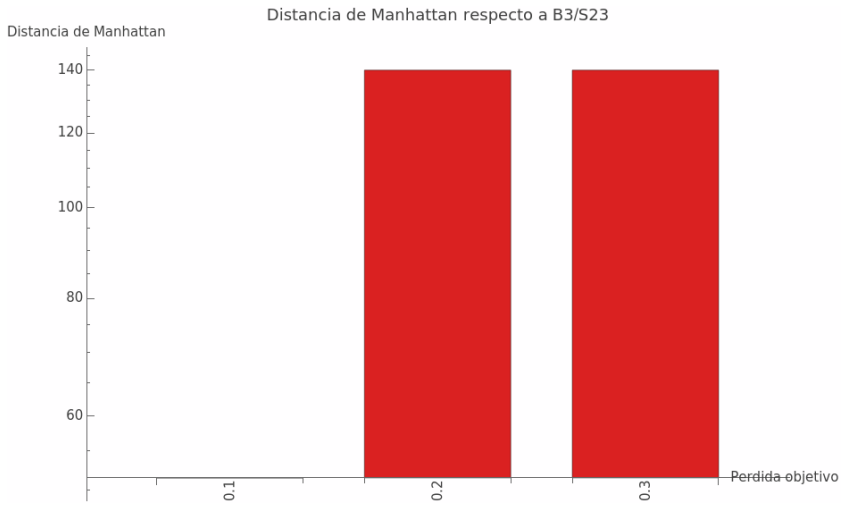


Figura 10.5: Distancia Manhattan de los vectores de regla de múltiples perdidas objetivo comparadas con B3/S23.

Los autómatas celulares neuronales que utilizaron la pérdida objetivo de 0.2 y 0.3 aprendieron la regla 0, es decir, todas las configuraciones evolucionan a 0. Podemos suponer que esta configuración es estable porque, aunque existan patrones de baja frecuencia como $C0N8$ y $C1N8$, sigue siendo capaz de replicar el comportamiento con una pérdida objetivo de 0.1. Una hipótesis es que la red neuronal se inicializa para generar ceros, al no recibir nunca esas entradas, se mantienen en 0.

Al entrenar con la regla B3/S23 el autómata celular neuronal no aprende ningún otro comportamiento. A continuación se muestra la comparativa de las pérdidas objetivo de las reglas B3/S238 (Figura 10.6) B38/S23 (10.7), B3/S238 (10.8) y B38/S238 (10.9).

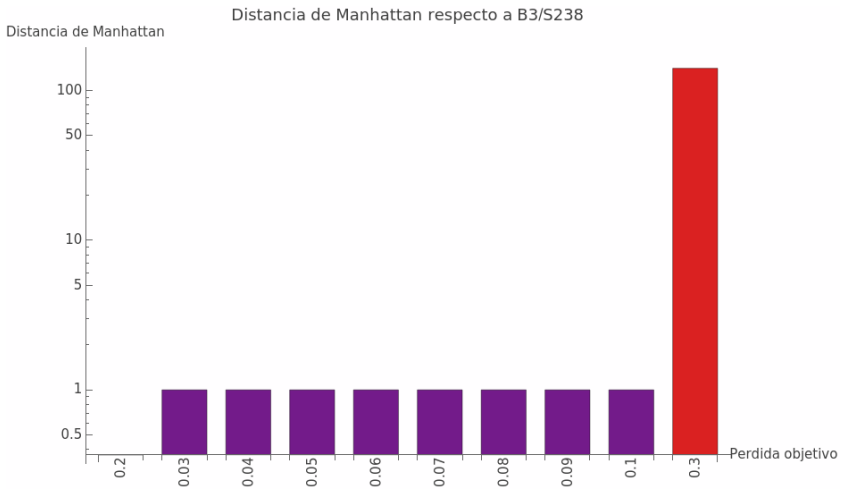


Figura 10.6: Distancia Manhattan de los vectores de regla de múltiples pérdidas objetivo comparadas con B3/S238.

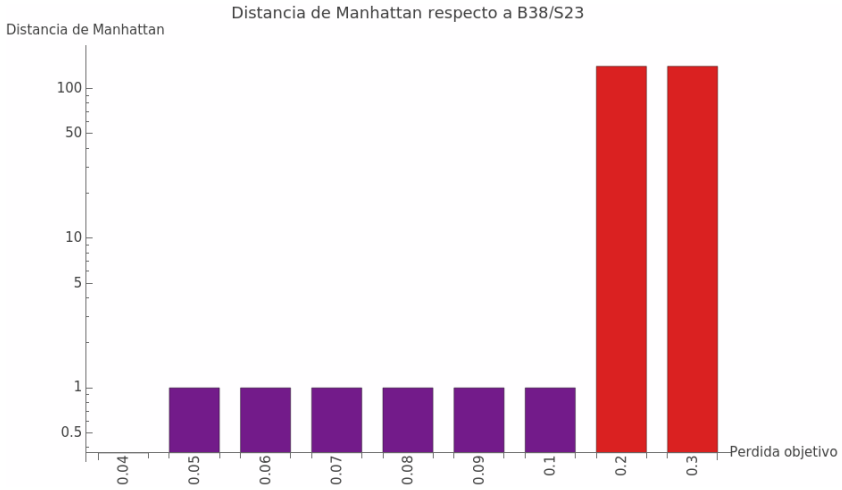


Figura 10.7: Distancia Manhattan de los vectores de regla de múltiples pérdidas objetivo comparadas con B38/S23.

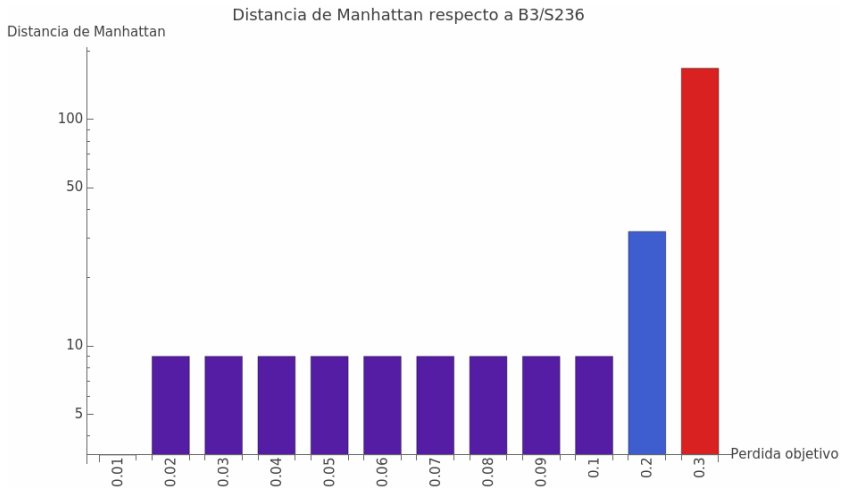


Figura 10.8: Distancia Manhattan de los vectores de regla de múltiples pérdidas objetivo comparadas con B3/S236.

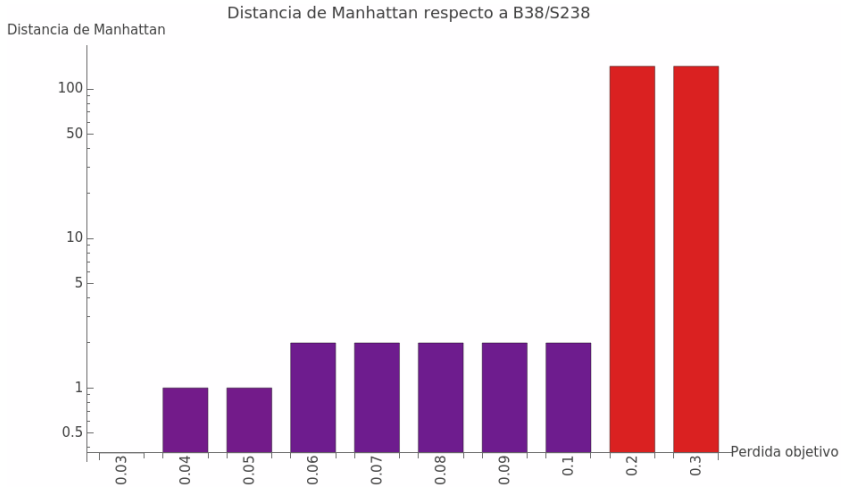


Figura 10.9: Distancia Manhattan de los vectores de regla de múltiples pérdidas objetivo comparadas con B38/S238.

La regla B3/S238 suele aprender otra regla que difiere únicamente en un elemento. Esta única diferencia pertenece al patrón $C1N8$ el cual solo tiene una posible representación:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

es decir, no considera la supervivencia con 8 vecinos, por lo que su regla aprendida es B3/S23. Esto lo corroboramos con la figura 10.10, en donde se muestra la única diferencia entre ambas reglas y cómo fue omitida durante el entrenamiento. Este mismo fenómeno ocurre en la regla B38/S238.



Figura 10.10: Últimos 50 elementos de los vectores de regla de B3/S238, B3/S23 y los aprendidos por el autómata celular neuronal.

Para la regla B38/S238 se aprendieron 2 comportamientos diferentes, la regla B3/S238 y B3/S23, como se observa en la figura 10.11.



Figura 10.11: Últimos 50 elementos de los vectores de regla de B38/S238, B3/S23, B3/S238 y los aprendidos por el autómata celular neuronal.

En la regla B3/S236 existe un comportamiento diferente. Por un lado, con la pérdida objetivo de 0.2 difiere en 32 elementos. 28 de ellos (figura 10.12a) aprendió un 0 en lugar de 1, todos estos patrones corresponden a $C1N6$, por lo que si fuera solo por estas diferencias, hubiera aprendido la regla B3/S23. Sin embargo, también tuvo diferencias en 4 patrones más, donde intercambió el 1 por el 0 (figura

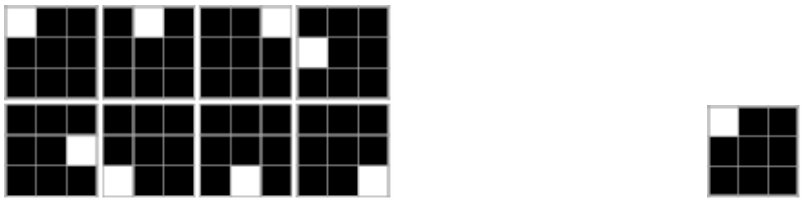
10.12b). Sin embargo, nótese que, aunque esos 4 patrones pertenecen a $C1N7$, faltan combinaciones. Por esta razón, la regla aprendida no es semitotalística, porque hace distinción de los elementos de su vecindad.



(a) Se intercambió 1 por 0 en $C1N6$ (b) Se intercambió 0 por 1.

Figura 10.12: Patrones que difieren entre B3/S236 y el autómata celular neuronal con una pérdida objetivo de 0.2.

Por otro lado, todas las reglas aprendidas erróneamente difieren en los patrones de las figuras 10.13a y 10.13b. Ambos patrones se encuentran completos ($C1N7$ y $C1N8$), por lo que ahora sobrevive con 7 y 8 vecinos. La regla aprendida es B3/S23678.



(a) Se intercambió 0 por 1 en $C1N7$ (b) Se intercambió 0 por 1 en $C1N8$

Figura 10.13: Patrones que difieren entre B3/S236 y el autómata celular neuronal con una pérdida objetivo de 0.1.

Los resultados de este primer experimento permiten establecer las siguientes consideraciones clave sobre el entrenamiento de autómatas celulares neuronales:

- La capacidad de generalización de la red neuronal depende del sesgo de muestra por la baja frecuencia de aparición de ciertos patrones (como $C0N8$ o $C1N8$). Es por esto que establecer una pérdida objetivo estricta fuerza a la red a ajustar sus pesos en regiones del espacio donde hay patrones menos representados. El caso contrario atenúa estos estados y lo conduce a otro atractor.

- Se comprueba que el entrenamiento de reglas semitotalísticas no garantiza que la regla aprendida pertenezca al mismo conjunto. Esto se observó en el caso B3/S236 con pérdida de 0.2, ya que no se expone a todas las permutaciones rotacionales de la vecindad. Por lo que patrones equivalentes son tratados de forma independiente, rompiendo la simetría propia de la vecindad.

Estos experimentos confirman que no se requieren arquitecturas neuronales complejas para replicar este tipo de dinámicas evolutivas. Asimismo, queda validada la metodología propuesta para extraer e identificar el vector de regla aprendido por el modelo, sentando las bases para los siguientes experimentos del capítulo.

Bibliografía

[1] W. MathWorld, "Cellular automaton," n.d., consultado el 28 de Marzo del 2025. [Online]. Available: https://mathworld.wolfram.com/CellularAutomaton.html

[2] H. Zenil and G. J. Martínez, "Cellular automata," *Scholarpedia*, vol. 19, no. 4, p. 53227, 2024. [Online]. Available: http://www.scholarpedia.org/article/Cellular_automata

[3] D. A. R. Gómez, "Descripción y aplicaciones de los autómatas celulares," 2011, consultado el 27 de Marzo del 2025. [Online]. Available: https://delta.cs.cinvestav.mx/~mcintosh/cellularautomata/Summer_Research_files/Arti_Ver_Inv_2011_DARG.pdf

[4] M. O. Domínguez, "Redes neuronales artificiales," *Ingenio y Conciencia Boletín Científico de la Escuela Superior Ciudad Sahagún*, vol. 12, no. 23, pp. 38–44, 2025. [Online]. Available: https://doi.org/10.29057/escs.v12i23.14132

[5] D. J. Matich, "Redes neuronales: Conceptos básicos y aplicaciones." n.d., consultado el 28 de Marzo del 2025. [Online]. Available: https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf

[6] D. Pub, "Growing cellular automata," 2020, consultado el 27 de Marzo del 2025. [Online]. Available: https://distill.pub/2020/growing-ca/

[7] S. E. Yacoubi, T. Plénet, S. Dridi, F. Bagnoli, L. Lefèvre, and C. Raïevsky, "Some control and observation issues in cellular automata," *Complex Systems*, vol. 30, no. 3, pp. 391–413, Sep. 2021, accedido el 14 de junio de 2025. [Online]. Available: https://doi.org/10.25088/complexsystems.30.3.391

[8] W. Gilpin, "Cellular automata as convolutional neural networks," *Physical Review E*, n.d., consultado el 29 de Marzo del 2025. [Online]. Available: https://journals.aps.org/pre/abstract/10.1103/PhysRevE.100.032402

[9] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, 2020, accedido el 29 de julio de 2025. [Online]. Available: https://distill.pub/2020/growing-ca

[10] ——, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[11] Instituto Nacional de Estadística e Informática, "¿qué es la teoría general de sistemas?" INEI - Sub-Jefatura de Informática, Lima, Perú, Manual, 1999, dirección Técnica Desarrollo Informático. Orden: 604-99-0I-OTDETI-INEI. [Online]. Available: https://cmapspublic.ihmc.us/rid=1G8TFJM82-16RKYJR-M62/TGS.pdf

[12] M. Arnold-Cathalifaud and F. Osorio, "Introducción a los conceptos básicos de la teoría general de sistemas," *Cinta de Moebio*, no. 3, mar 1998. [Online]. Available: https://www.redalyc.org/articulo.oa?id=10100306

[13] S. C. Milagros. (2024, jul) Ludwig von bertalanffy y su teoría general de los sistemas. Consultado el 26 de marzo de 2026. [Online]. Available: https://neuro-class.com/bertalanffy-y-su-tgs/

[14] R. R. Solano, "Orígenes de la teoría de sistemas," Universidade Federal do Paraná (UFPR), Material de apoyo, 2019, departamento de Informática, Curso CI-220: Teoria dos Sistemas. [Online]. Available: https://www.inf.ufpr.br/urban/2019-1_220/220_Ler_Estudar_Discutir/Solano__Origines_Teoria_Sistemas.pdf

[15] A. M. Caraballo Payares, *Evaluación de la gestión administrativa, financiera y comercial de las cooperativas asociadas a CONFECOOP Caribe.* eumed.net, 2013, sección: Teoría de los Sistemas. [Online]. Available: https://www.eumed.net/libros-gratis/2013a/1314/sistemas.html

[16] Equipo editorial Concepto. (2025) Sistema - características, tipos y ejemplos. Accedido el 8 de nov. de 2025. [Online]. Available: https://concepto.de/sistema

[17] J. D. Monsalve Ospina, "La dualidad de sistemas: sistemas abiertos y cerrados en la teoría general de sistemas," 2023, accedido el 8 de nov. de 2025. [Online]. Available: https://udecolombia.edu.co/wp-content/uploads/2024/12/Monsalve-JD.-2023_TGS-Sistemas-abiertos-y-cerrados.pdf

[18] M. S. Rojas, "Determinísticos y probabilísticos," 2018, accedido el 8 de nov. de 2025. [Online]. Available: https://www.scribd.com/document/380696672/Deterministicos-y-Probabilisticos

[19] Anónimo, "Teoría general de sistemas: Sistemas Estáticos - Dinámicos," 2025, accedido el 9 de nov. de 2025. [Online]. Available: https://8d8a87251d.cbaul-cdnwnd.com/b3abc3d20220b43c1b18cf3768b8f6e1/200000090-2218722189/Sistemas%20Estaticos%20-%20Dinamicos-0.pdf

[20] A. O. Cordero, "Introducción a sistemas dinámicos," 2025, accedido el 8 de nov. de 2025. [Online]. Available: https://umi.cinvestav.mx/Portals/UMI_LAB/Maestria%20material%20de%20apoyo/SeÃśales%20y%20sistemas%201.pdf

[21] E. E. Alberto, H. E. Tenoch, and E. E. V. Romero, "Exploración de universos discretos con autómatas celulares en dos dimensiones," 2021, accedido el 8 de nov. de 2025. [Online]. Available: https://www.comunidad.escom.ipn.mx/genaro/Papers/Thesis_files/tesisEricEduardoAlbertoTT2019-B067.pdf

[22] I. Altadill Simón and D. R. García Belinchón, "La teoría del caos," 2025, accedido el 8 de nov. de 2025. [Online]. Available: https://repositorio.unican.es/xmlui/bitstream/handle/10902/30650/AltadillSimonIgnacio.pdf

[23] J. Torres, "Introducción a los sistemas complejos," 2025, accedido el 8 de nov. de 2025. [Online]. Available: https://www.ugr.es/~jtorres/tema_1_sistemas_complejosv2.pdf

[24] I. Zelinka, A. Sanayei, H. Zenil, and O. E. Rössler, *How Nature Works*. Heidelberg: Springer International Publishing, 2014.

[25] I. Zelinka and J. Burguillo, "Complex systems," 01 2018, pp. 11–34.

[26] R. V. Solé and S. C. Manrubia, *Orden y caos en sistemas complejos*. Barcelona: Edicions UPC, 1996. [Online]. Available: https://unamer34.wordpress.com/wp-content/uploads/2008/09/ocsc.pdf

[27] S. Moriello. (2003) Sistemas complejos, caos y vida artificial. Accedido el 23 de sept. de 2025. [Online]. Available: https://www.redcientifica.com/doc/doc200303050001.html

[28] Gamco. (2024) Simulación: Definición y aplicaciones en el ámbito empresarial. Consultado el 28 de marzo de 2026. [Online]. Available: https://gamco.es/glosario/simulacion/

[29] L. R. Izquierdo, J. M. Galán Ordax, J. I. Santos, and R. del Olmo Martínez, "Modelado de sistemas complejos mediante simulación basada en agentes y mediante dinámica de sistemas," *Empiria. Revista de Metodología de Ciencias Sociales*, no. 16, pp. 85–112, oct 2008.

[30] A. Hoekstra, J. Kroc, and P. Sloot, "Introduction to modeling of complex systems using cellular automata," in *Simulating Complex Systems by Cellular Automata*, 05 2010, vol. 2010, pp. 1–16.

[31] J. Damerow. (2010) John von Neumann's cellular automata. Embryo Project Encyclopedia. Accedido el 23 de julio de 2025. [Online]. Available: https://embryo.asu.edu/pages/john-von-neumanns-cellular-automata

[32] E. W. Weisstein. (2002) Cellular automaton. MathWorld – A Wolfram Resource. Accedido el 23 de julio de 2025. [Online]. Available: https://mathworld.wolfram.com/CellularAutomaton.html

[33] S. Wolfram, "Cellular automata as simple self-organizing systems," California Institute of Technology, Technical Report CALT-68-938, 1982. [Online]. Available: https://content.wolfram.com/sw-publications/2020/07/cellular-automata-self-organizing-systems.pdf

[34] F. S. Caparrini. (2016) Autómatas celulares. Universidad de Sevilla. Accedido el 24 de septiembre de 2025. [Online]. Available: https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html

[35] K. Bhattacharjee, "Cellular automata: Reversibility, semi-reversibility and randomness," PhD Thesis, Indian Institute of Engineering Science and Technology, Shibpur, 2019, accedido el 23 de agosto de 2025. [Online]. Available: https://arxiv.org/abs/1911.03609

[36] H. Sayama. (2024) Definition of cellular automata. LibreTexts / Binghamton University. Accedido el 24 de septiembre de 2025. [Online]. Available: https://math.libretexts.org/Bookshelves/Scientific_Computing_Simulations_and_Modeling/Introduction_to_the_Modeling_and_Analysis_of_Complex_Systems_(Sayama)/11%3A_Cellular_Automata_I__Modeling/11.01%3A_DeïňĄnition_of_Cellular_Automata

[37] A. Ilachinski, *Cellular Automata: A Discrete Universe*. Singapore: World Scientific Publishing, 2001.

[38] J. von Neumann and A. W. Burks, *Theory of Self-Reproducing Automata*, A. W. Burks, Ed. Urbana, Illinois: University of Illinois Press, 1966. [Online]. Available: https://cba.mit.edu/events/03.11.ASE/docs/VonNeumann.pdf

[39] Y. W. Kwon, *Multiphysics and Multiscale Modeling: Techniques and Applications*, 1st ed. CRC Press, 2015.

[40] Conway's game of life. LifeWiki. Accedido el 24 de septiembre de 2025. [Online]. Available: https://conwaylife.com/wiki/Conway%27s_Game_of_Life

[41] S. Wolfram, *A New Kind of Science*. Champaign, Illinois: Wolfram Media, 2002.

[42] T. Tyler. (2018) Cellular automata - the margolus neighbourhood. Accedido el 24 de septiembre de 2025. [Online]. Available: https://cell-auto.com/neighbourhood/margolus

[43] O. Ureña Ponce, "Diseño de un procesador de autómatas celulares para la simulación de sistemas complejos," Master's thesis, Universidad de Guadalajara, Guadalajara, México, 2015. [Online]. Available: https://riudg.udg.mx/bitstream/20.500.12104/79915/1/MCUCEI01068FT.pdf

[44] G. J. Martinez, A. Adamatzky, and J. Seck-Tuoh-Mora, "Some notes about the game of life cellular automaton," in *Emergent Computation*. Springer, 2022, pp. 93–104. [Online]. Available: https://www.comunidad.escom.ipn.mx/genaro/Papers/Papers_on_CA_files/2022SomeNotesGoL.pdf

[45] E. W. Weisstein. (2002) Totalistic cellular automaton. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/TotalisticCellularAutomaton.html

[46] ——. (2003) von Neumann neighborhood. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/vonNeumannNeighborhood.html

[47] Wikipedia contributors. (2016) Cellular automaton — wikipedia, the free encyclopedia. Accedido el 12 de noviembre de 2025. [Online]. Available: https://en.wikipedia.org/wiki/Cellular_automaton

[48] M. Thoma. (2020, nov) John Conway's game of life - an introduction to cellular automata. Accedido el 11 de noviembre de 2025. [Online]. Available: https://beltoforion.de/en/game_of_life

[49] M. Sipper, "Paul Rendell: Turing machine universality of the Game of Life," *Genetic Programming and Evolvable Machines*, vol. 18, no. 1, pp. 115–117, 2017, accedido el 11 de noviembre de 2025. [Online]. Available: https://www.moshesipper.com/pubs/rendell_review.pdf

[50] P. Rendell. (2000) Turing machine in Conway's Game of Life. Rendell's Attic. Accedido el 29 de marzo de 2026. Imagen disponible en

http://rendell-attic.org/gol/turing_js_r.gif. [Online]. Available: http://rendell-attic.org/gol/tm.htm

[51] E. W. Weisstein. (2002) Rule 30. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule30.html

[52] ——. (2002) Rule 110. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule110.html

[53] ——. (2002) Rule 90. MathWorld – A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule90.html

[54] G. J. Martínez. (2000) Teoría del campo promedio en autómatas celulares similares a "the game of life". Accedido el 8 de septiembre de 2025. [Online]. Available: https://delta.cs.cinvestav.mx/~mcintosh/comun/tesismaestria/genaro/thesis.html

[55] G. J. Martinez. (2013) A note on elementary cellular automata classification. [Online]. Available: https://arxiv.org/abs/1306.5577

[56] K. C. II and S. Yu, "Undecidability of ca classification schemes," *Complex Systems*, vol. 2, pp. 177–190, 1988.

[57] D. Grattarola, L. Livi, and C. Alippi, "Learning graph cellular automata," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 20 983–20 994. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/af87f7cdcda223c41c3f3ef05a3aaeea-Paper.pdf

[58] P. Gács, "Probabilistic cellular automata with Andrei Toom," *Brazilian Journal of Probability and Statistics*, vol. 38, no. 2, 2024, accedido el 23 de julio de 2025. [Online]. Available: https://doi.org/10.1214/24-bjps605

[59] H. Hartman, P. Tamayo, and W. Klein, "Inhomogeneous cellular automata and statistical mechanics," *Complex Systems*, vol. 1, no. 2, 1987. [Online]. Available: https://content.wolfram.com/uploads/sites/13/2018/02/01-2-2.pdf

[60] L. Becker. (2023) Continuous cellular automata – HEGL. Heidelberg Experimental Geometry Lab. Accedido el 23 de julio de 2025. [Online]. Available: https://hegl.mathi.uni-heidelberg.de/continuous-cellular-automata

[61] D. J. Matich, *Redes Neuronales Artificiales: Conceptos Básicos y Aplicaciones*, Universidad Tecnológica Nacional (UTN), Facultad Regional Rosario, Argentina, 2001, consultado el 14 de abril de 2026. [Online]. Available: https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf

[62] C. Manning, "Artificial intelligence definitions," https://hai-production.s3.amazonaws.com/files/2020-09/AI-Definitions-HAI.pdf, 2020, consultado el 23 de septiembre de 2025.

[63] Google Cloud, "¿qué es la inteligencia artificial o ia?" 2025, consultado el 20 de marzo de 2025. [Online]. Available: https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419

[64] J. McCarthy, "What is artificial intelligence?" 2007, consultado el 20 de marzo de 2025. [Online]. Available: http://jmc.stanford.edu/articles/whatisai/whatisai.pdf

[65] Oracle, "¿qué es el machine learning?" 2024, consultado el 20 de marzo de 2025. [Online]. Available: https://www.oracle.com/mx/artificial-intelligence/machine-learning/what-is-machine-learning/

[66] Google Cloud. (s.f.) ¿qué es el aprendizaje automático? tipos y usos. Consultado el 07 de abril de 2025. [Online]. Available: https://cloud.google.com/learn/what-is-machine-learning?hl=es-419

[67] D. Bergmann. (s.f.) ¿qué es el aprendizaje automático? Consultado el 07 de abril de 2025. [Online]. Available: https://www.ibm.com/mx-es/think/topics/machine-learning

[68] A. Main, "What is ai training?" https://www.linkedin.com/pulse/what-ai-training-andy-main-2f4oe/, 2023, consultado el 07 de abril de 2025.

[69] Oracle. (2024) ¿qué es el big data? Consultado el 28 de abril de 2025. [Online]. Available: https://www.oracle.com/mx/big-data/what-is-big-data/

[70] Google Cloud. (s.f.) ¿qué es el aprendizaje no supervisado? Consultado el 07 de abril de 2025. [Online]. Available: https://cloud.google.com/discover/what-is-unsupervised-learning?hl=es-419

[71] Amazon Web Services. (s.f.) ¿qué es el aprendizaje mediante refuerzo? Consultado el 07 de abril de 2025. [Online]. Available: https://aws.amazon.com/es/what-is/reinforcement-learning/

[72] Adobe. (s.f.) Machine learning — definition, models, and applications. Consultado el 07 de abril de 2025. [Online]. Available: https://business.adobe.com/ar/products/real-time-customer-data-platform/what-is-machine-learning.html

[73] Clínica Universidad de Navarra, "Qué es la neurona," 2023, consultado el 17 de septiembre de 2025. [Online]. Available: https://www.cun.es/diccionario-medico/terminos/neurona

[74] P. Bordoli, "La neurona," *Repositorio Académico*, 2023, consultado el 17 de septiembre de 2025.

[75] brgfx, "Freepik graphic resource," http://www.freepik.com, 2025, consultado el 17 de septiembre de 2025.

[76] S. Mhatre. (2020) What is the relation between artificial and biological neuron? Consultado el 17 de septiembre de 2025. [Online]. Available: https://saurabhnativeblog.medium.com/what-is-the-relation-between-artificial-and-biological-neuron-18b05831036

[77] Y. Lao León, A. Rivas Méndez, M. Pérez Pravia, and F. Marrero Delgado, "Procedimiento para el pronóstico de la demanda mediante redes neuronales artificiales," *Ciencias Holguin*, vol. 23, pp. 43–59, 2017, consultado el 17 de septiembre de 2025.

[78] Google, "Neural networks: Activation functions," s.f., consultado el 17 de septiembre de 2025. [Online]. Available: https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions

[79] Amazon Web Services, "¿qué es una red neuronal?" s.f., consultado el 17 de septiembre de 2025. [Online]. Available: https://aws.amazon.com/es/what-is/neural-network

[80] Telefónica Tech, "Las matemáticas del machine learning: Redes neuronales (parte i)," 2019, consultado el 17 de septiembre de 2025. [Online]. Available: https://telefonicatech.com/blog/las-matematicas-del-machine-learning-redes-neuronales-parte-i

[81] IBM Corporation, *Neural networks - IBM SPSS Modeler*, IBM, 2021, consultado el 14 de abril de 2026. [Online]. Available: https://www.ibm.com/docs/es/spss-modeler/saas?topic=networks-neural-model

[82] IBM, "¿qué es la función de pérdida?" s.f., consultado el 24 de julio de 2025. [Online]. Available: https://www.ibm.com/mx-es/think/topics/loss-function

[83] Ultralytics, "Función de pérdida en ia/ml explicada," 2025, consultado el 24 de julio de 2025. [Online]. Available: https://www.ultralytics.com/es/glossary/loss-function

[84] A. Amidi and S. Amidi. (2025) Cs 230 - convolutional neural networks cheatsheet. Stanford University. Consultado el 13 de noviembre de 2025. [Online]. Available: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks

[85] ——. (2025) Cs 230 - recurrent neural networks cheatsheet. Stanford University. Consultado el 13 de noviembre de 2025. [Online]. Available: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

[86] W. Gilpin, "Cellular automata as convolutional neural networks," *Physical Review E*, vol. 100, no. 3, Sep. 2019, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1103/PhysRevE.100.032402

[87] E. Niklasson, A. Mordvintsev, and E. Randazzo, "Asynchronicity in neural cellular automata," in *ALIFE 2021: The 2021 Conference on Artificial Life*, Jul. 2021, p. 116. [Online]. Available: https://doi.org/10.1162/isal_a_00461

[88] A. Mordvintsev, E. Randazzo, and C. Fouts, "Growing isotropic neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2205.01681

[89] A. Mordvintsev and E. Niklasson, "Emergent multiscale structures and generative potential of isotropic neural cellular automata," in *ALIFE 2024: Proceedings of the 2024 Artificial Life Conference*, Jul. 2024, p. 28. [Online]. Available: https://doi.org/10.1162/isal_a_00745

[90] E. Randazzo, A. Mordvintsev, and C. Fouts, "Growing steerable neural cellular automata," 2023. [Online]. Available: https://arxiv.org/abs/2302.10197

[91] E. Randazzo, A. Mordvintsev, E. Niklasson, M. Levin, and S. Greydanus, "Self-classifying MNIST digits," *Distill*, 2020, accedido el 29 de julio de 2025. [Online]. Available: https://distill.pub/2020/selforg/mnist

[92] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, "Adversarial reprogramming of neural cellular automata," *Distill*, 2021, accedido el 29 de julio de 2025. [Online]. Available: https://distill.pub/selforg/2021/adversarial

[93] D. Zhang, C. Choi, J. Kim, and Y. M. Kim, "Learning to generate 3D shapes with generative cellular automata," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://openreview.net/forum?id=rABUmU3ulQh

[94] S. Sudhakaran, D. Grbic, S. Li, A. Katona, E. Najarro, C. Glanois, and S. Risi, "Growing 3D artefacts and functional machines with neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2103.08737

[95] K. Horibe, K. Walker, and S. Risi, "Regenerating soft robots through neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2102.02579

[96] E. Niklasson, A. Mordvintsev, E. Randazzo, and M. Levin, "Self-organising textures," *Distill*, 2021, accedido el 29 de julio de 2025. [Online]. Available: https://distill.pub/selforg/2021/textures

[97] A. Mordvintsev, E. Niklasson, and E. Randazzo, "Texture generation with neural cellular automata," *CoRR*, vol. abs/2105.07299, 2021. [Online]. Available: https://arxiv.org/abs/2105.07299

[98] A. Mordvintsev and E. Niklasson, "μNCA: Texture generation with ultra-compact neural cellular automata," *CoRR*, vol. abs/2111.13545, 2021. [Online]. Available: https://arxiv.org/abs/2111.13545

[99] E. Pajouheshgar, Y. Xu, A. Mordvintsev, E. Niklasson, T. Zhang, and S. Süsstrunk, "Mesh neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2311.02820

[100] E. Pajouheshgar, Y. Xu, A. Abbasi, A. Mordvintsev, W. Jakob, and S. Süsstrunk, "Neural cellular automata: From cells to pixels," 2025. [Online]. Available: https://arxiv.org/abs/2506.22899

[101] A. Hernandez Ruiz, A. Vilalta, and F. Moreno-Noguer, "Neural cellular automata manifold," 2021. [Online]. Available: https://arxiv.org/abs/2006.12155

[102] K. Frans, "StampCA: Growing emoji with conditional neural cellular automata," *kvfrans.com*, 2021, accedido el 29 de julio de 2025. [Online]. Available: http://kvfrans.com/stampca-conditional-neural-cellular-automata/

[103] M. Chen and Z. Wang, "Image generation with neural cellular automata," 2020. [Online]. Available: https://arxiv.org/abs/2010.04949

[104] R. B. Palm, M. González-Duque, S. Sudhakaran, and S. Risi, "Variational neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2201.12360

[105] M. Tesfaldet, D. Nowrouzezahrai, and C. Pal, "Attention-based neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2211.01233

[106] J. Tang, S. Kumar, L. De Lorenzis, and E. Hosseini, "Neural cellular automata for solidification microstructure modelling," *Computer Methods in Applied Mechanics and Engineering*, vol. 414, p. 116197, Sep. 2023. [Online]. Available: http://dx.doi.org/10.1016/j.cma.2023.116197

[107] G. H. Y. Li, C. R. Leefmans, J. Williams *et al.*, "Deep learning with photonic neural cellular automata," *Light: Science & Applications*, vol. 13, no. 283, 2024, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1038/s41377-024-01651-7

[108] C. Grasso and J. Bongard, "Empowered neural cellular automata," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '22)*. New York, NY, USA: ACM, 2022, pp. 108–111, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1145/3520304.3529067

[109] A. D. Richardson, T. Antal, R. A. Blythe, and L. J. Schumacher, "Learning spatio-temporal patterns with neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2310.14809

[110] A. E. Azuero Azuero, "Significatividad del marco metodológico en el desarrollo de proyectos de investigación," *Revista Arbitrada Interdisciplinaria Koinonía*, vol. 4, no. 8, pp. 110–127, julio 2019, accedido el 21 de abril de 2025.

[111] UNAM, "Los diseños de investigación en la psicología," https://enlinea.iztacala.unam.mx/resources/modules/UAPAS/Los%20disenos%20de%20investigacion%20en%20la%20psicologia/, 2025, accedido el 23 de abril de 2025.

[112] B. Capili and J. K. Anastasi, "An introduction to types of quasi-experimental designs," *American Journal of Nursing*, vol. 124, no. 11, pp. 50–52, noviembre 2024, accedido el 24 de octubre de 2024.

[113] Universidad de Jaén, "Metodología cuantitativa," https://web.ujaen.es/investiga/tics_tfg/enfo_cuanti.html, n.d., accedido el 11 de junio de 2025.

[114] T. Cueva Luza, O. Jara Córdova, J. Arias Gonzáles, F. A. Flores Limo, and C. A. Balmaceda Flores, *Métodos mixtos de investigación para principiantes*, primera ed. Puno, Perú: Instituto Universitario de Innovación Ciencia y Tecnología Inudi Perú S.A.C., 2023, accedido el 23 de abril de 2025.

[115] J. Arias-Gómez, M. A. Villasís-Keever, and M. G. Miranda-Novales, "El protocolo de investigación iii: la población de estudio," *Revista de Alergia de México*, vol. 63, no. 2, pp. 201–206, mayo 2016, accedido el 23 de abril de 2025. [Online]. Available: https://doi.org/10.29262/ram.v63i2.181

[116] M. E. Díaz Hernández, "Población, muestra y muestreo," https://www.uaeh.edu.mx/docencia/P_Presentaciones/huejutla/enfermeria/2017/Poblacion_Muestra_Muestreo.pdf, 2017, accedido el 23 de abril de 2025.

[117] *Emerging Trends in Applications and Infrastructures for Computational Biology, Bioinformatics, and Systems Biology*. Elsevier, 2016, accedido el 13 de junio de 2025. [Online]. Available: https://doi.org/10.1016/c2015-0-01779-8

[118] Universidad Veracruzana, "Introducción a la investigación: guía interactiva — tipos de investigación," https://www.uv.mx/apps/bdh/investigacion/unidad1/investigacion-tipos.html, n.d., accedido el 12 de junio de 2025.

[119] QuestionPro Team, "Estadística descriptiva: Qué es y cómo utilizarla," https://www.questionpro.com/blog/es/estadistica-descriptiva/, 2023, accedido el 15 de mayo de 2025.

[120] UAPA-UNAM, "Lenguajes de programación," https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/2655/mod_resource/content/1/UAPA-Lenguajes-Programacion/index.html, 2025, accedido el 11 de junio de 2025.

[121] ClickPanda. (2023) La importancia de seleccionar el entorno y el lenguaje de base de datos correcto. Accedido el 11 de junio de 2025. [Online]. Available: https://www.linkedin.com/pulse/la-importancia-de-seleccionar-el-entorno-y-lenguaje-base-2pnme/

[122] Universidad de Valladolid, "El lenguaje c++ — fundamentos de programación en c++," https://www2.eii.uva.es/fund_inf/cpp/temas/1_introduccion/introduccion.html, n.d., accedido el 11 de junio de 2025.

[123] MSMK University, "¿qué es c++ en programación?" https://msmk.university/que-es-c-en-programacion, 2025, accedido el 21 de mayo de 2025.

[124] Pybind11 Developers, "pybind11 — seamless operability between c++11 and python," https://pybind11.readthedocs.io/en/stable/index.html, n.d., accedido el 11 de junio de 2025.

[125] Microsoft Learn. (2025) iostream (c++). Accedido el 11 de noviembre de 2025. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/iostream

[126] ——. (2025) algorithm (c++). Accedido el 11 de noviembre de 2025. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/algorithm

[127] ——. (2024) cstring (string.h). Accedido el 12 de mayo de 2026. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/cstring

[128] Supermicro, "¿qué es cuda?" https://www.supermicro.com/es/glossary/cuda, n.d., accedido el 12 de mayo de 2026.

[129] NVIDIA Corporation. (2024) Cuda runtime api reference guide. Accedido el 12 de mayo de 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/index.html

[130] Python Software Foundation. (2025) El tutorial de python. Accedido el 11 de junio de 2025. [Online]. Available: https://docs.python.org/es/3.13/tutorial/index.html

[131] Amazon Web Services, Inc. (2025) ¿qué es python? - explicación del lenguaje python. Accedido el 11 de junio de 2025. [Online]. Available: https://aws.amazon.com/es/what-is/python/

[132] A. Alfaro. (2025) La librería numpy. Accedido el 11 de junio de 2025. [Online]. Available: https://aprendeconalf.es/docencia/python/manual/numpy/

[133] PyTorch, "Pytorch documentation," https://docs.pytorch.org/docs/stable/index.html, n.d., accedido el 11 de junio de 2025.

[134] D. Bergmann and C. Stryker, "¿qué es pytorch?" https://www.ibm.com/mx-es/think/topics/pytorch, n.d., accedido el 11 de junio de 2025.

[135] Riverbank Computing Limited, "Pyqt5 — comprehensive python bindings for qt v5," https://pypi.org/project/PyQt5/, 2024, accedido el 19 de julio de 2024.

[136] M. Fitzpatrick, "Pyqt6 vs pyside6: What's the difference?" https://www.pythonguis.com/faq/pyqt6-vs-pyside6, n.d., accedido el 12 de junio de 2025.

[137] A. Alférez. (2025) La librería matplotlib. Accedido el 12 de junio de 2025. [Online]. Available: https://aprendeconalf.es/docencia/python/manual/matplotlib/

[138] Master Data Scientist. (2025) Scikit-learn, herramienta básica para el data science en python. Accedido el 11 de junio de 2025. [Online]. Available: https://www.master-data-scientist.com/scikit-learn-data-science/

[139] Wolfram Research, Inc., *Mathematica, Version 14.1*, Champaign, IL, 2024, accedido el 12 de junio de 2025.

[140] G. Cimas Cuadrado. (2024) Visual studio code: Editor de código para desarrolladores. Accedido el 12 de junio de 2025. [Online]. Available: https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/

[141] F. García de Zúñiga. (2024) ¿qué es visual studio code y cuáles son sus ventajas? Accedido el 12 de junio de 2025. [Online]. Available: https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas

[142] GitHub. (2025) Acerca de github y git. Accedido el 12 de junio de 2025. [Online]. Available: https://docs.github.com/es/get-started/start-your-journey/about-github-and-git

[143] Canva. (2025) Acerca de canva. Accedido el 12 de junio de 2025. [Online]. Available: https://www.canva.com/es_es/about/

[144] B. van der Zander, "Texstudio - a latex editor," https://www.texstudio.org/#about, n.d., accedido el 12 de junio de 2025.

[145] JGraph Ltd., "draw.io (diagrams.net)," https://www.drawio.com, 2024, accedido el 12 de junio de 2025.

[146] Pymes y Calidad 2.0, "¿cómo se hace una matriz de riesgo? - ventajas y desventajas," https://www.pymesycalidad20.com/como-se-hace-una-matriz-de-riesgo-ventajas-y-desventajas.html, oct 2025, accedido el 28 de octubre de 2025.

[147] MDN Web Docs, "Mvc - glosario de mdn web docs," https://developer.mozilla.org/es/docs/Glossary/MVC, oct 2025, accedido el 28 de octubre de 2025.

[148] Microsoft Support, "Crear un diagrama de clase uml - soporte técnico de microsoft," https://support.microsoft.com/es-es/topic/crear-un-diagrama-de-clase-uml-de6be927-8a7b-4a79-ae63-90da8f1a8a6b, nov 2025, accedido el 11 de noviembre de 2025.